

UNIVERSIDAD MIGUEL HERNÁNDEZ DE ELCHE  
MÁSTER UNIVERSITARIO EN ROBÓTICA



REDES NEURONALES CONVOLUCIONALES PARA EL  
ANÁLISIS DE SEÑALES EEG DURANTE LA MARCHA CON  
UN EXOESQUELETO

TRABAJO FIN DE MÁSTER

Junio - 2026

AUTOR: Daniel Rosique Egea

DIRECTOR: Eduardo Iañez Martinez



*“The human brain, sculpted by millions of years of evolution, is the sole architect  
of the 'human universe'”*

- MIGUEL NICOLELIS



# AGRADECIMIENTOS

Agradezco a mi tutor del proyecto, Eduardo Iáñez, por su paciencia en el desarrollo de este trabajo y por la ayuda que me ha proporcionado durante todo el proceso. También por permitirme acercarme a la neuro-robótica y al análisis de señales EEG.

Gracias también a mi pareja por apoyarme y motivarme en los momentos en los que me he encontrado perdido en el proyecto.

Gracias a mi familia y a mis compañeros de trabajo por recordarme la importancia de este trabajo y por ayudarme a mantenerme motivado durante el proceso de desarrollo.

Finalmente, gracias a todos los que han estado conmigo durante este año con tantos cambios y dificultades.



## RESUMEN

Las interfaces cerebro-computador dedicadas al control de exoesqueletos constituyen una herramienta prometedora para la rehabilitación de personas con lesiones motoras. En este contexto, la detección precisa de la intención de marcha del usuario a partir de señales electroencefalográficas (EEG) resulta fundamental para garantizar un funcionamiento seguro y eficiente del sistema. Así pues, el desarrollo de modelos de clasificación robustos y generalizables puede ayudar a reducir la dependencia de calibraciones extensas y mejorar la experiencia del usuario, facilitando la integración de estas tecnologías en entornos clínicos y domésticos.

El objetivo de este trabajo es analizar el rendimiento de diferentes arquitecturas de redes neuronales convolucionales en la clasificación de señales EEG, así como evaluar distintas estrategias de entrenamiento, incluyendo modelos entrenados con datos del mismo día, enfoques acumulativos y técnicas de fine-tuning. Entre las arquitecturas evaluadas se incluyen *EEGNet*, *DeepConvNet* y *ShallowConvNet*, mientras que las estrategias de entrenamiento consideradas abarcan enfoques *Intra-Sesión*, *Inter-Sesión*, *Inter-Usuarios* y *Continuo*.

Los experimentos se han realizado utilizando registros de señales EEG correspondientes a cinco usuarios y cinco sesiones por participante. Las tareas objetivo han sido la detección de la intención de caminar y detenerse. Las diferentes configuraciones se han entrenado con varias semillas aleatorias y se han evaluado mediante la precisión media y su desviación estándar, permitiendo estudiar tanto el rendimiento como la estabilidad de los modelos. De esta manera, se ha analizado la influencia de la arquitectura, la tarea, el método de entrenamiento y la variabilidad entre usuarios.

Los resultados muestran que *ShallowConvNet* obtiene el mejor rendimiento global, con una precisión media aproximada del 76.6 %, seguida de *EEGNet*, con un 71.4 %, y *DeepConvNet*, con un 69.4 %. La tarea estática alcanza una precisión media cercana al 76.0 %, frente al 69.1 % obtenido en la tarea de movimiento. En cuanto a las estrategias de entrenamiento, el método *Inter-Sesión* presenta el mejor resultado medio, aunque las diferencias entre métodos son moderadas y dependen en gran

medida de la arquitectura, la tarea y el participante.

En conjunto, los resultados indican que la arquitectura seleccionada y la variabilidad entre usuarios tienen una mayor influencia sobre el rendimiento que la estrategia de entrenamiento. Este trabajo proporciona una base para el desarrollo de pruebas más exhaustivas orientadas a reducir el proceso de calibración de los sistemas de asistencia robótica y facilitar su integración en aplicaciones de rehabilitación.





# ÍNDICE

|                                                                            |             |
|----------------------------------------------------------------------------|-------------|
| <b>AGRADECIMIENTOS</b>                                                     | <b>III</b>  |
| <b>RESUMEN</b>                                                             | <b>V</b>    |
| <b>ÍNDICE DE TABLAS</b>                                                    | <b>XIII</b> |
| <b>ÍNDICE DE FIGURAS</b>                                                   | <b>XVI</b>  |
| <b>1. Introducción</b>                                                     | <b>1</b>    |
| 1.1. Definición del problema . . . . .                                     | 2           |
| 1.2. Objetivos a Conseguir . . . . .                                       | 2           |
| <b>2. Estado del Arte</b>                                                  | <b>4</b>    |
| 2.1. Lesiones y patologías del sistema nervioso central . . . . .          | 4           |
| 2.2. Electroencefalografía (EEG) . . . . .                                 | 6           |
| 2.3. Interfaces Cerebro-Computador (BCI) . . . . .                         | 9           |
| 2.4. Métodos tradicionales de extracción de características en señales EEG | 12          |
| 2.4.1. Características temporales, frecuenciales y tiempo-frecuencia .     | 12          |
| 2.4.2. Características espaciales: CSP y FBCSP . . . . .                   | 13          |
| 2.5. Redes neuronales convolucionales . . . . .                            | 15          |
| 2.6. Aplicaciones . . . . .                                                | 17          |
| 2.6.1. Comparación de modelos en la literatura . . . . .                   | 17          |

|                                                            |           |
|------------------------------------------------------------|-----------|
| 2.6.2. Distintos entrenamientos en la literatura . . . . . | 19        |
| <b>3. Material y Métodos</b>                               | <b>22</b> |
| 3.1. Adquisición de datos . . . . .                        | 22        |
| 3.1.1. Equipamiento experimental . . . . .                 | 22        |
| 3.1.2. Protocolo experimental . . . . .                    | 23        |
| 3.1.3. Estructura y cuantificación del dataset . . . . .   | 24        |
| 3.1.4. Preprocesamiento . . . . .                          | 26        |
| 3.2. Materiales para el estudio de los datos . . . . .     | 27        |
| 3.2.1. Hardware . . . . .                                  | 27        |
| 3.2.2. Software . . . . .                                  | 28        |
| 3.3. Modelos utilizados . . . . .                          | 29        |
| 3.3.1. DeepConvNet y ShallowConvNet . . . . .              | 29        |
| 3.3.2. EEGNet . . . . .                                    | 32        |
| 3.4. Metodología experimental . . . . .                    | 33        |
| 3.4.1. Variables del estudio . . . . .                     | 33        |
| 3.4.1.1. Resumen estrategias de entrenamiento . . . . .    | 41        |
| 3.5. Infraestructura Experimental . . . . .                | 43        |
| 3.5.1. Parámetros de entrenamiento . . . . .               | 46        |
| <b>4. Resultados y Discusión</b>                           | <b>49</b> |
| 4.1. Tarea Estática . . . . .                              | 51        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| 4.1.1.    | Comparación Global . . . . .                           | 51        |
| 4.1.2.    | Variabilidad Entre Usuarios . . . . .                  | 52        |
| 4.1.3.    | Mejor Configuración . . . . .                          | 53        |
| 4.2.      | Tarea Movimiento . . . . .                             | 54        |
| 4.2.1.    | Comparación de Global . . . . .                        | 54        |
| 4.2.2.    | Variabilidad Entre Usuarios . . . . .                  | 55        |
| 4.2.3.    | Mejor Configuración . . . . .                          | 55        |
| 4.3.      | Comparación entre usuarios . . . . .                   | 57        |
| 4.4.      | Comparación entre tareas . . . . .                     | 60        |
| 4.4.1.    | Rendimiento global de las tareas . . . . .             | 61        |
| 4.5.      | Comparación entre métodos de entrenamiento . . . . .   | 62        |
| <b>5.</b> | <b>Conclusiones</b>                                    | <b>65</b> |
| 5.1.      | Influencia de la arquitectura . . . . .                | 65        |
| 5.2.      | Influencia de la tarea . . . . .                       | 66        |
| 5.3.      | Variabilidad entre usuarios . . . . .                  | 67        |
| 5.4.      | Influencia de los métodos de entrenamiento . . . . .   | 68        |
| 5.5.      | Conclusión general . . . . .                           | 69        |
| 5.6.      | Trabajos futuros . . . . .                             | 70        |
| 5.6.1.    | Maquina de estados finitos . . . . .                   | 70        |
| 5.6.2.    | Interpretabilidad de los modelos utilizados . . . . .  | 71        |
| 5.6.3.    | Evaluación de la cantidad de datos necesaria . . . . . | 72        |

|                     |           |
|---------------------|-----------|
| <b>BIBLIOGRAFÍA</b> | <b>74</b> |
|---------------------|-----------|

|               |           |
|---------------|-----------|
| <b>ANEXOS</b> | <b>88</b> |
|---------------|-----------|

|      |                                                                   |    |
|------|-------------------------------------------------------------------|----|
| A.   | Esquema de la base de datos . . . . .                             | 88 |
| B.   | Tablas complementarias de resultados . . . . .                    | 90 |
| B.1. | Tasa de falsos positivos y falsos negativos . . . . .             | 90 |
| B.2. | Precisión media y desviación estandar de cada usuario . . . . .   | 90 |
| B.3. | Precisión media separada por tarea, método y modelo . . . . .     | 91 |
| B.4. | Evolución de la precisión por días separada por modelos . . . . . | 91 |
| C.   | Queries complementarias . . . . .                                 | 95 |
| C.1. | Filtrado de Outliers . . . . .                                    | 95 |



## ÍNDICE DE TABLAS

|                                                                                                                                                                                                                                                                     |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1. Arquitectura DeepConvNet utilizada. . . . .                                                                                                                                                                                                                    | 30 |
| 3.2. Arquitectura ShallowConvNet utilizada. . . . .                                                                                                                                                                                                                 | 31 |
| 3.3. Resumen de las estrategias de entrenamiento empleadas. . . . .                                                                                                                                                                                                 | 41 |
| 3.4. Parámetros de entrenamiento utilizados en el estudio. . . . .                                                                                                                                                                                                  | 47 |
| 4.1. Número de registros antes y después del proceso de filtrado. . . . .                                                                                                                                                                                           | 50 |
| A.1. Esquema de la base de datos relacional utilizada para almacenar los<br>resultados de los experimentos realizados. . . . .                                                                                                                                      | 89 |
| B.2. Tasas de falsos positivos y falsos negativos para cada combinación de<br>modelo, método de entrenamiento y tarea objetivo. Los valores más<br>altos se muestran en tonos rojizos, mientras que los valores más bajos<br>se muestran en tonos verdosos. . . . . | 90 |
| B.3. Precisión media y desviación estándar por usuario y tarea. . . . .                                                                                                                                                                                             | 91 |

## ÍNDICE DE FIGURAS

|                                                                                                                                                                        |    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. Esquema de las lesiones medulares en función del area de lesión [9] . .                                                                                           | 5  |
| 2.2. Esquema general del funcionamiento de una BCI [29] . . . . .                                                                                                      | 11 |
| 2.3. Esquema general de una red neuronal convolucional adaptada a la<br>clasificación de intención de movimiento a partir de señales EEG (Elaboración propia). . . . . | 16 |
| 2.4. Esquema general de una red neuronal convolucional para la tarea de<br>clasificación binaria en EEG (Elaboración propia). . . . .                                  | 16 |
| 3.1. Estructura temporal de las ventanas EEG utilizadas durante cada<br>ensayo. Adaptado de [66]. . . . .                                                              | 24 |
| 3.2. Estructura jerárquica de los tensores de entrada y etiquetas utilizados<br>en el modelo. . . . .                                                                  | 26 |
| 3.3. Arquitectura de EEGNet. las líneas muestran la conexión entre las<br>capas [52]. . . . .                                                                          | 33 |
| 3.4. Esquema simplificado del método de entrenamiento intra sesión (Elaboración propia). . . . .                                                                       | 36 |
| 3.5. Esquema simplificado del método de entrenamiento entre sesiones<br>(Elaboración propia). . . . .                                                                  | 38 |
| 3.6. Esquema simplificado del método de entrenamiento entre usuarios<br>(Elaboración propia). . . . .                                                                  | 40 |
| 3.7. Esquema simplificado del método de entrenamiento continuo (Elaboración propia). . . . .                                                                           | 42 |
| 3.8. Pipeline experimental . . . . .                                                                                                                                   | 45 |



|                                                                                                                                                                   |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1. Comparación general de las distintas arquitecturas y métodos de entrenamiento en la tarea estática (media obtenida a partir de los 5 usuarios). . . . .      | 51 |
| 4.2. Variabilidad entre usuarios en la tarea estática (media obtenida a partir de los métodos de entrenamiento). . . . .                                          | 52 |
| 4.3. Mejores configuraciones globales en la tarea estática (media obtenida a partir de los 5 usuarios). . . . .                                                   | 53 |
| 4.4. Comparación general de las distintas arquitecturas y métodos de entrenamiento en la tarea de movimiento (media obtenida a partir de los 5 usuarios). . . . . | 54 |
| 4.5. Variabilidad entre usuarios en la tarea de movimiento (media obtenida a partir de los métodos de entrenamiento). . . . .                                     | 56 |
| 4.6. Mejores configuraciones globales en la tarea de movimiento (media obtenida a partir de los 5 usuarios). . . . .                                              | 56 |
| 4.7. Comparación entre arquitecturas y entrenamientos para el usuario 0 en la tarea estática y en la de movimiento. . . . .                                       | 57 |
| 4.8. Comparación entre arquitecturas y entrenamientos para el usuario 1 en la tarea estática y en la de movimiento. . . . .                                       | 58 |
| 4.9. Comparación entre arquitecturas y entrenamientos para el usuario 2 en la tarea estática y en la de movimiento. . . . .                                       | 59 |
| 4.10. Comparación entre arquitecturas y entrenamientos para el usuario 3 en la tarea estática y en la de movimiento. . . . .                                      | 59 |
| 4.11. Comparación entre arquitecturas y entrenamientos para el usuario 4 en la tarea estática y en la de movimiento. . . . .                                      | 60 |
| 4.12. Variabilidad entre usuarios y día para cada tarea (media obtenida a partir de los métodos de entrenamiento y los modelos). . . . .                          | 61 |

|                                                                                                                                      |    |
|--------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.13. Comparación de la precisión entre la tarea estática y la de movimiento<br>(media obtenida a partir de los 5 usuarios). . . . . | 62 |
| 4.14. Variabilidad entre usuarios y día para cada método de entrenamiento<br>(media obtenida a partir de los modelos). . . . .       | 63 |
| 5.1. Ejemplo de arquitectura de control basada en una máquina de estados<br>finitos para sistemas BCI. Tomado de [64] . . . . .      | 71 |
| B.2. Variabilidad entre usuarios y día para cada método de entrenamiento<br>para DeepConvNet. . . . .                                | 92 |
| B.3. Variabilidad entre usuarios y día para cada método de entrenamiento<br>para ShallowConvNet. . . . .                             | 93 |
| B.4. Variabilidad entre usuarios y día para cada método de entrenamiento<br>para EEGNet. . . . .                                     | 94 |



## 1. Introducción

Según la Organización Mundial de la Salud (OMS), las lesiones medulares afectan a más de 15 millones de personas a nivel mundial. Estas pueden ser ocasionadas tanto por traumatismos como por causas no traumáticas, incluyendo tumores, enfermedades degenerativas y vasculares, infecciones o defectos congénitos. Como consecuencia, producen una pérdida completa o parcial de las funciones sensoriales y/o motoras por debajo del nivel de la lesión, siendo una de las principales causas de discapacidad de larga duración [1].

Las limitaciones en la movilidad derivadas de estas lesiones dificultan la participación plena en la sociedad. En adultos, esto se traduce en tasas de desempleo superiores al 60 %, mientras que en población infantil se observa una menor continuidad en el sistema educativo [1], [2].

La necesidad de abordar los problemas causados por estas lesiones ha impulsado en los últimos años el desarrollo de nuevas terapias y dispositivos de rehabilitación [3].

En este contexto, las Interfaces Cerebro-Computador (BCIs) han emergido como una tecnología prometedora para la asistencia y rehabilitación de pacientes con discapacidad motora, al permitir la comunicación directa entre la actividad cerebral y dispositivos externos. Estas interfaces emplean principalmente señales electroencefalográficas (EEG) para identificar patrones neuronales asociados a la intención de movimiento [4]. Su aplicación en sistemas de asistencia, como exoesqueletos robóticos, abre nuevas posibilidades para mejorar la autonomía y calidad de vida de personas con lesiones medulares.

Sin embargo, la interpretación de las señales EEG presenta importantes desafíos debido a su baja relación señal-ruido y a la elevada variabilidad entre sujetos y sesiones, lo que dificulta el desarrollo de modelos robustos y generalizables [5].

### 1.1. Definición del problema

Este trabajo aborda la clasificación de señales EEG registradas durante la imaginación activa de tareas de marcha, con el objetivo de detectar patrones cerebrales relacionados con la intención de caminar o detenerse.

El problema se enmarca en el contexto de las BCI orientadas a aplicaciones de rehabilitación, donde resulta fundamental identificar de forma fiable los estados mentales del usuario a partir de señales cerebrales no invasivas.

Para ello, se pretende evaluar la capacidad de diferentes arquitecturas de redes neuronales para extraer características discriminativas a partir de datos EEG pre-procesados, analizando su rendimiento bajo distintos esquemas de entrenamiento y validación.

Además, se estudia la capacidad de generalización de los modelos, evaluando su robustez frente a la variabilidad entre los sujetos y las diferentes sesiones de registro.

### 1.2. Objetivos a Conseguir

El objetivo principal de este trabajo es evaluar el rendimiento de distintas arquitecturas de redes neuronales en la clasificación de la intención de marcha del usuario a partir de señales EEG.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

1. Entrenar y evaluar diferentes arquitecturas de redes neuronales

- EEGNet
- DeepConvNet
- ShallowConvNet

2. Evaluar diferentes métodos de entrenamiento

- Intra-session

- Cross-session
  - Cross-subject
  - Aprendizaje continuo
3. Analizar variabilidad entre usuarios y su efecto en el rendimiento de los modelos
  4. Evaluar el comportamiento de los modelos en diferentes escenarios de validación, incluyendo validación cruzada y entornos pseudo-online.

## 2. Estado del Arte

Este capítulo trata de introducir los fundamentos que sustentan este trabajo, al igual que se ahonda en el estado actual de las técnicas de Interfaz Cerebro-Computador (BCI) y Electroencefalografía (EEG) y sus aplicaciones, para de esta manera poder entender el problema que se pretende resolver y la solución que se pretende dar.

### 2.1. Lesiones y patologías del sistema nervioso central

El Sistema Nervioso Central (SNC), encargado de la planificación, iniciación y modulación del movimiento, comprende estructuras como la corteza cerebral, el tronco encefálico, el cerebelo y la médula espinal. A través de estas estructuras, el SNC procesa la información sensorial y genera impulsos nerviosos que son transmitidos a los músculos para producir movimiento [6].

Las lesiones o patologías que afectan al SNC pueden dañar tanto las neuronas como las vías de comunicación entre ellas, alterando la transmisión de señales motoras y sensitivas. Como consecuencia, pueden aparecer deficiencias motoras, pérdida de sensibilidad, alteraciones del equilibrio o dificultades en la coordinación del movimiento [6].

Entre las lesiones más relevantes se encuentran las Lesiones de la Médula Espinal (LME), producidas por causas traumáticas o no traumáticas que dañan parcial o totalmente la médula espinal. Este daño interrumpe la transmisión de señales nerviosas entre el cerebro y las regiones corporales situadas por debajo de la lesión. Los síntomas dependen tanto de la localización como de la gravedad del daño. Las lesiones cervicales pueden provocar tetraplegia, mientras que lesiones en regiones más bajas suelen derivar en paraplegia [7], [8].

En la figura 2.1 se puede ver un esquema de las lesiones medulares en función del área de lesión.

Otra de las principales patologías del SNC son los accidentes cerebrovasculares o

|                  |  | Nivel de la lesión                                                                                                                      | Efecto*                                                                                                                                                                                                                                                           |  |
|------------------|--|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| C<br>T<br>L<br>S |  | C<br>E<br>R<br>V<br>I<br>C<br>A<br>L                                                                                                    | Entre C2 y C5<br>Parálisis de algunos o todos los músculos utilizados para respirar y todos los músculos de brazos y piernas. En general mortal, a menos que se use un ventilador.                                                                                |  |
|                  |  | Entre C5 y C6<br>Parálisis de las piernas, tronco, mano y muñeca.<br>Debilidad de los músculos que mueven el hombro y el codo.          |                                                                                                                                                                                                                                                                   |  |
|                  |  | Entre C6 y C7<br>Parálisis de las piernas, el tronco y parte de las muñecas y manos.<br>Movimiento normal de hombros y codos.           |                                                                                                                                                                                                                                                                   |  |
|                  |  | Entre C7 y C8<br>Parálisis de las piernas, el tronco y las manos.                                                                       |                                                                                                                                                                                                                                                                   |  |
|                  |  | T<br>O<br>R<br>Á<br>C<br>I<br>C<br>A                                                                                                    | C8 a T1<br>Parálisis de piernas y tronco.<br>Debilidad de los músculos que mueven dedos y manos.<br>Síndrome de Horner (con párpado caído, pupila constreñida y sudoración reducida en un lado de la cara).<br>Posiblemente movimiento normal de hombros y codos. |  |
|                  |  | T2 a T4<br>Parálisis de piernas y tronco.<br>Pérdida de la sensibilidad debajo de los pezones.<br>Movimiento normal de hombros y codos. |                                                                                                                                                                                                                                                                   |  |
|                  |  | T5 a T8<br>Parálisis de las piernas y la parte inferior del tronco.<br>Pérdida de la sensibilidad debajo de la caja torácica.           |                                                                                                                                                                                                                                                                   |  |
|                  |  | T9 a T11<br>Parálisis de las piernas.<br>Pérdida de la sensibilidad debajo del ombligo.                                                 |                                                                                                                                                                                                                                                                   |  |
|                  |  | T11 a L1<br>Parálisis y pérdida de sensibilidad en caderas y piernas.                                                                   |                                                                                                                                                                                                                                                                   |  |
|                  |  | L<br>U<br>M<br>B<br>A<br>R                                                                                                              | L2 a S2<br>Diversos patrones de debilidad y entumecimiento, según el nivel concreto de la lesión.                                                                                                                                                                 |  |
|                  |  | S<br>A<br>C<br>R<br>O                                                                                                                   | S3 a S5<br>Entumecimiento en el perineo.                                                                                                                                                                                                                          |  |
|                  |  | *Las lesiones graves pueden causar la pérdida del control de la vejiga y el intestino en cualquier nivel de la médula espinal.          |                                                                                                                                                                                                                                                                   |  |

\*Las lesiones graves pueden causar la pérdida del control de la vejiga y el intestino en cualquier nivel de la médula espinal.

Figura 2.1: Esquema de las lesiones medulares en función del area de lesión [9]



ictus. Estos pueden clasificarse en hemorrágicos, cuando se produce la rotura de un vaso sanguíneo cerebral, o isquémicos, cuando una obstrucción reduce o bloquea el flujo sanguíneo. En ambos casos se produce daño neuronal debido a la falta de oxígeno y nutrientes en el tejido cerebral afectado [10]. Los ictus representan una de las principales causas de discapacidad motora adquirida y mortalidad a nivel mundial [6], [10], [11].

A pesar de la gravedad de estas lesiones, existe un potencial de rehabilitación en la neuroplasticidad. La neuroplasticidad es la capacidad del SNC para modificar su estructura y funcionamiento, así como crear nuevas conexiones neuronales en respuesta a estímulos externos o internos [6], [12]. La rehabilitación neurológica busca potenciar esta reorganización funcional de las áreas cerebrales no afectadas [13], creando nuevas conexiones mediante la repetición de tareas.

La rehabilitación constituye un proceso complejo que requiere un enfoque multidisciplinar y tratamientos personalizados. Aunque parte de la recuperación motora suele producirse durante los primeros meses tras la lesión, muchos pacientes continúan presentando secuelas permanentes en fases crónicas [11]. En estos casos, las tecnologías de asistencia, incluyendo los sistemas basados en interfaces cerebro-computadora (BCI), representan una herramienta prometedora para mejorar la autonomía y la interacción con el entorno.

### 2.2. Electroencefalografía (EEG)

La electroencefalografía (EEG) consiste en la captación de la actividad eléctrica cerebral generada por la comunicación entre neuronas. Estas señales se registran mediante electrodos no invasivos colocados sobre el cuero cabelludo y son posteriormente amplificadas y representadas en función del tiempo [14], [15].

Dentro de estas señales se encuentran respuestas neuronales asociadas a distintos tipos de eventos (sensoriales, cognitivos y motores) conocidas como potenciales relacionados con eventos (ERP), así como aquellas asociadas a la imaginación motora (IM) [14], [16].

Es común realizar una separación de estas señales en función de su frecuencia, ya que se ha demostrado que distintas bandas de frecuencia se corresponden con estados mentales concretos [15], [17].

- **Delta** ( $<4$  Hz): Ubicadas en la región frontal en adultos y posterior en los niños. Se asocian principalmente al sueño, siendo también frecuentes en niños y en tareas de atención sostenida.
- **Theta** (4–7 Hz): Predominan en las regiones parietal y temporal en niños. En adultos, aparecen durante algunas fases del sueño o en estados de estrés
- **Alpha** (8–13 Hz): Se localizan en la región occipital. Se asocian a estados de relajación con los ojos cerrados, disminuyendo durante la concentración y el sueño.
- **Beta** (13–30 Hz): Son detectadas en los lóbulos frontal y parietal y presentan una distribución relativamente simétrica. Aparecen en tareas de calma activa con los ojos abiertos. Pero también pueden aparecer durante actividad mental intensa o situaciones de estrés.
- **Gamma** ( $>30$  Hz): Se han relacionado con actividad en la región somatosensorial durante tareas de procesamiento multisensorial y en procesos cognitivos complejos, como la memoria.
- **Mu** (8–12 Hz): Se localizan en el córtex sensoriomotor y se asocian a la actividad de este sistema en reposo.

El análisis de señales EEG es una herramienta muy potente que permite revelar información sobre la actividad cerebral de un individuo, teniendo muchas aplicaciones en neurociencia y en el control de BCI [18].

Entre las principales ventajas del uso de EEG se encuentran [14], [15], [19], [20]:

- **Alta resolución temporal:** Permite registrar cambios en la actividad cerebral en el rango de milisegundos.

- **Medición directa de actividad eléctrica:** A diferencia de otras técnicas que estiman la actividad cerebral de forma indirecta mediante cambios en los niveles de oxígeno en sangre, las señales EEG registran directamente la actividad eléctrica generada por poblaciones neuronales.
- **Carácter no invasivo:** La señal puede registrarse mediante electrodos colocados sobre el cuero cabelludo, sin necesidad de procedimientos quirúrgicos. Esto reduce el riesgo de sufrir complicaciones durante la captura de la señal y las molestias a los pacientes.
- **Bajo coste relativo:** En comparación con otras técnicas de neuroimagen, como la resonancia magnética funcional, los sistemas EEG suelen ser más accesibles y fáciles de utilizar.

Algunas de las desventajas que convierten a estas señales en difíciles de analizar son las siguientes [14], [21], [22]:

- **Baja resolución espacial:** Las señales EEG no permiten localizar con precisión la fuente de actividad cerebral, ya que representan una media de múltiples fuentes de actividad neuronales.
- **Variabilidad inter-sujeto e intra-sujeto:** Existe una gran variabilidad en las señales EEG entre diferentes individuos, lo que dificulta la observación de patrones en ellas. A su vez, las propiedades estadísticas de las señales pueden variar a lo largo del tiempo (son señales no estacionarias), incluso para un mismo sujeto y tarea.
- **Ruido:** Las señales EEG son altamente susceptibles a artefactos, como el movimiento, la actividad muscular (EMG) o interferencias externas. Además, la amplitud de las señales EEG es muy baja en comparación con el ruido presente, lo que exige un preprocesado avanzado para eliminar el ruido y mejorar la calidad de las señales.

Estas limitaciones justifican el uso de técnicas avanzadas de procesamiento y aprendizaje automático. En los últimos años el uso de modelos de aprendizaje profundo

(deep learning) ha supuesto un avance significativo, facilitando el desarrollo de aplicaciones más prácticas y reduciendo la dependencia de procesamiento manual por parte de expertos [18].

En el contexto del aprendizaje profundo (*deep learning*) surge una cuarta limitación que es la **escasez de datos** disponibles para el entrenamiento. Los algoritmos de deep learning requieren una gran cantidad de muestras para entrenar y generalizar, lo que dificulta su aplicación en tareas con pocos ejemplos.

A diferencia de otros dominios donde pueden obtenerse grandes volúmenes de datos de forma relativamente sencilla, la adquisición de señales EEG requiere equipamiento especializado, personal cualificado y una preparación cuidadosa de cada sesión experimental. Además, la anotación de los datos suele ser costosa y dependiente del protocolo utilizado [23].

La recogida de datos EEG también presenta limitaciones logísticas, ya que las sesiones experimentales suelen requerir tiempos prolongados de preparación y ejecución, lo que dificulta la obtención de medidas consecutivas [24]. En aplicaciones BCI, este problema se ve agravado por la fatiga física y mental de los participantes, especialmente cuando se requieren múltiples ensayos o sesiones de larga duración [25].

Como consecuencia de estas limitaciones, los conjuntos de datos EEG suelen ser significativamente más reducidos que los utilizados habitualmente en otras áreas del aprendizaje profundo.

### 2.3. Interfaces Cerebro-Computador (BCI)

Una interfaz cerebro-computador (BCI) es un sistema capaz de adquirir señales cerebrales, procesarlas y traducirlas en comandos de control para un dispositivo externo, sin necesidad de utilizar las vías motoras convencionales [26], [27]. Inicialmente, las aplicaciones de estas interfaces se centraban en la asistencia y rehabilitación de pacientes con lesiones neurológicas, permitiendo restaurar o compensar parcialmente funciones motoras perdidas [27], [28].

El esquema general de las BCI se representa en la figura 2.2. En él se observan las distintas fases de las que se compone el sistema de control [27], [29], [30].

El primer paso es la adquisición de la señal cerebral, en el cual se registran los potenciales eléctricos cerebrales mediante electrodos posicionados en distintas zonas de la cabeza.

Después de la adquisición de la señal, se realiza un paso de preprocesado en el cual se elimina el ruido y los posibles artefactos de la señal, como las interferencias de red. A continuación, se aplican técnicas de extracción de características con el objetivo de obtener información relevante para la clasificación de la señal. Este procesamiento puede incluir técnicas de análisis temporal, espectral o espacial, dependiendo de la aplicación considerada.

El cuarto paso es la clasificación de la señal, en el cual se utiliza un clasificador, que detecta los patrones en las características extraídas de la señal. Los patrones detectados son posteriormente traducidos en comandos de control para el dispositivo externo. Finalmente se proporciona una retroalimentación al usuario con información acerca del resultado de los comandos enviados. Esta retroalimentación puede presentarse de forma visual, auditiva o mediante la respuesta física del propio dispositivo controlado.

Las aplicaciones de las BCI abarcan desde la sustitución o restauración de funciones perdidas debido a lesiones o enfermedades neurológicas, hasta la mejora o ampliación de las capacidades del usuario. Entre sus usos más habituales se encuentran los sistemas de comunicación para personas con parálisis, el control de dispositivos motores como sillas de ruedas, prótesis o exoesqueletos, así como sistemas de neuroestimulación destinados recuperar funciones motoras perdidas mediante el estímulo en los músculos o nervios de la zona afectada [29], [31], [32].

Las BCI pueden clasificarse en invasivas, parcialmente invasivas y no invasivas según la ubicación de los electrodos. Las BCI invasivas registran la actividad neuronal mediante electrodos implantados directamente en el cerebro, proporcionando una elevada resolución espacial y temporal a costa de requerir procedimientos quirúrgicos complejos. Las BCI parcialmente invasivas, basadas habitualmente en electrocorti-

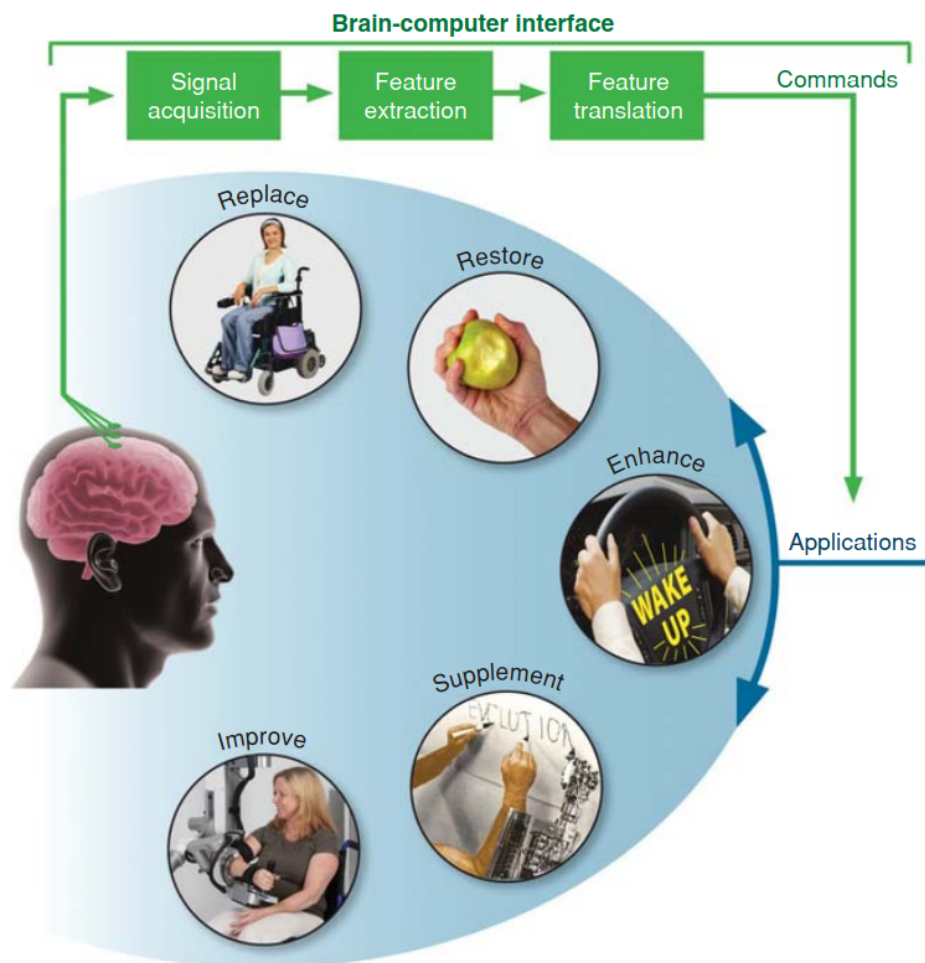


Figura 2.2: Esquema general del funcionamiento de una BCI [29]

cografía (ECoG), sitúan los electrodos sobre la superficie cortical y representan una solución intermedia entre calidad de señal y riesgo quirúrgico. Finalmente, las BCI no invasivas utilizan sensores externos, como EEG o MEG, colocados sobre el cuero cabelludo. Aunque presentan una menor resolución espacial debido a la atenuación de la señal por los tejidos craneales, destacan por su seguridad, menor coste y facilidad de uso, siendo las más empleadas en aplicaciones clínicas y de investigación [27]-[29], [33].

En este trabajo se emplea una BCI no invasiva basada en señales EEG para detectar la intención de marcha del usuario, permitiendo su futura integración con sistemas de asistencia robótica como exoesqueletos.

### **2.4. Métodos tradicionales de extracción de características en señales EEG**

El análisis de señales EEG ha estado tradicionalmente asociado al proceso de extracción de características capaces de representar la información relevante contenida en la señal. Como se ha mencionado previamente, en el ámbito de las BCI, la extracción de características es un paso fundamental en el proceso de clasificación. En este proceso, el rendimiento del sistema depende en gran medida de la capacidad de las características extraídas para representar diferencias discriminativas entre clases [34]-[36].

Las características comúnmente empleadas en señales EEG pueden agruparse en distintas familias. Entre las más habituales se encuentran las características temporales, frecuenciales, tiempo-frecuencia y espaciales [34], [37], [38].

#### **2.4.1. Características temporales, frecuenciales y tiempo-frecuencia**

Las características temporales se calculan directamente sobre la señal registrada en una ventana de tiempo determinada. Este tipo de características permite describir propiedades estadísticas de la señal a lo largo del tiempo. Algunos ejemplos habi-

tuales son la media, la varianza, la desviación estándar, la amplitud pico a pico o la asimetría. Su principal ventaja es su sencillez y bajo coste computacional, aunque pueden resultar limitadas cuando la información discriminativa se encuentra asociada a cambios espectrales o a ritmos cerebrales concretos.

Por otro lado, las características frecuenciales analizan la distribución de potencia de la señal en distintas bandas de frecuencia. En el caso de las señales EEG, este enfoque resulta especialmente relevante, ya que determinados ritmos cerebrales se han relacionado con tareas cognitivas y motoras. En particular, las bandas  $\mu$  y  $\beta$ , se han relacionado con tareas motoras y la imaginación de las mismas [39]. Para extraer este tipo de información suelen emplearse técnicas como la transformada de Fourier, la estimación de la densidad espectral de potencia o el cálculo de potencia en bandas específicas.

Sin embargo, una de las dificultades del análisis de señales EEG es que estas presentan un carácter no estacionario, por lo que sus propiedades estadísticas pueden variar a lo largo del tiempo. Por este motivo, también se emplean métodos de análisis tiempo-frecuencia, como la transformada wavelet o la transformada de Fourier de tiempo corto (STFT) [37], [38], que permiten estudiar simultáneamente la evolución temporal y espectral de la señal.

#### **2.4.2. Características espaciales: CSP y FBCSP**

Además del contenido temporal y frecuencial, las señales EEG contienen información espacial, ya que la actividad cerebral se registra mediante múltiples electrodos distribuidos sobre el cuero cabelludo. En lugar de analizar cada electrodo de forma independiente, los métodos espaciales buscan calcular nuevas señales como combinaciones lineales de los canales originales, buscando aumentar la discriminación entre clases.

Uno de los métodos de filtrado espacial más utilizados en sistemas BCI es el algoritmo de **Patrón Espacial Común (CSP)**. Este método calcula filtros espaciales que proyectan la señal multicanal en un nuevo espacio, maximizando la varianza de una



clase y minimizando simultáneamente la varianza de la otra. De este modo, CSP permite obtener componentes espaciales especialmente útiles para la clasificación [40], [41].

Una limitación importante de CSP es que su rendimiento depende en gran medida de la banda frecuencial seleccionada, la cual puede variar entre sujetos, sesiones o tareas.

Para abordar esta limitación se propuso el método de **Patrón Espacial Común mediante Banco de Filtros (FBCSP)** [42], [43]. Este algoritmo aplica primero un banco de filtros para dividir la señal EEG en múltiples bandas de frecuencia. Posteriormente, se aplica CSP de forma independiente sobre cada banda, obteniendo características espaciales asociadas a distintos rangos frecuenciales.

Finalmente, FBCSP añade una etapa de selección de características para conservar aquellas con mayor capacidad discriminativa. De esta forma, el método permite seleccionar tanto los patrones espaciales como las bandas de frecuencia más relevantes para cada sujeto o tarea.

Por esto, FBCSP ha sido ampliamente utilizado en sistemas BCI de imaginación motora, habitualmente junto con clasificadores convencionales como *Naïve Bayesian* *Parzen Window* (NVPW) [42], [43].

Sin embargo, a pesar de su eficacia, los métodos tradicionales de extracción de características presentan varias limitaciones importantes. Por ejemplo, al mantener la extracción y selección de características separadas de la clasificación, estos métodos requieren un conocimiento previo sobre la señal y la tarea a analizar. Esta dependencia del diseño manual de características ha motivado el uso de enfoques basados en *deep learning*, capaces de aprender características discriminativas directamente a partir de los datos.

## 2.5. Redes neuronales convolucionales

Las **Redes Neuronales Convolucionales (CNN)** constituyen una de las arquitecturas de aprendizaje profundo más empleadas para el análisis de datos complejos. Aunque fueron desarrolladas originalmente para tareas de visión artificial, su capacidad para extraer características relevantes de manera automática y eficiente las ha convertido en una herramienta ampliamente utilizada en ámbitos como: el procesamiento de imágenes, el reconocimiento de voz, el procesamiento del lenguaje natural y, más recientemente, el análisis de señales biomédicas como el electroencefalograma (EEG) [33], [44]-[46].

La operación en la que se basan estas redes es la convolución, una transformación matemática que aplica pequeños filtros o *kernels* sobre la entrada para detectar patrones específicos. Cada filtro aprende durante el entrenamiento a responder a determinadas características presentes en los datos, generando como salida un conjunto de mapas de características (*feature maps*). Estos *feature maps* se vuelven progresivamente más abstractos y complejos conforme se avanza en la red, lo que le permite aprender automáticamente las características más discriminativas para la tarea de clasificación. Las capas convolucionales requieren un número significativamente menor de parámetros que las capas completamente conectadas (*fully connected*), reduciendo el coste computacional y favoreciendo la capacidad de generalización del modelo [33], [44], [45], [47].

Habitualmente, una CNN está formada por una sucesión de capas convolucionales y capas de reducción dimensional (*pooling*), seguidas de una o varias capas *fully connected*. Las capas convolucionales se encargan de extraer características relevantes de la señal de entrada, mientras que las capas de *pooling* reducen la dimensionalidad de las representaciones obtenidas y contribuyen a disminuir el sobreajuste. Finalmente, las capas *fully connected* utilizan las características extraídas para realizar la clasificación o regresión correspondiente [44], [48].

En la figura 2.3 se muestra un ejemplo simplificado de la arquitectura de una red neuronal convolucional.

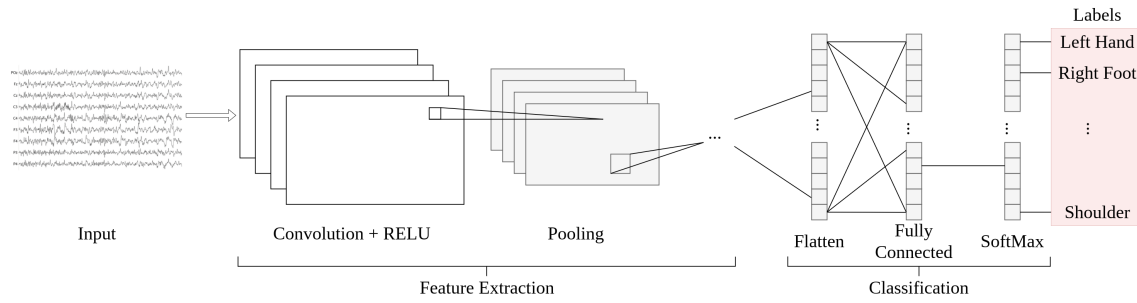


Figura 2.3: Esquema general de una red neuronal convolucional adaptada a la clasificación de intención de movimiento a partir de señales EEG (Elaboración propia).

### CUAL COJO?

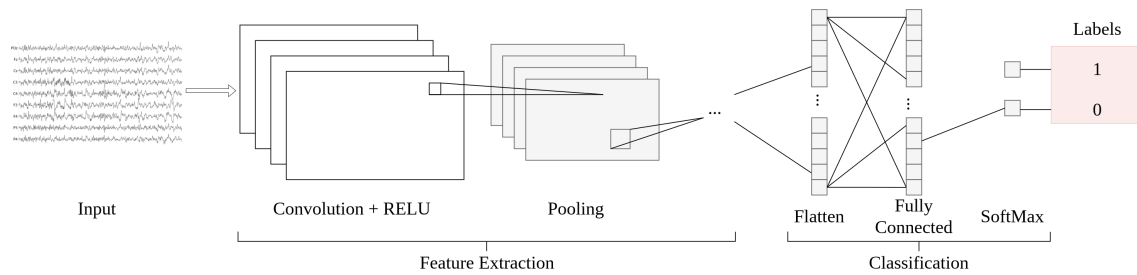


Figura 2.4: Esquema general de una red neuronal convolucional para la tarea de clasificación binaria en EEG (Elaboración propia).

A pesar de las ventajas que aportan estas redes neuronales, uno de los principales desafíos en su aplicación es el diseño de la arquitectura, la selección de hiperparámetros y la necesidad de disponer de una cantidad suficiente de datos para entrenar modelos robustos. Para mitigar estas limitaciones, se han propuesto estrategias como el *transfer learning*, la cual consiste en utilizar modelos previamente entrenados en tareas similares y adaptarlos a la tarea específica mediante un proceso de ajuste fino (*fine-tuning*). El transfer learning se utiliza habitualmente en escenarios donde la cantidad de datos disponibles es limitada, ya que permite aprovechar el conocimiento adquirido por el modelo en un dominio diferente; o donde la arquitectura de la red es compleja y modificarla desde cero sería costoso en términos de tiempo y recursos computacionales [49], [50].

En el ámbito de las interfaces cerebro-computador (BCI), las CNN se han consolidado como una de las técnicas más utilizadas para la decodificación de señales EEG [18], [20], [33]. A diferencia de las imágenes, las señales EEG presentan simultáneamente una dimensión temporal y una dimensión espacial asociada a la distribución de

los electrodos sobre el cuero cabelludo. Por este motivo, las arquitecturas diseñadas para EEG suelen combinar convoluciones temporales, encargadas de extraer información espectral y patrones dependientes del tiempo, con convoluciones espaciales que modelan las relaciones entre los distintos canales de la señal [33].

Como se ha mencionado en el punto anterior, los sistemas BCI basados en EEG dependían de técnicas de extracción manual de características combinadas con clasificadores convencionales. Sin embargo, la aparición de arquitecturas de redes neuronales profundas específicas para EEG, como DeepConvNet, ShallowConvNet y EEGNet, supuso un cambio de paradigma al integrar de forma conjunta las etapas de extracción de características y clasificación dentro de una misma red neuronal [20], [51], [52].

## **2.6. Aplicaciones**

### **2.6.1. Comparación de modelos en la literatura**

El uso de técnicas de aprendizaje profundo para el análisis de señales EEG ha experimentado un crecimiento significativo durante la última década. Este incremento ha motivado la aparición de numerosos trabajos orientados a comparar distintas arquitecturas de redes neuronales, así como a evaluar su capacidad de generalización en diferentes paradigmas BCI, conjuntos de datos y protocolos experimentales.

Uno de los trabajos más influyentes en este ámbito es el realizado por Schirrmester et al. [20], quienes introdujeron las arquitecturas DeepConvNet y ShallowConvNet para la clasificación de señales EEG. Además de proponer ambas redes, los autores realizaron una comparación frente a métodos clásicos basados en extracción manual de características, como Filter Bank Common Spatial Patterns (FBCSP), demostrando que las arquitecturas convolucionales eran capaces de alcanzar resultados comparables o superiores sin necesidad de diseñar manualmente las características relevantes de la señal. Este trabajo supuso uno de los primeros éxitos significativos del aprendizaje profundo aplicado a sistemas BCI.

Posteriormente, Lawhern et al. [52] propusieron EEGNet, una arquitectura compacta diseñada específicamente para señales EEG. A diferencia de otros modelos convolucionales más profundos, EEGNet está enfocada en reducir significativamente el número de parámetros entrenables. Los autores demostraron que era posible obtener resultados competitivos en diferentes paradigmas BCI utilizando modelos considerablemente más ligeros, facilitando su aplicación en escenarios con recursos computacionales limitados o conjuntos de datos reducidos.

Köllöd et al. [51] desarrollaron un estudio exhaustivo sobre diferentes modelos de aprendizaje profundo aplicados a EEG. Entre las arquitecturas evaluadas se encontraban EEGNet, DeepConvNet y ShallowConvNet, además de otros modelos recientes. Los autores analizaron no sólo el efecto de la arquitectura utilizada, sino también el impacto de distintas estrategias de entrenamiento, incluyendo enfoques dependientes del sujeto y métodos basados en transferencia de aprendizaje. Sus resultados mostraron que no existe una arquitectura claramente superior para todos los conjuntos de datos y escenarios experimentales, observándose una fuerte dependencia tanto del protocolo de entrenamiento como de las características específicas de cada tarea.

La conclusión anterior coincide con las observaciones recogidas en diversas revisiones bibliográficas sobre aprendizaje profundo aplicado a EEG, como los de Craik et al. [18] y Roy et al. [53]. Estos trabajos destacan que las diferencias de rendimiento entre arquitecturas suelen depender en gran medida del conjunto de datos utilizado, del paradigma BCI considerado y del protocolo experimental empleado. Asimismo, coinciden en la ausencia de una arquitectura universalmente superior y recalcan la necesidad de utilizar benchmarks comunes que permitan realizar comparaciones más rigurosas entre arquitecturas.

En conjunto, la literatura actual sugiere que tanto la arquitectura seleccionada como la estrategia de entrenamiento empleada desempeñan un papel fundamental en el rendimiento final de los sistemas BCI basados en EEG. Por este motivo, en el presente trabajo se evalúan tres de las arquitecturas convolucionales más utilizadas en la literatura (EEGNet, DeepConvNet y ShallowConvNet) junto con diferentes estrategias de entrenamiento.

### 2.6.2. Distintos entrenamientos en la literatura

Según Wan et al. [54], las diferencias inter-sujeto y la escasez de datos constituyen dos de los principales obstáculos para el desarrollo de sistemas BCI basados en aprendizaje profundo. Como consecuencia, numerosas investigaciones han propuesto distintas estrategias de transferencia y reutilización del conocimiento con el objetivo de mejorar la generalización de los modelos EEG.

#### Entrenamiento intra-sesión

El entrenamiento intra-sesión constituye el enfoque tradicional en sistemas BCI. En este escenario, el modelo se entrena y evalúa utilizando únicamente datos obtenidos durante una misma sesión del usuario. Este planteamiento minimiza el impacto de la variabilidad temporal y entre sujetos, permitiendo obtener una alta precisión para la sesión concreta utilizada durante el entrenamiento.

Ma et al. [55] propusieron un benchmark intra-sesión en el que los datos de cada sesión eran divididos aleatoriamente en conjuntos de entrenamiento, validación y test siguiendo una proporción 8:1:1. La evaluación final se realizaba mediante validación cruzada de 10 particiones. Este protocolo fue utilizado para comparar diferentes arquitecturas de aprendizaje profundo, entre ellas EEGNet, DeepConvNet y ShallowConvNet.

A pesar de proporcionar buenos resultados, este tipo de entrenamiento presenta una importante limitación práctica: requiere una nueva fase de calibración para cada sesión del usuario, dificultando su utilización en aplicaciones reales de larga duración, como mencionan Sung et al. [56].

#### Transferencia entre sesiones

Con el objetivo de reducir la necesidad de re-entrenar el modelo para cada sesión, diversos autores han estudiado estrategias que aprovechan información procedente de sesiones anteriores del mismo usuario.

En el benchmark cross-sesión propuesto por Ma et al. [55], los datos de la primera sesión de cada sujeto se utilizaron para el entrenamiento, mientras que las cuatro sesiones restantes se emplearon para evaluación. Este protocolo permite cuantificar el impacto de la variabilidad temporal de las señales EEG sobre el rendimiento de los clasificadores.

Posteriormente, Sung et al. [56] compararon diferentes estrategias de transferencia entre sesiones. Entre ellas destaca el método *Whole Session Transfer* (WST), que incorpora todas las sesiones anteriores al conjunto de entrenamiento de la sesión actual. Los autores observaron que este enfoque obtenía mejores resultados que el entrenamiento exclusivamente intra-sesión. Además, propusieron una estrategia denominada *Relevant Session Transfer* (RST), basada en la selección de sesiones previas mediante medidas de similitud, obteniendo mejoras adicionales en la precisión de clasificación.

De forma similar, Wu et al. [57] identifican la transferencia cross-sesión como uno de los principales escenarios de *transfer learning* en sistemas EEG, donde los datos de sesiones previas son utilizados para facilitar la calibración de nuevas sesiones del mismo usuario.

### Transferencia entre sujetos

Otra línea de investigación busca reducir la dependencia de datos específicos del usuario mediante la utilización de información procedente de otros participantes.

Según la clasificación propuesta por Wu et al. [57], el escenario *cross-subject* utiliza datos de varios sujetos como dominios fuente para facilitar la calibración de un nuevo usuario. Este enfoque resulta especialmente interesante debido a que permite disminuir significativamente el tiempo necesario para la adquisición de datos de calibración.

Diversos trabajos recientes han explorado estrategias de adaptación basadas en modelos pre-entrenados con múltiples usuarios y posteriormente ajustados mediante *fine-tuning* utilizando una cantidad reducida de datos del sujeto objetivo Otarbay

y Kyziryanov [58] y An et al. [59]. Estos métodos han demostrado que es posible aprovechar conocimiento adquirido en otros usuarios para mejorar el rendimiento inicial de los sistemas BCI.

### **Aprendizaje continuo y adaptación progresiva**

Más recientemente, varios autores han estudiado estrategias de adaptación progresiva en las que el modelo incorpora nueva información conforme se dispone de sesiones adicionales del usuario.

Lu et al. [60] proponen un procedimiento basado en el entrenamiento inicial de un modelo general utilizando datos fuente y posteriormente su adaptación mediante *fine-tuning* con datos específicos del usuario objetivo. Por su parte, Wimpff et al. [61] demostraron que la adaptación progresiva utilizando únicamente datos disponibles antes de cada sesión de evaluación permite mejorar tanto la precisión como la estabilidad del sistema a lo largo del tiempo.

Este tipo de enfoques se encuentra estrechamente relacionado con el aprendizaje continuo (*continual learning*), cuyo objetivo es incorporar nueva información preservando simultáneamente el conocimiento previamente adquirido y reduciendo el problema de la pérdida de rendimiento en sesiones largas (Huang et al. [62]). En esta misma línea, Abu-Rmileh et al. [63] propusieron un mecanismo de actualización periódica del clasificador utilizando los datos de los dos ensayos anteriores del usuario para adaptar progresivamente el modelo durante varios días de uso.

Las estrategias descritas anteriormente constituyen la base de los distintos métodos de entrenamiento evaluados en este trabajo, los cuales buscan estudiar el impacto del método de transferencia sobre el rendimiento de modelos convolucionales aplicados a señales EEG.



## 3. Material y Métodos

### 3.1. Adquisición de datos

El dataset utilizado en este trabajo está compuesto por registros EEG obtenidos durante sesiones experimentales de imaginación motora y relajación realizadas con un exoesqueleto robótico de miembro inferior. En concreto, el conjunto de datos incluye registros de 5 sujetos, adquiridos a lo largo de 5 días consecutivos de pruebas, cada prueba compuesta por 11 ensayos. Durante las sesiones, los participantes alternaron entre tareas de imaginación motora asociadas a la marcha o parada mientras utilizaban el exoesqueleto, y periodos de relajación.

La adquisición de los datos se llevó a cabo siguiendo el protocolo experimental descrito en el trabajo «A BMI Based on Motor Imagery and Attention for Commanding a Lower-Limb Robotic Exoskeleton» realizado por Ferrero et al. [64].

#### 3.1.1. Equipamiento experimental

Durante los ensayos, los sujetos utilizaron el exoesqueleto H3 de Technaid, diseñado para asistir la marcha, empleando además muletas como apoyo[64]. Un dispositivo actiCap de Brain Products fue empleado para registrar la actividad cerebral. De los 32 electrodos disponibles, se seleccionaron 27 canales EEG distribuidos siguiendo el sistema internacional 10-10. Los electrodos utilizados fueron:

F3, FZ, FC1, FCZ, C1, CZ, CP1, CPZ, FC5, FC3, C5, C3, CP5, CP3, P3, PZ,

F4, FC2, FC4, FC6, C2, C4, CP2, CP4, C6, CP6 y P4.

Además de estos, se emplearon 4 electrodos en la zona frontal para registrar artefactos de electrooculográfica, mientras que los electrodos de tierra y referencia se situaron en los lóbulos derecho e izquierdo, respectivamente. Las señales obtenidas fueron amplificadas mediante un amplificador BrainVision BrainAmp y transmitidas de forma inalámbrica al software BrainVision Recorder [64]. La frecuencia de mues-

teoreo utilizada fue de 200 Hz. De la señal obtenida se extrajeron épocas de 2 segundos de duración con un desplazamiento de 0.5 segundos entre ventanas consecutivas, con lo que se obtiene una cantidad de ventanas de 400.

Por tanto, cada instancia del dataset puede interpretarse como una matriz de tamaño  $27 \times 400$ , donde cada fila representa la evolución temporal de un canal EEG durante la ventana analizada.

### 3.1.2. Protocolo experimental

Cada sesión experimental se estructuró en ensayos o *trials*. En cada ensayo, el usuario alternaba entre periodos de relajación e imaginación motora. En concreto, cada *trial* se dividió en tres fases principales:

- relajación inicial,
- imaginación motora de la marcha,
- relajación final.

La segmentación utilizada en este trabajo divide cada ensayo en 56 ventanas temporales. Estas ventanas se organizan de la siguiente forma:

- 14 ventanas de relajación inicial, equivalentes a 7 s,
- 28 ventanas de imaginación motora, equivalentes a 14 s,
- 14 ventanas de relajación final, equivalentes a 7 s.

Antes de cada tarea mental se proporcionaba una instrucción al usuario. Estos periodos de instrucción no corresponden directamente a ninguna de las tareas mentales objetivo, por lo que fueron eliminados durante el procesamiento previo de los datos.

Adicionalmente, antes de cada ensayo existía un periodo inicial de duración aleatoria en el que el sujeto realizaba una tarea mental distinta. Este segmento tenía como

objetivo reducir artefactos y estabilizar la señal antes de la tarea principal, por lo que también fue descartado durante el preprocesamiento.

Durante las sesiones experimentales los sujetos alternaban entre dos tareas con el objetivo de reducir la fatiga y evitar la monotonía durante la adquisición [65].:

- *Estática/Static*, donde los usuarios comenzaban parados y realizaban un esfuerzo mental activo orientado a iniciar el movimiento
- *Movimiento/Motion*, donde la imaginación motora consistía en detener la marcha.

La Figura 3.1 muestra un esquema de la segmentación temporal de las ventanas empleada durante los ensayos.

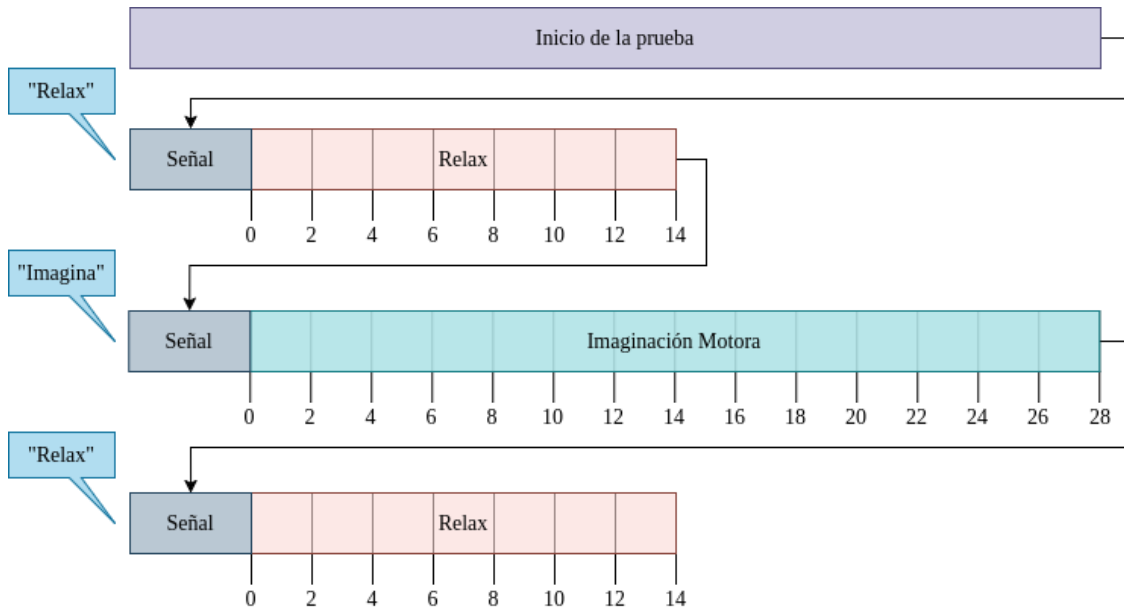


Figura 3.1: Estructura temporal de las ventanas EEG utilizadas durante cada ensayo. Adaptado de [66].

#### 3.1.3. Estructura y cuantificación del dataset

A partir del protocolo experimental descrito, el dataset queda organizado jerárquicamente en sujetos, días, ensayos y ventanas temporales. En concreto, se dispone de:

- 5 sujetos,
- 5 días consecutivos de adquisición por sujeto,
- 11 ensayos por día,
- 56 ventanas temporales por ensayo.

Por tanto, el número total de ventanas disponibles es:

$$5 \times 5 \times 11 \times 56 = 15400 \text{ ventanas}$$

Cada ventana contiene 27 canales EEG y 400 muestras temporales. Así, los datos de entrada presentan inicialmente una estructura de tipo:

$$(15400, 27, 400)$$

mientras que las etiquetas asociadas presentan una dimensión:

$$(15400, 1)$$

donde cada etiqueta indica la clase correspondiente a la ventana temporal analizada.

Los datos originales se encuentran distribuidos en cuatro ficheros MATLAB diferenciados según la tarea experimental:

|               | <b>Labels</b>          | <b>Training</b>          |
|---------------|------------------------|--------------------------|
| <b>Static</b> | labels_static_data.mat | training_static_data.mat |
| <b>Motion</b> | labels_motion_data.mat | training_motion_data.mat |

Los ficheros de etiquetas contienen la intención de movimiento asociada a cada ventana temporal, mientras que los ficheros de entrenamiento almacenan las señales EEG correspondientes.

Con el objetivo de facilitar el entrenamiento y evaluación de los modelos propuestos, los datos fueron reorganizados en tensores que preservan la jerarquía original del dataset. La figura 3.2 muestra la estructura de los tensores de entrada y etiquetas obtenidos a partir de los datos originales.

| <b>Input tensor shape: (5, 5, 11, 56, 27, 400)</b> |       |      |        |         |          |         |
|----------------------------------------------------|-------|------|--------|---------|----------|---------|
| <b>Dim.</b>                                        | 1     | 2    | 3      | 4       | 5        | 6       |
| <b>Name</b>                                        | Users | Days | Trials | Windows | Channels | Samples |
| <b>Size</b>                                        | 5     | 5    | 11     | 56      | 27       | 400     |
| <b>Labels tensor shape: (5, 5, 11, 56, 1)</b>      |       |      |        |         |          |         |
| <b>Dim.</b>                                        | 1     | 2    | 3      | 4       | 5        |         |
| <b>Name</b>                                        | Users | Days | Trials | Windows | Label    |         |
| <b>Size</b>                                        | 5     | 5    | 11     | 56      | 1        |         |

Figura 3.2: Estructura jerárquica de los tensores de entrada y etiquetas utilizados en el modelo.

Esta organización permite mantener explícitamente la procedencia experimental de cada ventana, diferenciando el sujeto, el día, el ensayo y la posición temporal dentro del ensayo. Esto resulta especialmente útil para definir posteriormente las distintas estrategias de entrenamiento y evaluación empleadas en este trabajo.

#### 3.1.4. Preprocesamiento

Previo al entrenamiento de los modelos, las señales EEG fueron sometidas a una etapa de preprocesamiento con el objetivo de reducir el ruido y conservar únicamente los componentes frecuenciales relevantes para las tareas mentales objetivo.

Se aplicaron dos filtros en la etapa inicial de preprocesamiento: un filtro notch a 50 Hz para reducir el ruido provocado por la red eléctrica y un filtro paso alto a 0.1 Hz para eliminar la componente continua y derivas lentas de la señal [64]. Se aplicó también el algoritmo  $H_{\infty}$  para la eliminación de artefactos oculares, que permite reducir la influencia de movimientos oculares y parpadeos en la señal EEG [67].

Finalmente, a cada canal EEG se le aplicó un filtro paso banda entre 8 y 40 Hz.

Este rango frecuencial incluye principalmente las bandas  $\mu$  (8–13 Hz) y  $\beta$  (13–30 Hz), ampliamente utilizadas en aplicaciones BCI basadas en imaginación motora debido a su relación con la actividad cortical motora [39].

Posteriormente, se eliminaron los periodos correspondientes a instrucciones y segmentos iniciales no pertenecientes a las tareas mentales objetivo. De esta forma, el análisis se centró únicamente en las ventanas asociadas a relajación e imaginación motora.

## 3.2. Materiales para el estudio de los datos

### 3.2.1. Hardware

Para ejecutar los distintos entrenamientos se ha utilizado un ordenador con las siguientes características:

- **CPU:** Intel Core i9-14900HX
- **RAM:** 32 GiB
- **GPU:** NVIDIA GeForce RTX 4060
- **GPU RAM:** 8 GiB

Estas características han resultado suficientes para completar los experimentos planteados sin limitaciones críticas. El tiempo medio de entrenamiento por iteración ha sido  $\approx 40$  minutos, dependiendo ligeramente de la configuración del modelo y de los parámetros de entrenamiento.

El uso de hardware con mayores prestaciones permitiría reducir significativamente el tiempo total de entrenamiento y facilitar la realización de un mayor número de experimentos.

#### 3.2.2. Software

Para el desarrollo del proyecto se ha utilizado una infraestructura basada en herramientas ampliamente empleadas en el ámbito del aprendizaje automático y la ingeniería de software.

Como lenguaje de programación principal se ha utilizado Python<sup>1</sup>, debido a su amplio ecosistema de librerías científicas y de aprendizaje automático, así como su facilidad de integración con frameworks de alto nivel.

Para la construcción y entrenamiento de los modelos se han utilizado TensorFlow<sup>2</sup> y Keras<sup>3</sup>, frameworks de aprendizaje profundo que permiten definir, entrenar y evaluar redes neuronales de forma eficiente. Keras se ha empleado como interfaz de alto nivel sobre TensorFlow, facilitando la experimentación rápida con distintas arquitecturas.

Durante las etapas iniciales del proyecto se empleó Jupyter Notebook<sup>4</sup> para la realización de pruebas exploratorias y la visualización preliminar de resultados. Sin embargo, su uso fue descartado en fases posteriores debido a la dificultad para mantener una estructura de código modular y escalable en experimentos de mayor complejidad.

Posteriormente, se adoptó Visual Studio Code<sup>5</sup> como entorno de desarrollo principal, debido a su capacidad para gestionar múltiples tipos de archivos y su integración con herramientas de desarrollo. Se utilizaron extensiones específicas para Python, Jupyter y GitHub, permitiendo un flujo de trabajo unificado.

Como sistema de control de versiones se ha utilizado GitHub<sup>6</sup>, que facilita la gestión del código, el seguimiento de cambios y la colaboración con el tutor del proyecto. Además, permite la sincronización del repositorio entre distintos dispositivos de forma eficiente. El código final desarrollado durante este trabajo se encuentra disponible en un repositorio público, referenciado en la bibliografía [68].

---

<sup>1</sup><https://www.python.org/>

<sup>2</sup><https://www.tensorflow.org/>

<sup>3</sup><https://keras.io/>

<sup>4</sup><https://jupyter.org/>

<sup>5</sup><https://code.visualstudio.com/>

<sup>6</sup><https://github.com/>

Para la visualización de resultados se ha utilizado Vega-Altair<sup>7</sup>, una librería de visualización estadística declarativa enfocada en la simplicidad, eficiencia y legibilidad. A diferencia de enfoques imperativos como Matplotlib, Altair permite definir visualizaciones de forma estructurada y reproducible, enfocándose en lo que se quiere observar en lugar de en los detalles de la implementación. Esto conduce a un código más limpio y legible [69]

Como gestor de base de datos se ha utilizado DuckDB<sup>8</sup>, un sistema de bases de datos analítico embebido que permite realizar consultas SQL de forma eficiente directamente sobre archivos locales. Su integración con Python y su alto rendimiento lo hacen especialmente adecuado para el procesamiento de grandes volúmenes de resultados experimentales [70].

Todas estas herramientas han sido integradas en un flujo de trabajo unificado orientado a la ejecución reproducible de experimentos.

### 3.3. Modelos utilizados

Se van a definir las arquitecturas de los modelos usados en el estudio y a justificar su elección.

#### 3.3.1. DeepConvNet y ShallowConvNet

Las arquitecturas DeepConvNet y ShallowConvNet, propuestas por Schirrmeyer et al. [20], fueron de los primeros modelos convolucionales en lograr un impacto significativo en la clasificación de señales EEG. Desde entonces, estas arquitecturas han sido ampliamente utilizadas como referencia en numerosos trabajos relacionados con interfaces cerebro-computador y Deep Learning aplicado a EEG [51].

La arquitectura DeepConvNet está formada por cuatro bloques convolucionales seguidos de operaciones de *max-pooling* y una capa final de clasificación *softmax*. El primer bloque convolucional fue diseñado específicamente para adaptarse a las carac-

---

<sup>7</sup><https://altair-viz.github.io/>

<sup>8</sup><https://duckdb.org/>



terísticas de las señales EEG, las cuales presentan un número elevado de canales de entrada correspondientes a los distintos electrodos. Para ello, el bloque es separado en dos capas. La primera realiza un filtrado temporal sobre la señal de cada canal, permitiendo extraer patrones relacionados con las frecuencias presentes en el EEG. Posteriormente, la segunda convolución aplica un filtrado espacial combinando la información obtenida en la primera capa. Esta separación entre convolución temporal y espacial actúa además como una forma implícita de regularización, forzando a la red a aprender transformaciones más estructuradas [20].

| Block      | Layer      | # filters | Size    | # params | Output        |
|------------|------------|-----------|---------|----------|---------------|
| 1          | Input      | –         | –       | 0        | (27, 400, 1)  |
|            | Conv2D     | 25        | (1, 8)  | 225      | (27, 393, 25) |
|            | Conv2D     | 25        | (27, 1) | 16900    | (1, 393, 25)  |
|            | BatchNorm  | –         | –       | 100      | (1, 393, 25)  |
|            | Activation | –         | ELU     | 0        | (1, 393, 25)  |
|            | MaxPool2D  | –         | (1, 2)  | 0        | (1, 196, 25)  |
|            | Dropout    | –         | –       | 0        | (1, 196, 25)  |
| 2          | Conv2D     | 50        | (1, 8)  | 10050    | (1, 189, 50)  |
|            | BatchNorm  | –         | –       | 200      | (1, 189, 50)  |
|            | Activation | –         | ELU     | 0        | (1, 189, 50)  |
|            | MaxPool2D  | –         | (1, 2)  | 0        | (1, 94, 50)   |
|            | Dropout    | –         | –       | 0        | (1, 94, 50)   |
| 3          | Conv2D     | 100       | (1, 8)  | 40100    | (1, 87, 100)  |
|            | BatchNorm  | –         | –       | 400      | (1, 87, 100)  |
|            | Activation | –         | ELU     | 0        | (1, 87, 100)  |
|            | MaxPool2D  | –         | (1, 2)  | 0        | (1, 43, 100)  |
|            | Dropout    | –         | –       | 0        | (1, 43, 100)  |
| 4          | Conv2D     | 200       | (1, 8)  | 160200   | (1, 36, 200)  |
|            | BatchNorm  | –         | –       | 800      | (1, 36, 200)  |
|            | Activation | –         | ELU     | 0        | (1, 36, 200)  |
|            | MaxPool2D  | –         | (1, 2)  | 0        | (1, 18, 200)  |
|            | Dropout    | –         | –       | 0        | (1, 18, 200)  |
|            | Flatten    | –         | –       | 0        | (3600)        |
| Classifier | Dense      | –         | –       | 7202     | (2)           |
|            | Activation | –         | softmax | 0        | (2)           |

Tabla 3.1: Arquitectura DeepConvNet utilizada.

Por otro lado, la arquitectura ShallowConvNet fue diseñada específicamente para la clasificación de características de la potencia de banda. Su estructura busca aproximarse al funcionamiento del método FBCSP mediante operaciones equivalentes implementadas mediante capas convolucionales. Al igual que DeepConvNet, las primeras capas realizan convoluciones temporales y espaciales. Sin embargo, en este caso la convolución temporal utiliza kernels de mayor tamaño con el objetivo de

capturar mejor la información frecuencial de la señal.

Después de estas convoluciones, ShallowConvNet incorpora una secuencia de operaciones compuesta por una *no linealidad cuadrática*, una capa de *mean-pooling* y una activación logarítmica. Este conjunto de transformaciones es análogo al cálculo de la varianza logarítmica utilizado tradicionalmente en FBCSP. Además, el uso de múltiples regiones de *pooling* dentro de un mismo ensayo permite a la red aprender variaciones temporales de la potencia de banda a lo largo del tiempo [20].

| Block      | Layer         | # filters | Size      | # params | Output        |
|------------|---------------|-----------|-----------|----------|---------------|
| 1          | Input         | –         | –         | 0        | (27, 400, 1)  |
|            | Conv2D        | 40        | (1, 20)   | 840      | (27, 381, 40) |
|            | Conv2D        | 40        | (27, 1)   | 43200    | (1, 381, 40)  |
|            | BatchNorm     | –         | –         | 160      | (1, 381, 40)  |
|            | Activation    | –         | square    | 0        | (1, 381, 40)  |
|            | AveragePool2D | –         | (1, 60)   | 0        | (1, 27, 40)   |
|            | Activation    | –         | log       | 0        | (1, 27, 40)   |
|            | Dropout       | –         | $p = 0,4$ | 0        | (1, 27, 40)   |
|            | Flatten       | –         | –         | 0        | (1080)        |
| Classifier | Dense         | –         | –         | 2162     | (2)           |
|            | Activation    | –         | softmax   | 0        | (2)           |

Tabla 3.2: Arquitectura ShallowConvNet utilizada.

Las Tablas 3.2 y 3.1 muestran las configuraciones específicas de las arquitecturas ShallowConvNet y DeepConvNet utilizadas en este trabajo. En ambos casos, se adaptaron los parámetros originales propuestos por Schirrmeister et al. [20] para ajustarse a las características del conjunto de datos empleado, compuesto por señales EEG de 27 canales y ventanas temporales de 400 muestras.

Puede observarse que ShallowConvNet está formada por un único bloque convolucional seguido de las operaciones de agrupamiento y clasificación, mientras que DeepConvNet emplea una estructura jerárquica compuesta por cuatro bloques convolucionales sucesivos, incrementando progresivamente el número de filtros y la capacidad de representación del modelo.

Los resultados obtenidos por Schirrmeister et al. mostraron que estas arquitecturas alcanzaban resultados comparables a métodos clásicos como Patrón Espacial Común mediante Banco de Filtros (FBCSP) [20], [33].

#### 3.3.2. EEGNet

EEGNet es un modelo de red neuronal convolucional (CNN) diseñado específicamente para la clasificación de señales EEG. Su arquitectura se basa en el uso de convoluciones *Depthwise* y *Separable*, ampliamente utilizadas en tareas de clasificación de imágenes, adaptándolas al procesamiento de señales EEG. Este enfoque permite extraer características espaciales y temporales de forma eficiente, utilizando conceptos como el filtrado espacial y la construcción de bancos de filtros [52].

Las características principales de este modelo son:

- Independencia respecto al paradigma BCI, permitiendo su aplicación en distintos tipos de señales EEG.
- Menor número de parámetros en comparación con otros modelos de CNN.
- Buena capacidad de generalización incluso con conjuntos de datos limitados.
- Arquitectura compacta con técnicas de regularización integradas, como *Batch Normalization* y *Dropout*.

El modelo comienza con una capa convolucional temporal encargada de extraer información frecuencial de la señal EEG. Posteriormente, se aplica una convolución *Depthwise* que aprende filtros espaciales específicos para cada mapa de características. A continuación, la arquitectura utiliza una convolución *Separable*, compuesta por una convolución *Depthwise* seguida de una convolución *Pointwise*. La primera resume temporalmente cada mapa de características de manera independiente, mientras que la segunda combina la información obtenida entre los distintos mapas de características [52]. El esquema de esta arquitectura se muestra en la figura 3.3.

Finalmente, el modelo se entrena utilizando el optimizador Adam y minimizando la función de pérdida de entropía cruzada categórica.

Estudios posteriores han mostrado que EEGNet es capaz de aprender representaciones similares a las obtenidas mediante métodos clásicos de extracción de características, como Filter Bank Common Spatial Patterns (FBCSP), manteniendo la ventaja

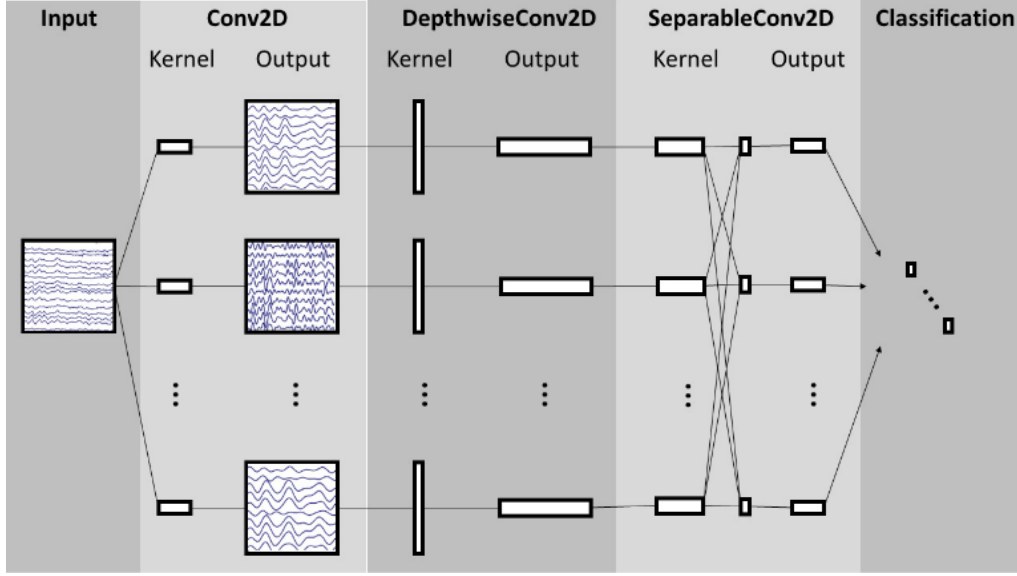


Figura 3.3: Arquitectura de EEGNet. las líneas muestran la conexión entre las capas [52].

de realizar dicho proceso de forma automática dentro de la propia red neuronal. Finalmente, esta arquitectura ha mostrado buenos resultados al clasificar potenciales en la señal EEG relacionados con el movimiento (MRCP) [51], [52].

La elección de este modelo se debe principalmente a su arquitectura compacta y a su capacidad para obtener buenos resultados incluso con conjuntos de datos reducidos, una situación habitual en aplicaciones basadas en EEG. Además, el uso de convoluciones *Depthwise* y *Separable* permite reducir significativamente el número de parámetros del modelo respecto a otras arquitecturas convolucionales, disminuyendo así el coste computacional del entrenamiento [52].

### 3.4. Metodología experimental

#### 3.4.1. Variables del estudio

En este trabajo se realiza un análisis comparativo entre las arquitecturas *EEGNet*, *ShallowConvNet* y *DeepConvNet*. Para ello, se han definido cuatro estrategias de entrenamiento con el objetivo de evaluar cómo se adapta cada modelo a diferentes usuarios, sesiones y condiciones experimentales.

Las tareas de imaginación motora consideradas son la imaginación de movimiento (*Static*  $\Rightarrow$  *Motion*) y la imaginación de parada (*Motion*  $\Rightarrow$  *Static*). En la primera, el sujeto realiza un esfuerzo mental activo orientado a iniciar o mantener el movimiento del exoesqueleto. En la segunda, el sujeto realiza un esfuerzo mental activo orientado a detener dicho movimiento.

El dataset utilizado en este trabajo contiene datos de 5 sujetos, cada uno con 5 sesiones de adquisición, 11 ensayos por sesión y 56 ventanas temporales por ensayo, como se detalla en la sección 3.1.3. Durante los distintos entrenamientos se dividen los ensayos de la sesión objetivo en dos subconjuntos: un conjunto de entrenamiento formado por los primeros 8 ensayos y un conjunto de validación formado por los 3 ensayos restantes. De este modo, se mantiene una proporción aproximada del 70 % para entrenamiento y del 30 % para validación.

De este modo, se evalúa la capacidad del modelo de aprender a partir de los primeros ensayos de un usuario y generalizar su conocimiento a los ensayos posteriores. Esta elección se basa en la hipótesis de que los sujetos pueden presentar una mejora progresiva en el control de la imaginación motora a lo largo de la sesión, por lo que evaluar el modelo sobre los últimos ensayos permite analizar su rendimiento en condiciones más estables. Además sirve como aproximación a un escenario real de uso, en el que los primeros ensayos de cada sesión se utilizan para un pre-entrenamiento del modelo, mientras que los últimos ensayos se evalúan a tiempo real en un sistema de bucle cerrado.

Las estrategias de entrenamiento seleccionadas se basan en los paradigmas descritos previamente en la sección 2.6.2. La diferencia principal entre ellas reside en qué datos se utilizan para inicializar, entrenar o ajustar el modelo en cada caso: datos de una única sesión, datos acumulados de sesiones anteriores, datos de otros usuarios o una combinación progresiva de estos enfoques.

### Entrenamiento intra sesión

El entrenamiento intra-sesión representa el enfoque tradicional en sistemas BCI. En esta estrategia se utilizan únicamente los datos de un único usuario y de una única sesión o día experimental. Es decir, para entrenar y validar el modelo correspondiente a un usuario  $u$  en un día  $d$ , no se emplea información procedente de otros usuarios ni de otras sesiones del mismo usuario.

Para cada combinación usuario-día se genera un modelo independiente desde cero. Los datos de la sesión se dividen en un conjunto de entrenamiento y un conjunto de validación utilizando el mismo procedimiento descrito anteriormente.

Por ejemplo, para el usuario  $U_1$  en el día  $D_1$ , el modelo se entrena únicamente con los ensayos de entrenamiento de  $U_1D_1$  y se valida con los ensayos restantes de esa misma sesión. Para el día  $D_2$ , se repite el proceso desde cero, generando un nuevo modelo independiente entrenado únicamente con los datos de  $U_1D_2$ . Este procedimiento se repite para todos los días y todos los usuarios.

Esta estrategia permite analizar el rendimiento del modelo cuando sólo se dispone de información específica de la sesión actual. Además, al repetir el mismo procedimiento para todos los días de adquisición, es posible estudiar la evolución del rendimiento a lo largo de los distintos días de adquisición. Esta evolución puede reflejar, entre otros factores, cambios en la calidad de la señal, variabilidad entre sesiones o una posible mejora progresiva del usuario en la realización de la tarea de imaginación motora.

### Entrenamiento entre sesiones

El entrenamiento entre sesiones tiene como objetivo evaluar si la información adquirida en sesiones anteriores de un mismo usuario puede mejorar el rendimiento del modelo en sesiones posteriores. A diferencia del entrenamiento intra sesión, en este caso el conjunto de entrenamiento no solo utiliza los datos del día actual, sino que incorpora progresivamente información de días anteriores del mismo usuario.

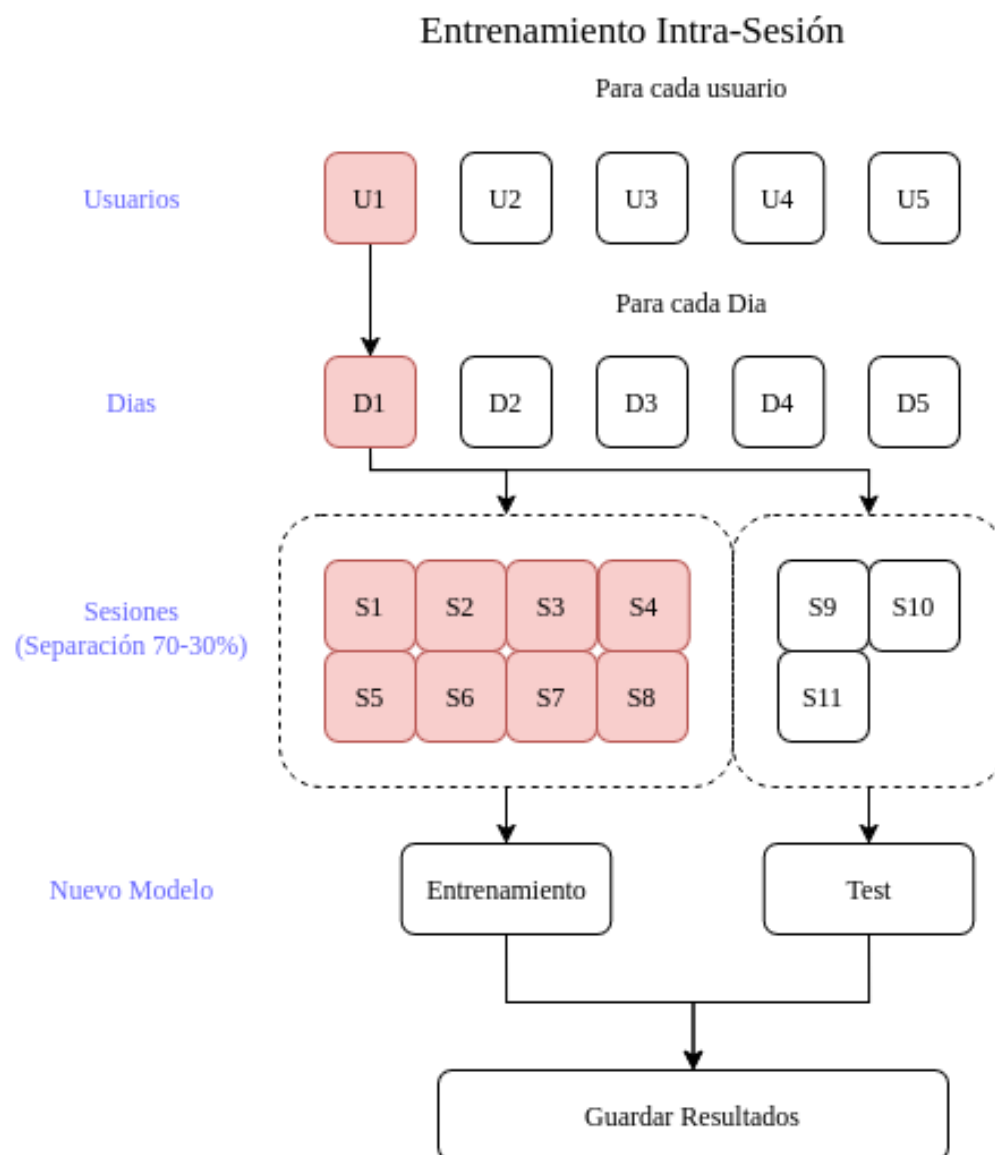


Figura 3.4: Esquema simplificado del método de entrenamiento intra sesión (Elaboración propia).

El primer día de cada usuario se trata de forma equivalente al entrenamiento intra sesión, ya que no existen sesiones previas disponibles. A partir del segundo día, el conjunto de entrenamiento se amplía. Para este día, un nuevo modelo se entrena utilizando los ensayos del  $D_1$  junto el conjunto de entrenamiento del  $D_2$  y se valida con los ensayos reservados del  $D_2$ . De forma análoga, para el día  $D_3$ , se utilizan los datos de entrenamiento del día actual y los datos acumulados de los días  $D_1$  y  $D_2$ . Este proceso continúa sucesivamente hasta el último día experimental, incrementando progresivamente la cantidad de información disponible para cada modelo. En todos los casos, la validación se realiza sobre los ensayos reservados del día que se está evaluando.

Esta estrategia permite estudiar si la acumulación de información intra-usuario mejora la adaptación del modelo a lo largo de la semana. Además, al igual que en el caso intra sesión, permite analizar la evolución temporal del rendimiento, pero considerando ahora un escenario en el que el modelo dispone cada vez de más información previa del mismo sujeto.

### **Entrenamiento entre usuarios**

El entrenamiento entre usuarios introduce una etapa previa de aprendizaje basada en información procedente de sujetos distintos al usuario objetivo. En esta estrategia se genera inicialmente un modelo general entrenado con los datos de todos los usuarios disponibles excepto el usuario que se desea evaluar. De este modo, el modelo inicial parte de una representación aprendida a partir de patrones presentes en otros sujetos.

Una vez obtenido el modelo general, se realiza un ajuste mediante *fine-tuning* sobre una copia de este modelo utilizando los datos de entrenamiento del usuario objetivo en la sesión correspondiente, siguiendo el esquema del entrenamiento intra-sesión. Es decir, para cada usuario y cada día, el modelo no parte de una inicialización aleatoria, sino de un modelo previamente entrenado con datos de otros usuarios. Posteriormente, dicho modelo se adapta con los ensayos de entrenamiento del usuario objetivo y se valida con los ensayos reservados de ese mismo día.



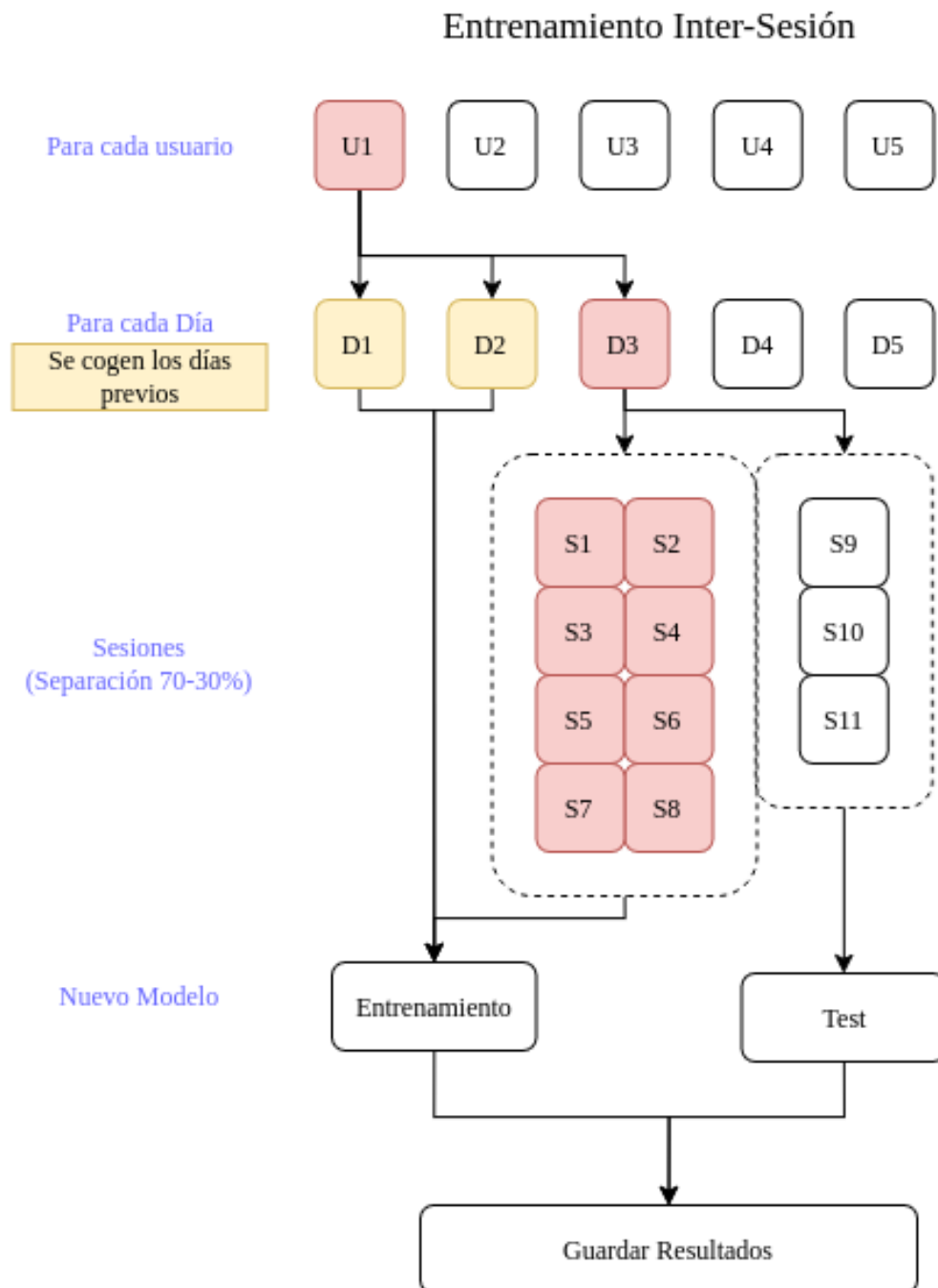


Figura 3.5: Esquema simplificado del método de entrenamiento entre sesiones (Elaboración propia).

Por ejemplo, si el usuario objetivo es  $U_1$ , el modelo inicial se entrena con los datos de los usuarios  $U_2, U_3, U_4, U_5$ . Después, para evaluar el día  $D_1$  de  $U_1$ , una copia del modelo general se ajusta con los ensayos de entrenamiento de  $U_1 D_1$  y se valida con los ensayos restantes de  $U_1 D_1$ . Para el día  $D_2$ , se parte de una nueva copia del modelo general entrenado con el resto de usuarios y se realiza un nuevo ajuste independiente con los datos de  $U_1 D_2$ . Por tanto, cada sesión del usuario objetivo genera un modelo adaptado independiente.

El objetivo de esta estrategia es analizar si un modelo pre-entrenado con datos de otros usuarios es capaz de transferir conocimiento útil al usuario objetivo. Esto resulta especialmente relevante en señales EEG, donde la variabilidad entre sujetos suele ser elevada y la cantidad de datos disponible por usuario puede ser limitada. Mediante esta estrategia se estudia si la información inter-usuario puede servir como punto de partida para mejorar el rendimiento frente a un entrenamiento realizado únicamente con datos del usuario objetivo.

### **Entrenamiento continuo**

El entrenamiento continuo combina la información inter-usuario con una adaptación progresiva al usuario objetivo. Al igual que en el entrenamiento entre usuarios, el proceso comienza con un modelo general entrenado con los datos del resto de usuarios, excluyendo al usuario que se desea evaluar. Este modelo inicial proporciona una base de conocimiento común obtenida a partir de sujetos distintos al usuario objetivo.

La diferencia principal respecto al entrenamiento entre usuarios es que, en este caso, el modelo no se reinicia para cada sesión. En lugar de generar un modelo adaptado independiente para cada día, el modelo se conserva entre sesiones y se actualiza progresivamente con los datos del usuario objetivo. De este modo, cada nuevo día parte del modelo ajustado en la sesión anterior, incorporando de forma acumulativa la información aprendida hasta ese momento.

Por ejemplo, para evaluar al usuario  $U_1$ , se genera primero un modelo general entre-

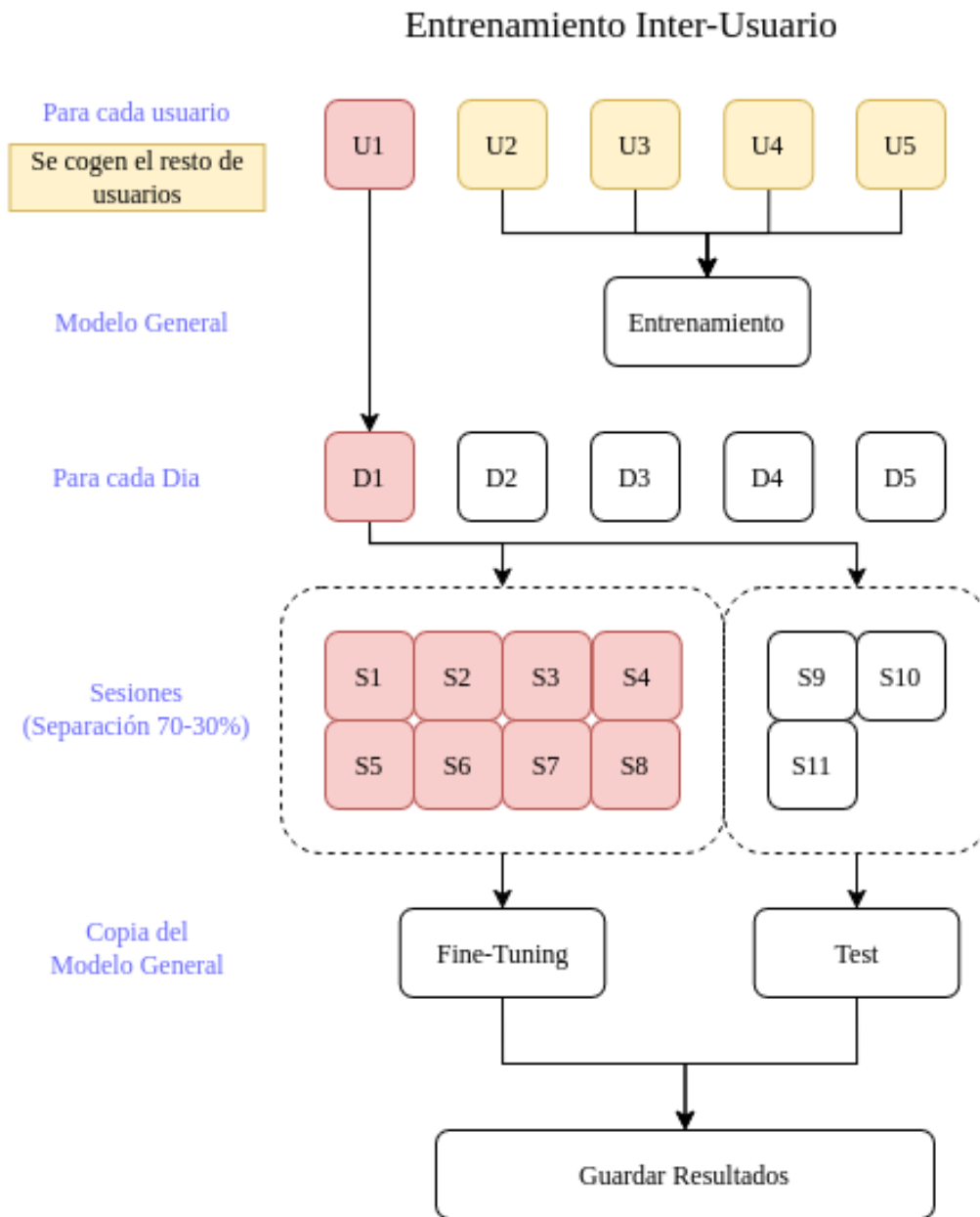


Figura 3.6: Esquema simplificado del método de entrenamiento entre usuarios (Elaboración propia).

nado con los datos de los usuarios  $U_2, U_3, U_4, U_5$ . En el día  $D_1$ , este modelo se ajusta con los ensayos de entrenamiento de  $U_1D_1$  y se valida con los ensayos reservados de esa misma sesión. Para el día  $D_2$ , no se vuelve al modelo general inicial, sino que se parte del modelo ya adaptado con  $U_1D_1$  y se continúa el entrenamiento con los datos de  $U_1D_2$ . Este procedimiento se repite durante el resto de sesiones, de forma que el modelo va acumulando progresivamente la información específica del usuario objetivo.

Esta estrategia permite estudiar un escenario de adaptación continua, en el que el sistema mejora gradualmente a medida que dispone de más datos del usuario. Desde el punto de vista de una aplicación real, este enfoque resulta especialmente interesante, ya que un sistema BCI podría partir de un modelo general entrenado con usuarios previos y adaptarse progresivamente al usuario final conforme se realizan nuevas sesiones de experimento.

#### 3.4.1.1 Resumen estrategias de entrenamiento

| Estrategia     | Datos iniciales                                   | Datos de adaptación                                       | Evolución entre días                           |
|----------------|---------------------------------------------------|-----------------------------------------------------------|------------------------------------------------|
| Intra sesión   | Ninguno, modelo inicializado desde cero           | Sólo datos del usuario y día actual                       | Modelos independientes por día                 |
| Inter Sesión   |                                                   | Datos del usuario actual acumulados hasta el día evaluado | Aumenta la cantidad de datos del mismo usuario |
| Inter usuarios | Modelo general entrenado con el resto de usuarios | Datos del usuario objetivo y día actual                   | Modelos adaptados independientes por día       |
| Continuo       |                                                   | Datos acumulados progresivamente del usuario objetivo     | El modelo se conserva y mejora entre días      |

Tabla 3.3: Resumen de las estrategias de entrenamiento empleadas.

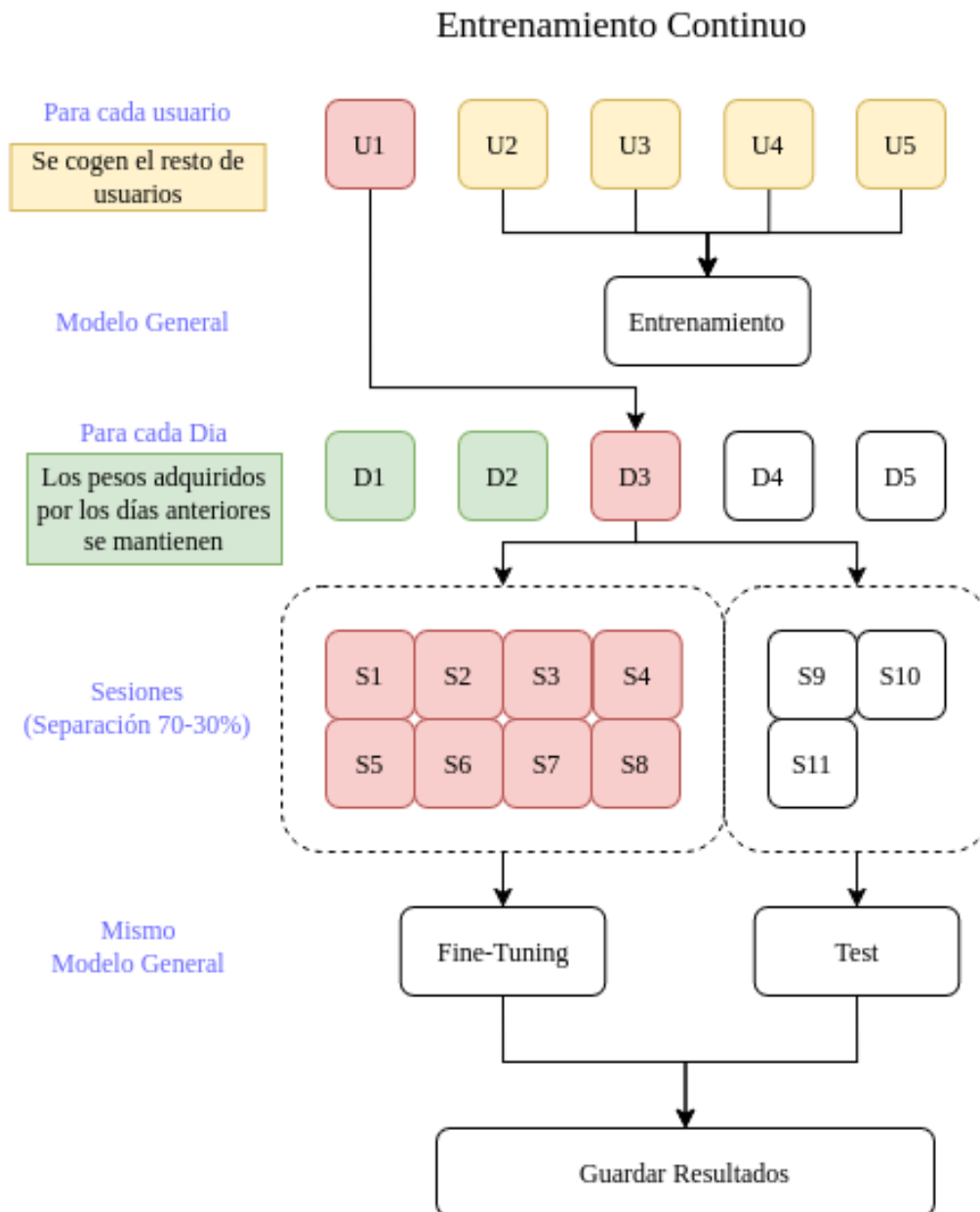


Figura 3.7: Esquema simplificado del método de entrenamiento continuo (Elaboración propia).

### 3.5. Infraestructura Experimental

La complejidad de orquestar distintas ejecuciones, junto con el elevado número de estas, hizo que durante las etapas iniciales se tomara la decisión de diseñar una infraestructura experimental común, modular y reutilizable, que permitiera modificar fácilmente la configuración de cada ejecución y automatizar la realización de múltiples experimentos de manera secuencial.

Para lograr esto, se ha definido un pipeline de ejecución que gestiona las diferentes etapas del proceso experimental: la carga de la configuración, el preprocesamiento de los datos, la inicialización de los modelos, su entrenamiento, la evaluación de los resultados y el almacenamiento de estos.

Con el objetivo de facilitar la reproducibilidad y la exploración sistemática de las variables de estudio, el código se estructuró como un framework experimental ligero que permite modificar de forma sencilla las variables de estudio, así como controlar la ejecución de los experimentos y mantener una estructura común para los diferentes métodos de entrenamiento.

Para entender la organización de las ejecuciones, se ha definido una estructura jerárquica en tres niveles:

- **Experimento:** nivel superior que agrupa el conjunto completo de configuraciones evaluadas. Un experimento consiste en la ejecución de distintas combinaciones de las variables experimentales, formadas por la arquitectura, el método de entrenamiento y el tipo de tarea, junto con una configuración de entrenamiento común.
- **Iteración:** ejecución individual del pipeline asociada a una combinación concreta de arquitectura, método de entrenamiento y tipo de tarea. En este nivel se carga el conjunto de datos correspondiente, se inicializa la configuración experimental y se ejecuta el método de entrenamiento
- **Entrenamiento:** nivel más bajo de la jerarquía, correspondiente al entrenamiento y evaluación de un modelo para un usuario y una sesión objetivo

determinados. La ejecución del entrenamiento y el uso de los datos depende del método de entrenamiento seleccionado. Por ejemplo, para el método de entrenamiento *intra-sesión* se utilizarán los datos de un solo día y para el *entre-sesiones* se utilizan los datos de las sesiones anteriores, los métodos de entrenamiento *entre-usuarios* y *continuo* incluyen el

De este modo, cada experimento recorre sistemáticamente todas las combinaciones definidas de arquitectura, método de entrenamiento y tipo de tarea. Es decir, se realiza una iteración para cada combinación. Además se utiliza un fichero auxiliar que contiene los distintos parámetros comunes del proceso de entrenamiento, como el número máximo de épocas, los callbacks utilizados o la métrica monitorizada.

En cada iteración, el sistema inicializa los parámetros de entrenamiento y carga el conjunto de datos correspondiente al tipo de tarea. A continuación, se ejecuta el método de entrenamiento seleccionado, que determina cómo se recorren los usuarios y las sesiones y qué datos se utilizan en cada caso. Para cada usuario y sesión objetivo se construye o recupera el modelo, se entrena y se evalúa sobre el conjunto de prueba.

Finalmente, los resultados obtenidos y el identificador de cada entrenamiento se almacenan de forma estructurada en una base de datos relacional usando el motor de DuckDB, permitiendo su posterior análisis. Un análisis detallado del esquema de la base de datos se presenta en el anexo A.

Para permitir la reanudación de la ejecución en caso de interrupción, se ha establecido un sistema de control donde se registra el estado de cada etapa del pipeline. De este modo, cuando se produce un error o interrupción, la ejecución puede reanudarse desde el punto en el que ocurrió.

Este proceso permite realizar un gran número de experimentos de manera secuencial y facilita el posterior estudio de los resultados obtenidos por cada combinación de parámetros definida.

La Figura 3.8 resume la organización jerárquica del pipeline experimental.

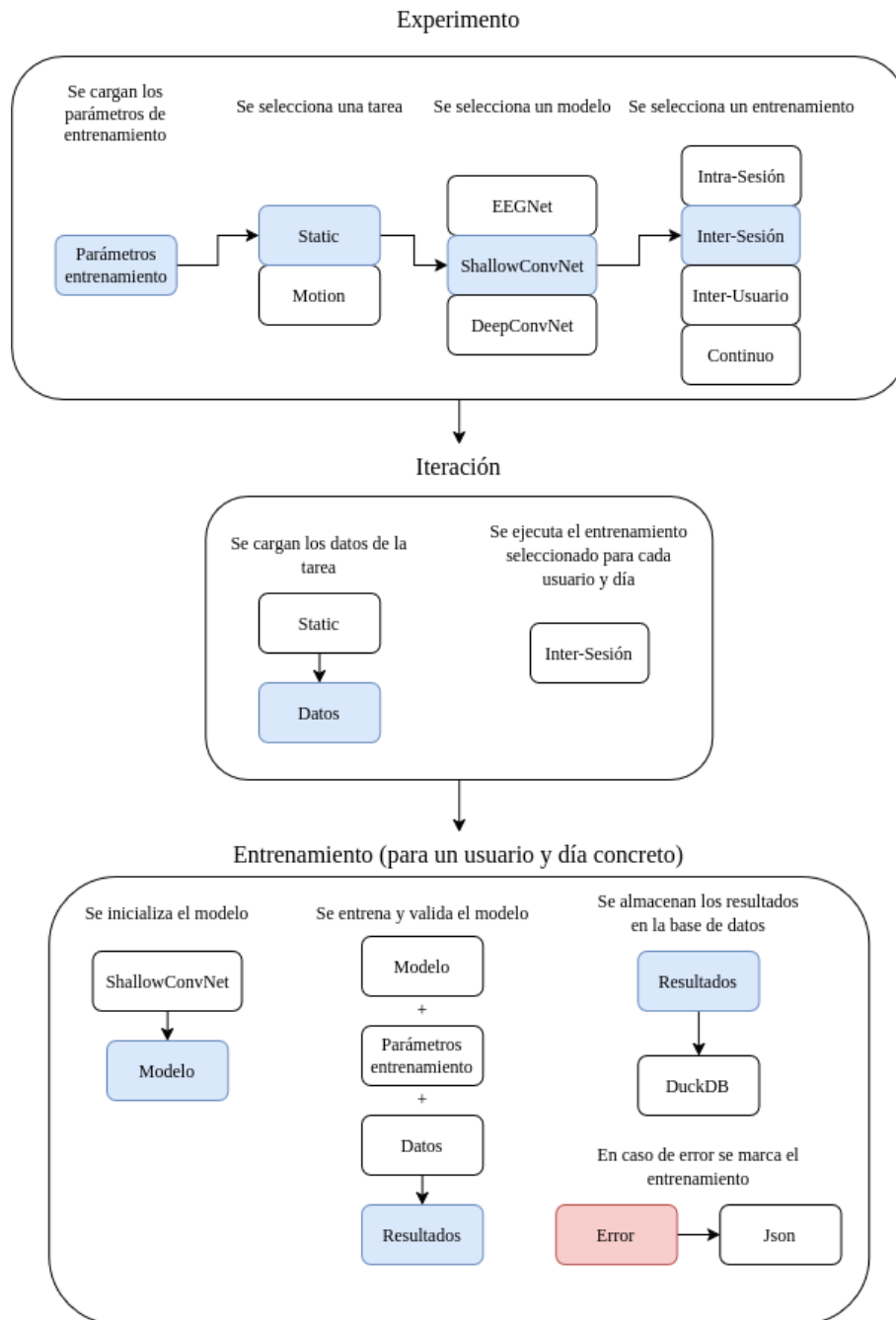


Figura 3.8: Pipeline experimental



#### 3.5.1. Parámetros de entrenamiento

Un paso importante en la implementación y estudio de modelos de *Machine Learning* es la elección de los distintos parámetros que se van a usar para definir la red neuronal y para realizar el entrenamiento.

Es importante diferenciar entre los hiperparámetros, relacionados con la estructura de cada red neuronal, y los parámetros del proceso de entrenamiento, que incluyen aspectos como el número máximo de épocas, el optimizador empleado, los callbacks utilizados y la métrica monitorizada para seleccionar el mejor modelo.

En este estudio no se ha realizado una optimización exhaustiva de los hiperparámetros de las redes, ya que el objetivo principal no era proponer una nueva arquitectura ni maximizar el rendimiento de un modelo concreto, sino comparar distintas estrategias de entrenamiento bajo una configuración común. Además, en las arquitecturas empleadas (EEGNet, ShallowConvNet y DeepConvNet) varios de sus parámetros se encuentran condicionados por la propia naturaleza de los datos, como el número de canales de entrada, la frecuencia de muestreo o la longitud temporal de las ventanas analizadas.

Por ejemplo, para la EEGNet se define una longitud de kernel convolucional igual a la mitad de la frecuencia de muestreo utilizada al obtener los datos, un número de canales de entrada igual al de canales EEG, etc [52].

Sin embargo, sí se han realizado varios experimentos preliminares modificando algunos parámetros relevantes con el objetivo de definir una configuración común de entrenamiento en los experimentos finales. En concreto, se analizaron diferentes valores para la tasa de aprendizaje inicial (0,1, 0,01 y 0,001), la tasa de *dropout* (0,2,0,3 y 0,4), el factor de reducción empleado por *ReduceLROnPlateau* (0,1, 0,01 y 0,001) y el umbral de mejora mínima (*min\_delta* del mismo *callback*) (0,1, 0,01 y 0,001).

Se realizaron los entrenamientos utilizando todo el conjunto de días de cada usuario. Sin embargo, dichas pruebas no mostraron mejoras consistentes en el rendimiento, por lo que se optó por mantener una configuración estable para todos los experi-

mentos. Esto permite reducir la variabilidad de la búsqueda de hiperparámetros y la complejidad del estudio, además de garantizar unas condiciones experimentales comunes para cada modelo y método de entrenamiento.

La configuración de entrenamiento empleada se muestra en la Tabla 3.4. El número máximo de épocas se establece en 500, aunque este valor actuó únicamente como límite superior, ya que la duración efectiva del entrenamiento estuvo controlada mediante el callback de *EarlyStopping*.

| Categoría                    | Parámetro                     | Valor                                   |
|------------------------------|-------------------------------|-----------------------------------------|
| <b>Configuración general</b> | Número máximo de épocas       | 500                                     |
|                              | Tasa de aprendizaje inicial   | 0,01                                    |
|                              | Tasa de <i>dropout</i>        | 0,4                                     |
|                              | Optimizador                   | Adam                                    |
|                              | Función de pérdida            | <i>Sparse Categorical Cross-Entropy</i> |
|                              | Tamaño de lote                | 32                                      |
| <b>ModelCheckpoint</b>       | Métrica monitorizada          | <b>val_accuracy</b>                     |
|                              | Criterio de selección         | Maximización                            |
|                              | Almacenamiento                | Mejores pesos                           |
| <b>EarlyStopping</b>         | Métrica monitorizada          | Exactitud de validación                 |
|                              | Paciencia                     | 40 épocas                               |
|                              | Mejora mínima                 | 0,001                                   |
|                              | Época de inicio               | 10                                      |
|                              | Restauración del mejor modelo | Sí                                      |
| <b>ReduceLROnPlateau</b>     | Métrica monitorizada          | <b>val_loss</b>                         |
|                              | Paciencia                     | 15 épocas                               |
|                              | Factor de reducción           | 0,1                                     |
|                              | Mejora mínima                 | $10^{-4}$                               |

Tabla 3.4: Parámetros de entrenamiento utilizados en el estudio.

Se utilizaron tres *callbacks*: *ModelCheckpoint*, *EarlyStopping* y *ReduceLROnPlateau*. El *ModelCheckpoint* se configuró para almacenar únicamente los pesos correspondientes al modelo que alcanzó la mayor precisión sobre el conjunto de validación.

El mecanismo de *EarlyStopping* funciona monitorizando también la precisión de validación. A partir de la época 10, el entrenamiento se detiene si esta métrica no presenta una mejora mínima de 0,001 durante 40 épocas consecutivas. Una vez detenido el proceso, se restauran los pesos de la mejor época. De esta forma, el

modelo que se utiliza durante la etapa de test no tiene por qué coincidir con el obtenido en la última época ejecutada.

Finalmente, la tasa de aprendizaje se adapta dinámicamente mediante el *callback ReduceLROnPlateau*. Cuando la pérdida de validación no mejora al menos en  $10^{-4}$  durante 15 épocas consecutivas, la tasa de aprendizaje se multiplica por un factor de 0,1.

Esta configuración se mantuvo constante entre los distintos experimentos. De esta manera, las diferencias observadas en los resultados pueden atribuirse principalmente a la arquitectura y a la estrategia de entrenamiento, en lugar de a variaciones en la configuración de entrenamiento.

La *semilla* se establece de manera aleatoria para cada experimento, evitando repetir las mismas condiciones de inicialización. Esta semilla controla distintos procesos del entrenamiento, como la inicialización de los pesos, el orden de selección de los lotes y la generación de las máscaras utilizadas por las capas de *dropout*. Al almacenar la semilla junto con el resto de los parámetros, es posible reproducir una ejecución concreta bajo las mismas condiciones experimentales.

El utilizar distintas semillas en la comparación entre modelos es esencial para poder evaluar la estabilidad de los resultados y evitar que las conclusiones dependan de una inicialización concreta. De este modo, puede determinarse con mayor certeza si una modificación de los hiperparámetros o de la arquitectura produce una mejora consistente, en lugar de atribuir esta mejora al azar.

## 4. Resultados y Discusión

Los resultados obtenidos durante la experimentación se han almacenado en una base de datos DuckDB, tal y como se describe en la Sección 3.5.

Se generaron tres tablas distintas en la base de datos donde almacenar los resultados de los experimentos realizados:

- **Runs\_t**: Contiene información general sobre cada ejecución del experimento, junto con las métricas obtenidas en el proceso de entrenamiento y validación.
- **Trials\_t**: Almacena los resultados obtenidos durante el test con ensayos individuales.
- **Classes\_t**: Contiene las métricas de cada una de las clases que componen los ensayos evaluados.

Por lo tanto, las tablas generadas contienen tanto los metadatos asociados al entrenamiento como las métricas obtenidas durante su evaluación.

Con el objetivo de reducir la influencia de posibles valores atípicos (*outliers*) sobre el análisis estadístico, se han creado vistas derivadas de cada tabla en las que se eliminan aquellos registros cuyos valores de *accuracy* se encuentran fuera de un intervalo de tres desviaciones estándar respecto a la media. Este procedimiento, basado en el método *Z-Score* [71], se define como:

$$z = \frac{x - \mu}{\sigma}$$

donde  $x$  representa el valor analizado,  $\mu$  la media de la distribución y  $\sigma$  la desviación estándar. De esta forma, se consideran valores atípicos aquellos que cumplen:

$$|z| > 3$$

Para más detalles sobre el proceso de eliminación de outliers, se puede consultar el Anexo C.1.

Además, también se han eliminado aquellos registros correspondientes a entrenamientos incompletos o interrumpidos, con el fin de garantizar la consistencia de los resultados analizados.

Tras aplicar este proceso de filtrado, el número total de registros válidos quedó reducido de la siguiente manera:

| Tabla     | Elementos Pre-Filtro | Elementos Post-Filtro | Outliers |
|-----------|----------------------|-----------------------|----------|
| runs_t    | 1917                 | 1917                  | 0        |
| trials_t  | 5751                 | 5739                  | 12       |
| classes_t | 15336                | 15218                 | 118      |

Tabla 4.1: Número de registros antes y después del proceso de filtrado.

Para mostrar los resultados obtenidos durante el experimento, se han desarrollado distintas gráficas utilizando la librería Vega-Altair y apoyándose en distintas queries de la base de datos sobre las tablas filtradas. Una descripción más detallada sobre el proceso de obtención de estas gráficas se encuentra en el Anexo B.

El análisis se organiza en tres partes. En primer lugar, se estudian por separado los resultados de la tarea estática y de la tarea de movimiento. Posteriormente, se realiza una comparación conjunta para identificar las diferencias asociadas a la tarea, la arquitectura, el usuario y el método de entrenamiento.

Como se ha mencionado en el apartado 3.5.1, los experimentos han sido realizados con los mismos parámetros de entrenamiento, variando únicamente la semilla. Por lo tanto, en las gráficas de resultados obtenidas se utiliza la media de las métricas obtenidas a partir de las distintas inicializaciones.

## 4.1. Tarea Estática

### 4.1.1. Comparación Global

La figura 4.1 muestra la media de la precisión y su desviación estándar para cada combinación de modelo y método de entrenamiento.

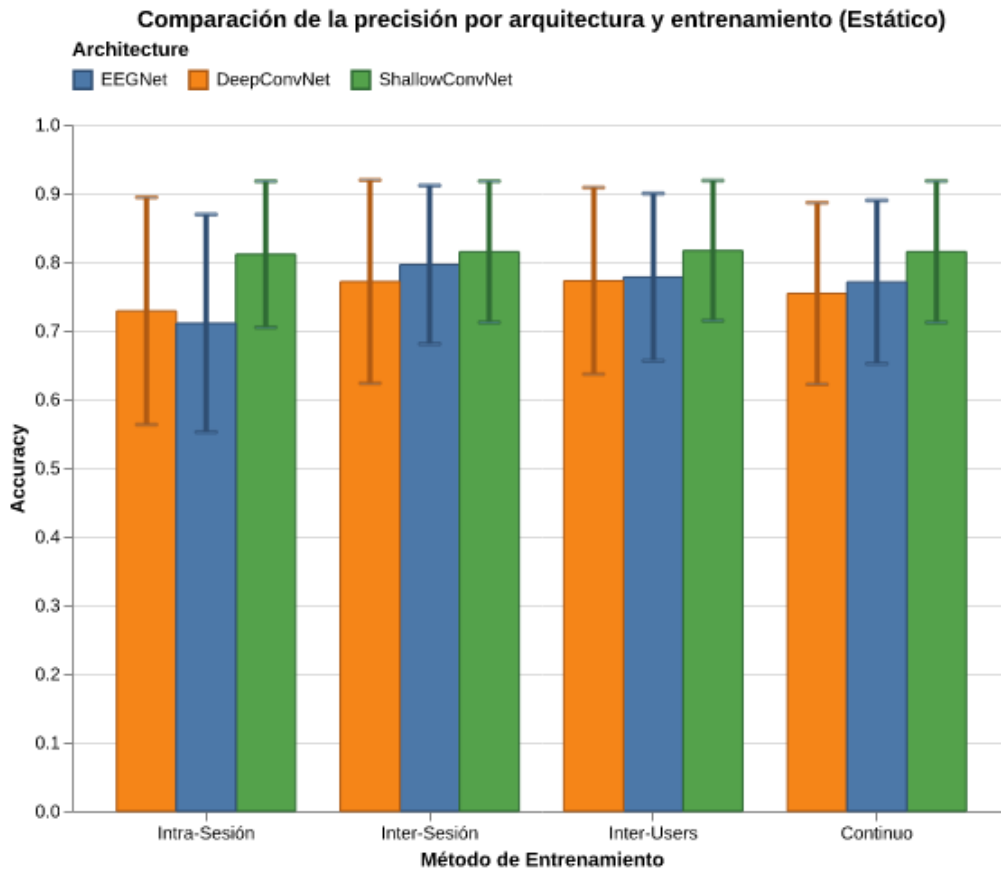


Figura 4.1: Comparación general de las distintas arquitecturas y métodos de entrenamiento en la tarea estática (media obtenida a partir de los 5 usuarios).

La arquitectura *ShallowConvNet* obtiene los mejores resultados para todos los métodos de entrenamiento considerados. Además, su precisión media se mantiene relativamente estable entre los distintos métodos, lo que puede indicar que su rendimiento depende en menor medida de la estrategia de entrenamiento utilizada.

Al contrario, *DeepConvNet* presenta generalmente los valores de precisión más bajos. La única excepción se produce en el entrenamiento *Intra-Sesión*, donde supera ligeramente a *EEGNet*. Esta diferencia es reducida y no se mantiene en el resto de métodos, por lo que no se pueden obtener conclusiones sólidas.

Para los modelos *EEGNet* y *DeepConvNet*, el entrenamiento *Intra-Sesión* muestra una precisión algo inferior y una mayor desviación estándar que los otros métodos que sí incorporan datos de otras sesiones y usuarios. Sin embargo, las diferencias entre los otros tres métodos son pequeñas en comparación con las observadas entre arquitecturas. Por tanto, estos resultados no permiten identificar un método de entrenamiento claramente superior para todas las redes estudiadas.

#### 4.1.2. Variabilidad Entre Usuarios

La figura 4.2 muestra la media de la precisión y su desviación estándar por usuario y modelo para la tarea estática.

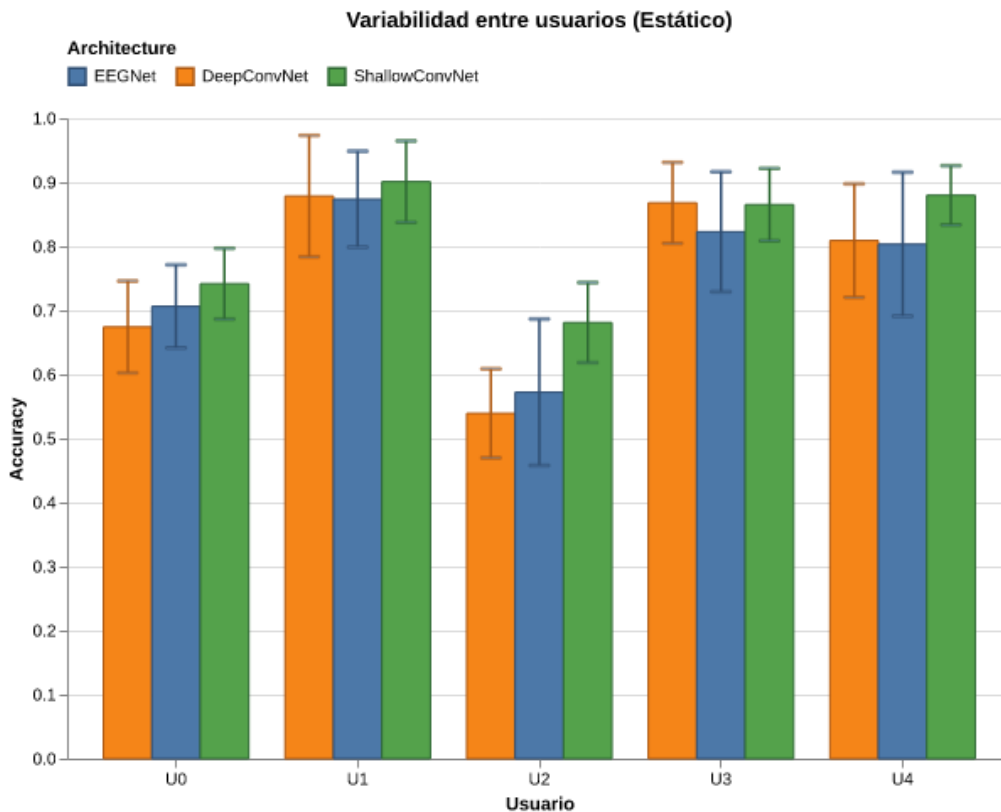


Figura 4.2: Variabilidad entre usuarios en la tarea estática (media obtenida a partir de los métodos de entrenamiento).

Las diferencias entre usuarios son considerablemente mayores que las observadas entre métodos de entrenamiento. El usuario 1 presenta los mejores resultados generales, mientras que el usuario 2 obtiene los valores más bajos. La diferencia entre ambos es de  $\approx 40\%$  en la precisión media.

Esta variabilidad se observa en las tres arquitecturas, incluida *ShallowConvNet*. En la gráfica anterior todos los modelos presentaban un comportamiento más estable. Por tanto, se puede establecer una correlación mucho más fuerte entre los usuarios y el modelo que entre el método de entrenamiento y el modelo.

También destacan los resultados usuario 3, para el cual *DeepConvNet* obtiene resultados mejores a los de las otras arquitecturas, mientras que en el resto de casos *ShallowConvNet* obtiene mejores resultados con una gran diferencia. Este resultado parece apuntar a la existencia de interacciones entre las características particulares de cada sujeto y la arquitectura empleada.

### 4.1.3. Mejor Configuración

La figura 4.3 muestra todas las configuraciones de modelo y método de entrenamiento con su respectiva media de la precisión y su desviación estándar.

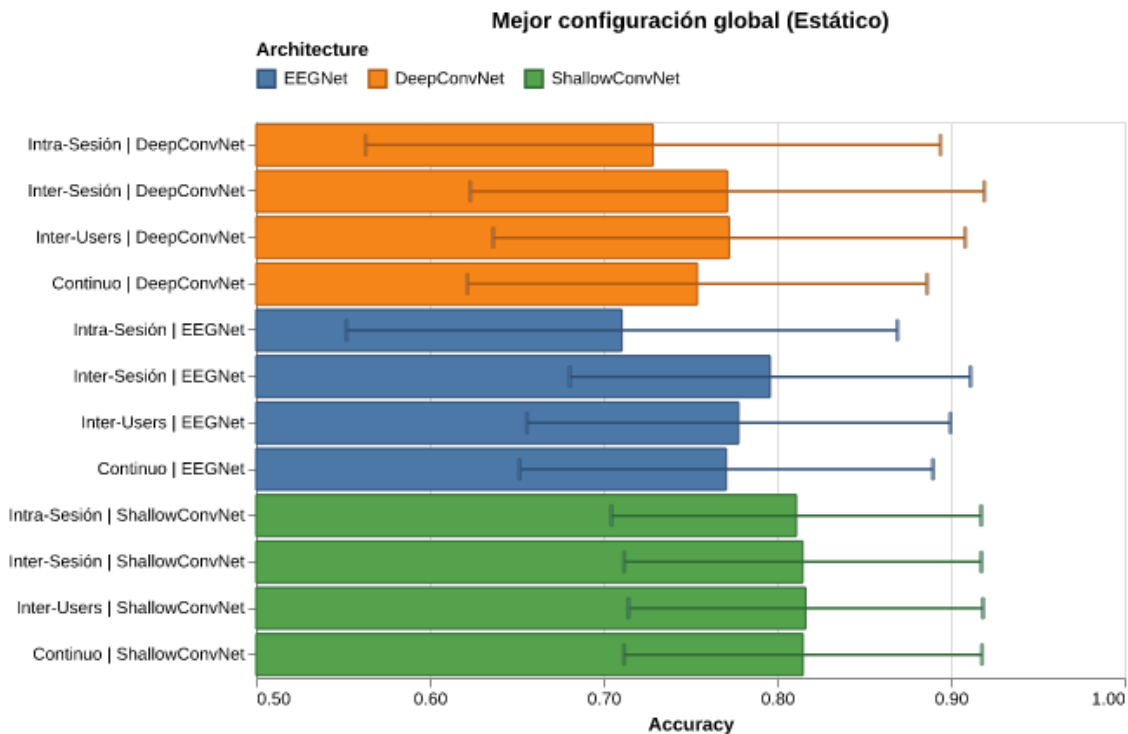


Figura 4.3: Mejores configuraciones globales en la tarea estática (media obtenida a partir de los 5 usuarios).

Esta representación confirma que las configuraciones que usan el modelo *ShallowConvNet* obtienen mejor precisión media, independientemente del método de en-



trenamiento. Las diferencias entre estas configuraciones son mínimas, por lo que no puede seleccionarse una estrategia de entrenamiento concreta como claramente superior.

Por el contrario, la elección de la arquitectura sí produce una separación más evidente. *DeepConvNet* y *EEGNet* tienen peores resultados, aunque *EEGNet* presenta una ligera mejora frente a *DeepConvNet*.

## 4.2. Tarea Movimiento

### 4.2.1. Comparación de Global

La figura 4.4 presenta la precisión media y la desviación estándar de cada arquitectura y método de entrenamiento para la tarea de movimiento.

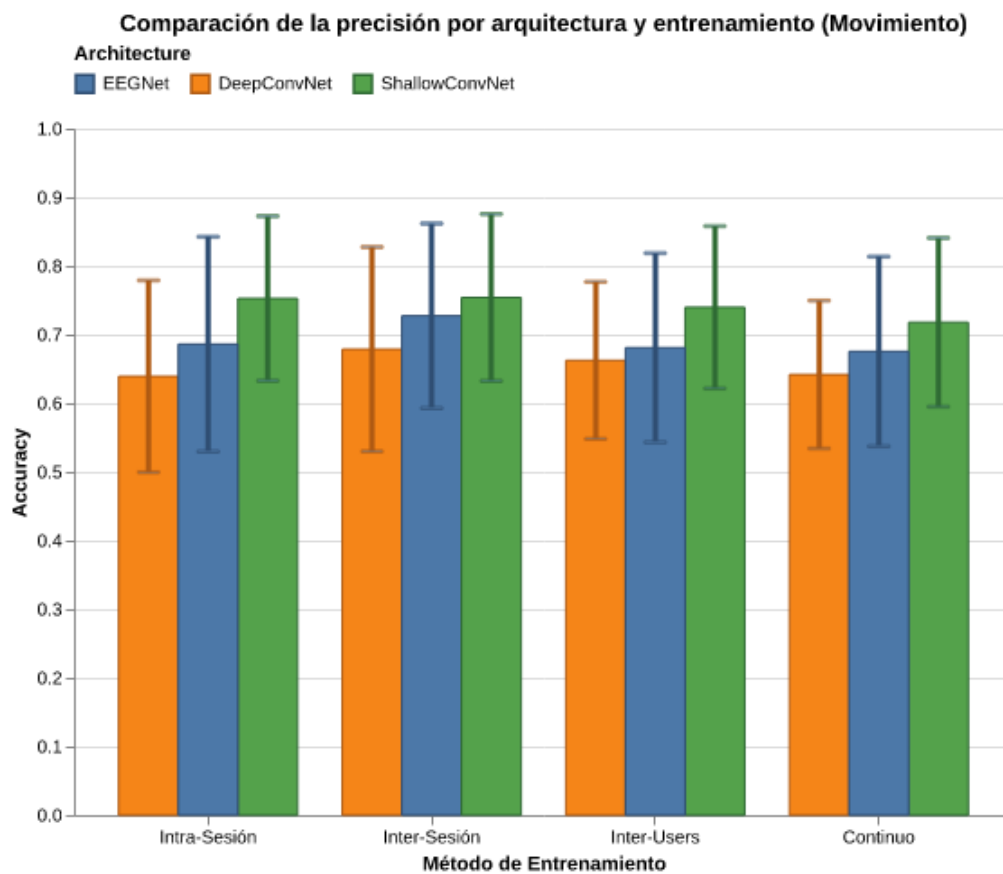


Figura 4.4: Comparación general de las distintas arquitecturas y métodos de entrenamiento en la tarea de movimiento (media obtenida a partir de los 5 usuarios).

La distribución de los resultados es similar a la observada en la tarea estática, aunque con una una peor precisión y una mayor de la dispersión.

*ShallowConvNet* continúa siendo la arquitectura con mejor rendimiento para todos los métodos de entrenamiento, seguida por *EEGNet* y, finalmente, por *DeepConvNet*. Para esta tarea, *EEGNet* presenta una mayor diferencia respecto a *DeepConvNet*, la cual no supera a los demás modelos para ninguno de los métodos de entrenamiento. Esto puede indicar que la pequeña ventaja observada para esta red en el entrenamiento *Intra-Sesión* de la tarea estática no se trata de una tendencia general.

Nuevamente, las diferencias entre los distintos métodos de entrenamiento son pequeñas y no siguen un patrón consistente entre arquitecturas. Aunque algunos métodos presentan pequeñas mejoras en la precisión media, estas variaciones se pueden atribuir a una mayor dispersión de los resultados. Por este motivo, para la tarea de movimiento tampoco se puede establecer una estrategia de entrenamiento superior.

#### **4.2.2. Variabilidad Entre Usuarios**

La figura 4.5 representa los resultados de cada usuario y arquitectura para la tarea de movimiento.

Los resultados son, en general, peores a los obtenidos en la tarea estática. Sin embargo, la caída de rendimiento no se produce igual para todos los sujetos.

En este caso, el usuario 4 alcanza la mayor precisión media, mientras que los usuarios 0 y 3 muestran una disminución considerable respecto a sus resultados en la tarea estática.

#### **4.2.3. Mejor Configuración**

La figura 4.6 ordena las configuraciones evaluadas para la tarea de movimiento.

Las configuraciones basadas en *ShallowConvNet* vuelven a obtener la mejor precisión media. No obstante, sus resultados son inferiores y presentan una mayor dispersión

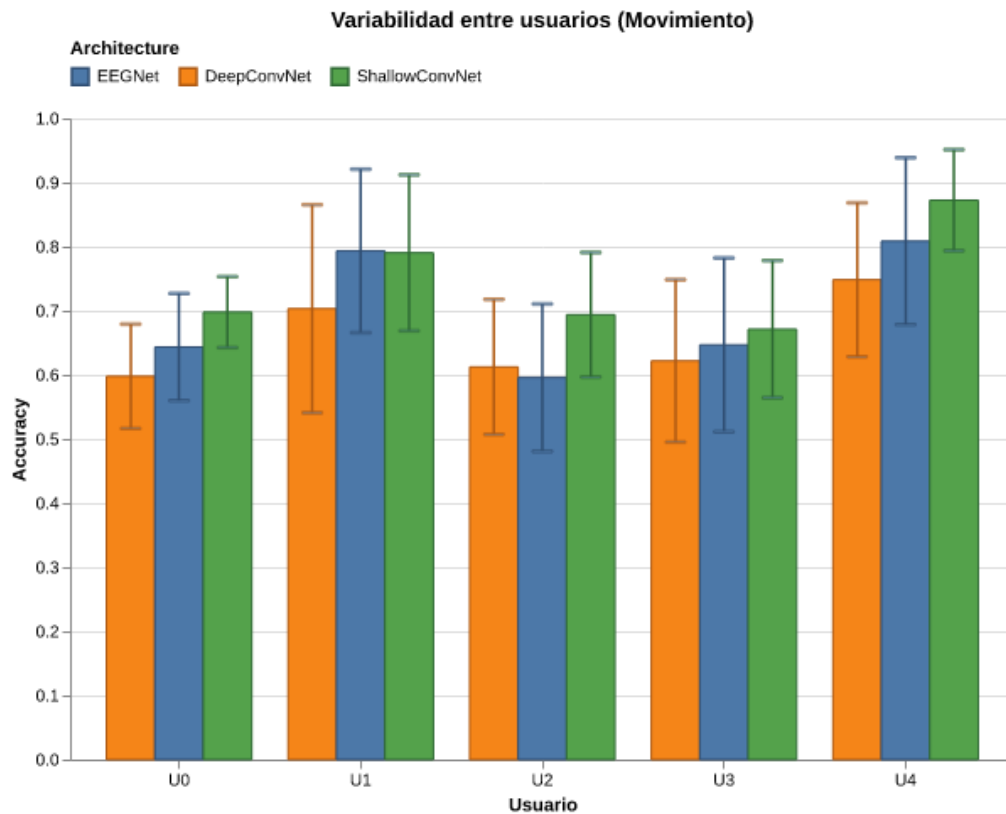


Figura 4.5: Variabilidad entre usuarios en la tarea de movimiento (media obtenida a partir de los métodos de entrenamiento).

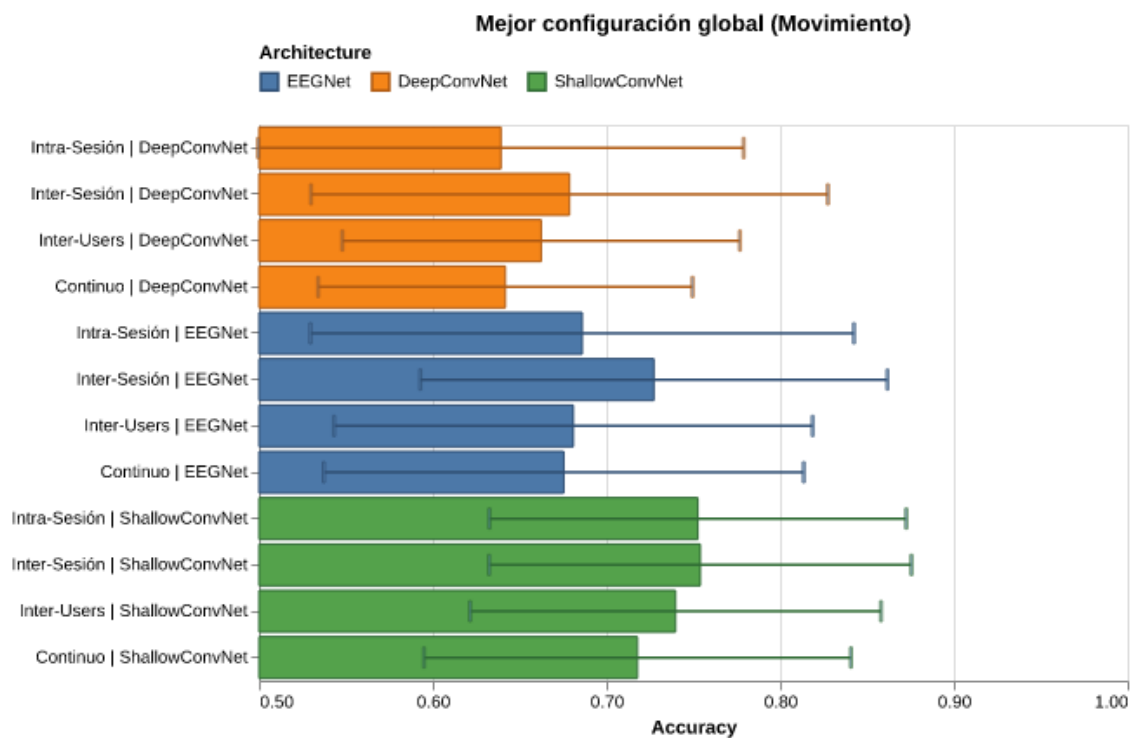


Figura 4.6: Mejores configuraciones globales en la tarea de movimiento (media obtenida a partir de los 5 usuarios).

que en la tarea estática.

La clasificación de las arquitecturas se mantiene a pesar del cambio de tarea, lo que proporciona más evidencias sobre una mayor influencia de la arquitectura respecto al método de entrenamiento.

### 4.3. Comparación entre usuarios

Las Figuras 4.7, 4.8, 4.9, 4.10 y 4.11 muestran la precisión media y su desviación estándar para cada usuario, arquitectura y método de entrenamiento, separando los resultados obtenidos en la tarea estática y en la tarea de movimiento.

Esta representación permite analizar con mayor detalle las diferencias entre participantes y observar cómo varía el rendimiento de cada arquitectura y método de entrenamiento en función del usuario y de la tarea evaluada.

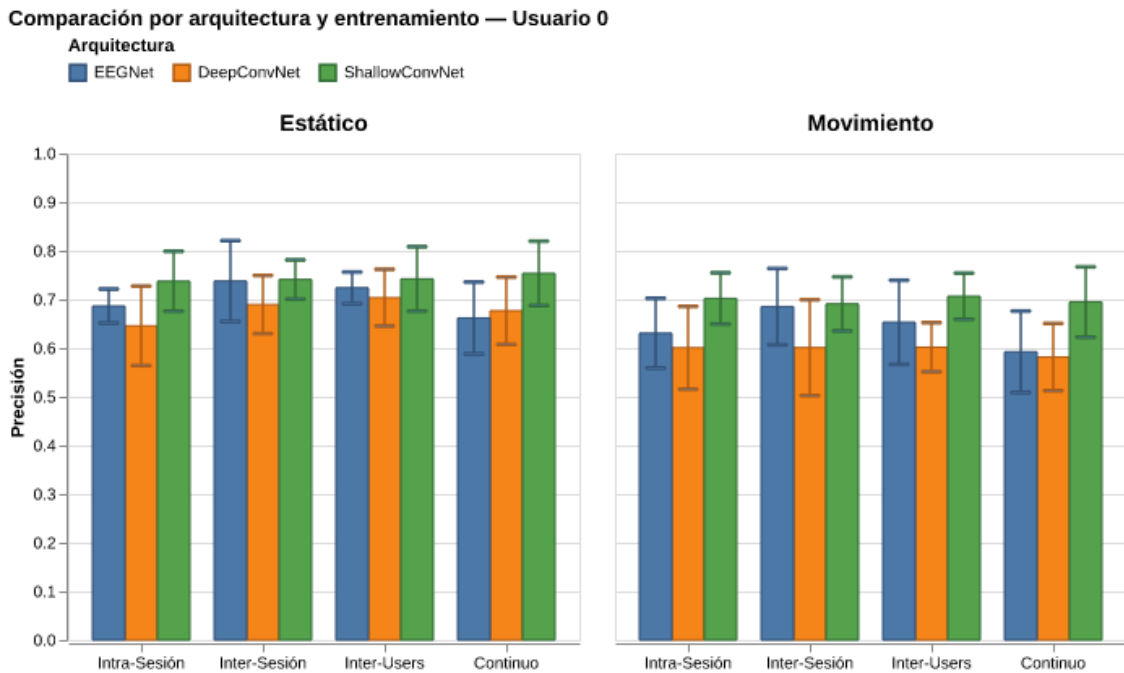


Figura 4.7: Comparación entre arquitecturas y entrenamientos para el usuario 0 en la tarea estática y en la de movimiento.

Los resultados muestran comportamientos diferentes entre los participantes. El usuario 3 presenta la mayor caída en precisión para la tarea de movimiento. En cambio, el usuario 4 mantiene valores similares en ambas tareas e incluso obtiene una precisión

ligeramente superior en la tarea de movimiento para algunas configuraciones.

**Comparación por arquitectura y entrenamiento — Usuario 1**

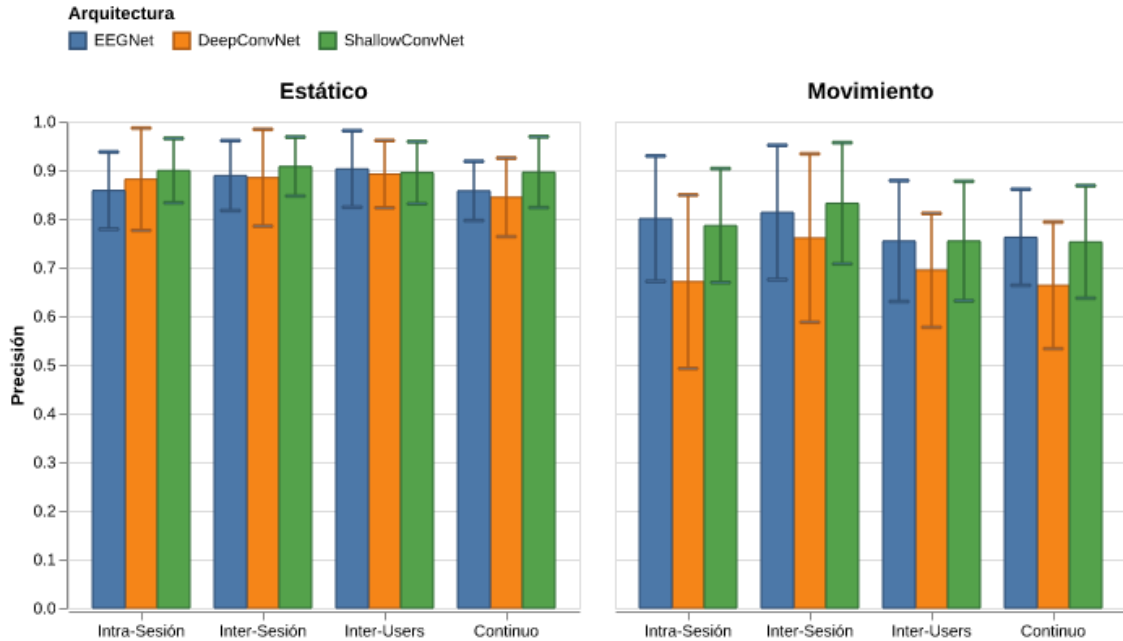


Figura 4.8: Comparación entre arquitecturas y entrenamientos para el usuario 1 en la tarea estática y en la de movimiento.

El usuario 2 mantiene un rendimiento claramente inferior al resto de participantes en ambas tareas. Sin embargo, es el único usuario para el que se observa una pequeña mejora general en la tarea de movimiento respecto a la tarea estática. El usuario 1 alcanza los mejores resultados en la tarea estática, aunque en la tarea de movimiento su precisión queda por debajo de la obtenida por el usuario 4.

Las diferencias entre tareas también varían en función de la arquitectura. *DeepConvNet* y *EEGNet* presentan cambios más grandes entre la tarea estática y la de movimiento. En particular, *DeepConvNet* presenta una mayor reducción relativa en la tarea de movimiento para varios usuarios. Por el contrario, *ShallowConvNet* mantiene, de forma general, un rendimiento más estable entre ambas tareas.

En cuanto a los métodos de entrenamiento, no se aprecia un comportamiento común a todos los participantes. La configuración que obtiene los mejores resultados cambia según el usuario, la arquitectura y la tarea, sin que ninguno de los métodos destaque de forma consistente en todas las condiciones analizadas.

Comparación por arquitectura y entrenamiento — Usuario 2

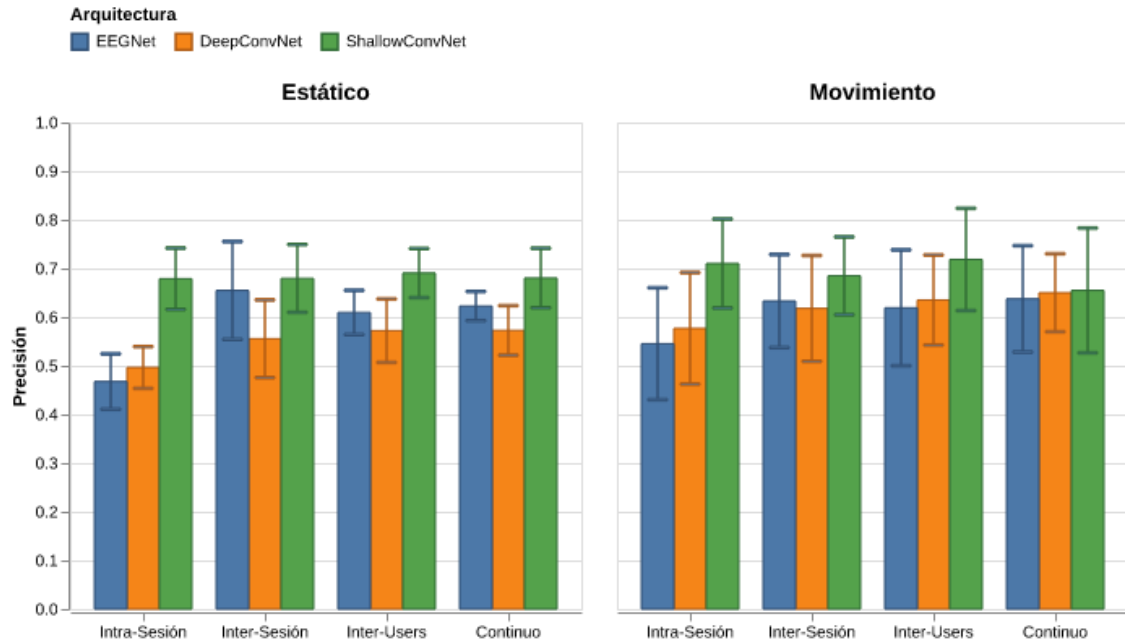


Figura 4.9: Comparación entre arquitecturas y entrenamientos para el usuario 2 en la tarea estática y en la de movimiento.

Comparación por arquitectura y entrenamiento — Usuario 3

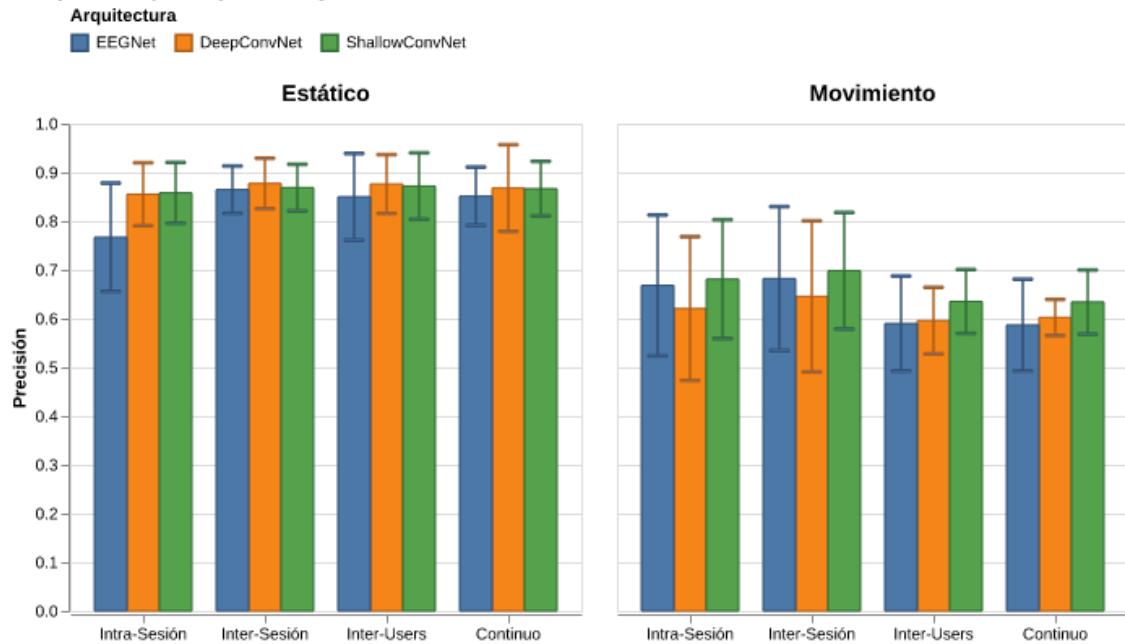


Figura 4.10: Comparación entre arquitecturas y entrenamientos para el usuario 3 en la tarea estática y en la de movimiento.

Comparación por arquitectura y entrenamiento — Usuario 4

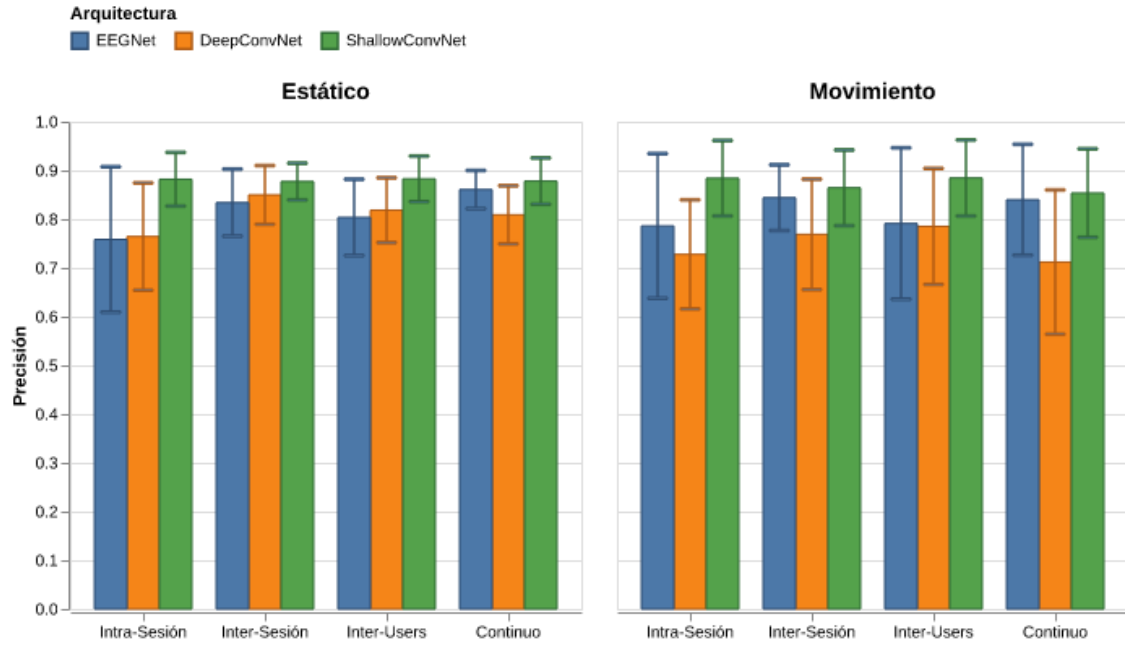


Figura 4.11: Comparación entre arquitecturas y entrenamientos para el usuario 4 en la tarea estática y en la de movimiento.

#### 4.4. Comparación entre tareas

La Figura 4.12 muestra la evolución de la precisión media para cada usuario y día, separando los resultados correspondientes a la tarea estática y a la tarea de movimiento. La línea horizontal representa la precisión media global de cada tarea y se utiliza como referencia para interpretar la evolución de cada participante.

La tarea de movimiento presenta una mayor desviación estándar que la tarea estática para la mayor parte de los usuarios. Esta mayor dispersión indica que su rendimiento es menos estable entre ejecuciones, usuarios y días.

Además, no se observa una evolución temporal común a todos los participantes. Para el usuario 1, la precisión aumenta durante los primeros días para ambas tareas, mientras que el usuario 3 muestra una evolución descendente en la tarea estática y una precisión constante durante la tarea de movimiento. Por otra parte, el usuario 0 mantiene resultados relativamente estables en ambas tareas.

Al igual que en las otras gráficas de comparación entre usuarios, el usuario 2 muestra un rendimiento claramente inferior a la media global de ambas tareas. Ocurre lo

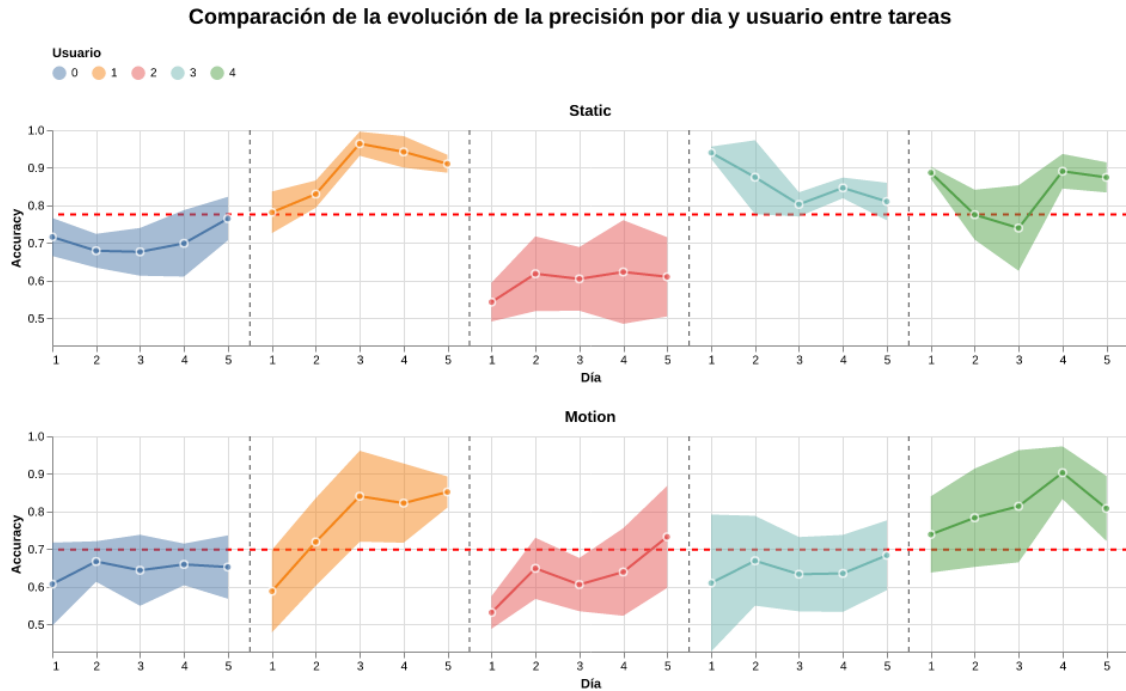


Figura 4.12: Variabilidad entre usuarios y día para cada tarea (media obtenida a partir de los métodos de entrenamiento y los modelos).

mismo con el usuario 0.

La ausencia de una tendencia general indica que la incorporación de otras sesiones no produce necesariamente una mejora continua de la precisión. El efecto de añadir datos puede depender de la similitud entre las sesiones, de la variabilidad diaria de la señal y de las características de cada usuario.

Por tanto, el día de adquisición no parece estar relacionado con una evolución del rendimiento. Su efecto debe interpretarse conjuntamente con el usuario, la tarea y la arquitectura utilizada.

#### 4.4.1. Rendimiento global de las tareas

La Figura 4.13 compara directamente la precisión media obtenida en las tareas estática y de movimiento según el modelo y el método de entrenamiento.

La tarea estática alcanza una precisión media global de  $\approx 77\%$ , frente a un valor de  $\approx 70\%$  en la tarea de movimiento. Esto supone una diferencia de 7 puntos porcentuales sobre la tarea de movimiento.



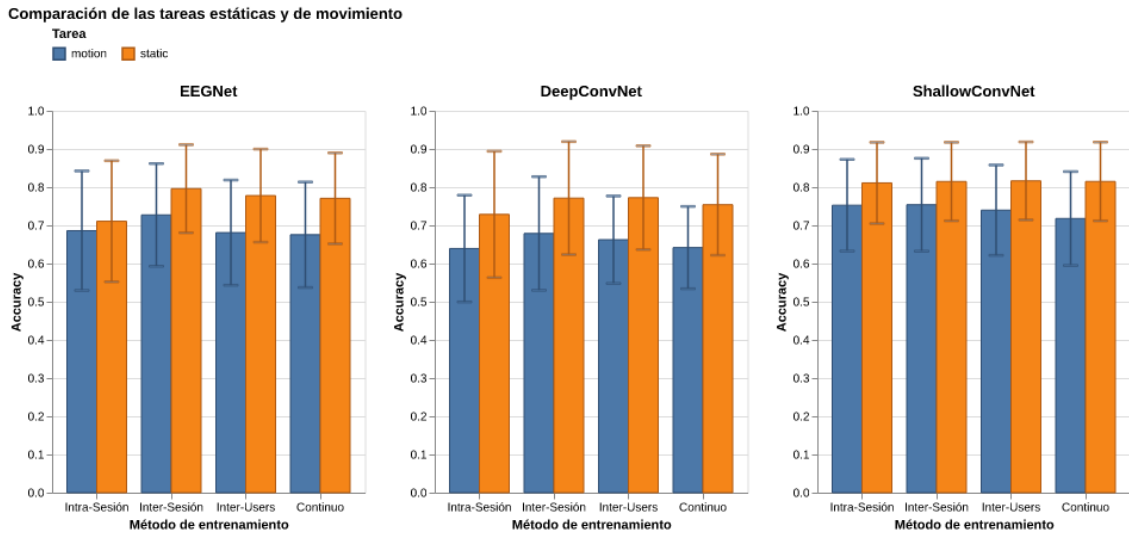


Figura 4.13: Comparación de la precisión entre la tarea estática y la de movimiento (media obtenida a partir de los 5 usuarios).

Esta diferencia se mantiene de forma general los distintos modelos y métodos de entrenamiento, aunque varía ligeramente en función de la configuración. Además, ambas tareas presentan valores de desviación estándar similares en esta agrupación, por lo que la menor precisión observada en la tarea de movimiento no constituye un aumento de la dispersión global.

### 4.5. Comparación entre métodos de entrenamiento

La figura 4.14 presenta la evolución de la precisión por usuario y día, agrupando los resultados según el método de entrenamiento. La línea horizontal de cada sub-gráfica representa la media de precisión global para todos los métodos de entrenamiento, esto permite una comparación directa entre los distintos métodos.

quitar esta y poner las cuatro redes. después evaluamos la media de los tres ensayos evaluados sobre el modelo elegido.

No se observa un diferencias notables entre los distintos métodos de entrenamiento en cuanto a la evolución de la precisión durante los distintos días de la semana. Las curvas presentan comportamientos similares para todos los los métodos y sus diferencias parecen depender principalmente del usuario y del día analizado.

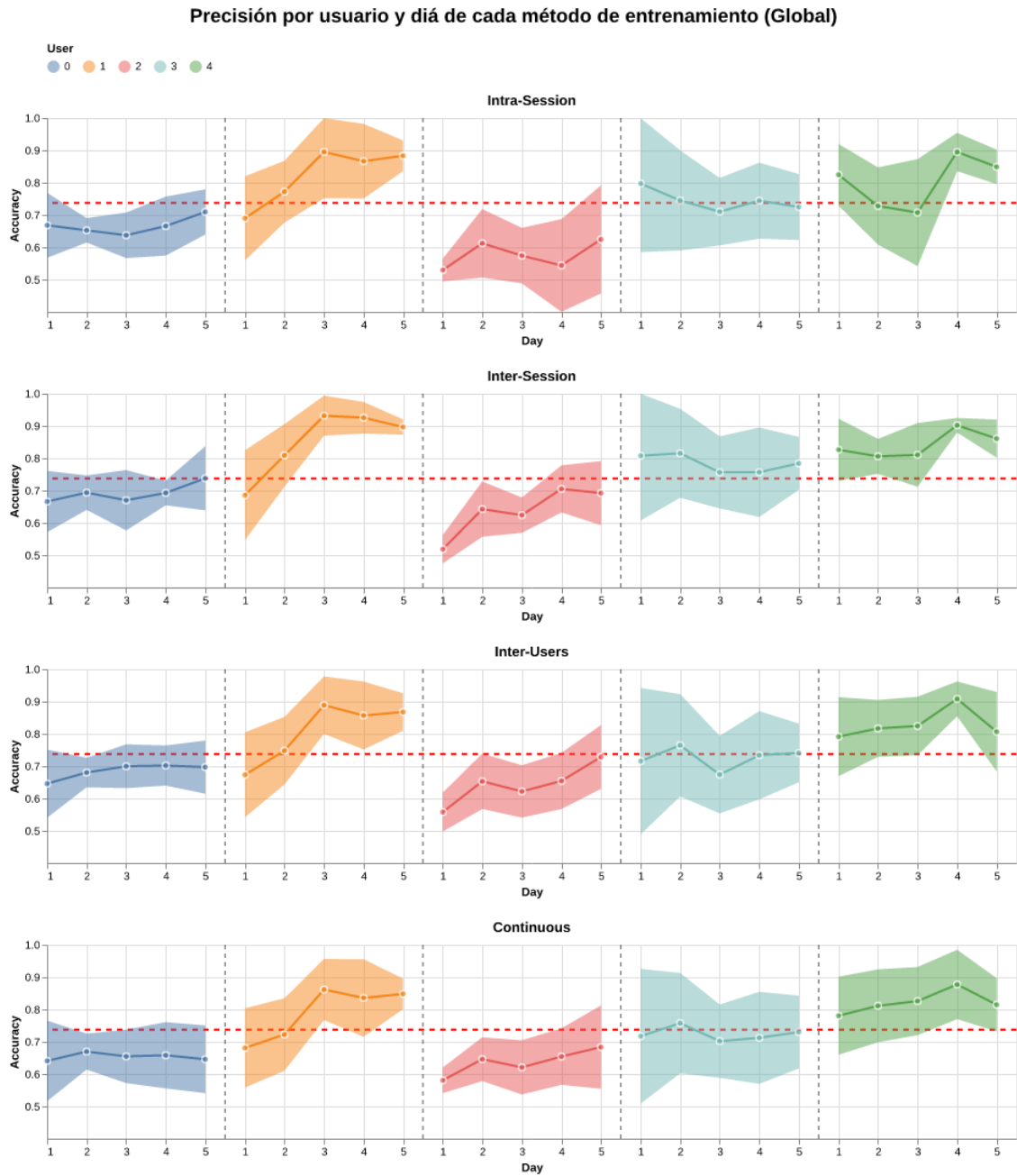


Figura 4.14: Variabilidad entre usuarios y día para cada método de entrenamiento (media obtenida a partir de los modelos).

El método *Inter-Sesión* alcanza una precisión media ligeramente superior y una menor desviación estándar para los cinco usuarios, salvo el usuario 3, que mantiene una desviación estándar similar en todos los métodos. Este resultado podría indicar que el incorporar información procedente de sesiones anteriores de un mismo usuario aporta una pequeña mejora en la estabilidad del modelo. Sin embargo, esta ventaja es reducida.

Los métodos *Inter-Users* y *Continuous* no producen una mejora claramente diferenciada respecto al entrenamiento *Intra-Sesión*. El uso de un modelo inicial entrenado con otros usuarios no garantiza un aumento de la precisión tras su adaptación al usuario objetivo.

En el Anexo B se incluyen gráficas adicionales que muestran la evolución de la precisión por usuario y día para cada arquitectura.

## 5. Conclusiones

En este trabajo se ha estudiado el rendimiento de tres arquitecturas de redes neuronales convolucionales, *EEGNet*, *DeepConvNet* y *ShallowConvNet*, para la clasificación de señales EEG asociadas a las tareas estática y de movimiento durante el uso de un exoesqueleto. Asimismo, se han comparado cuatro estrategias de entrenamiento: *Intra-Sesión*, *Inter-Sesión*, *Inter-Usuarios* y *Continuo*.

Hay estudios que señalan que el rendimiento de los modelos de DeepLearning para EEG depende en gran medida del conjunto de datos, del paradigma experimental y de la estrategia de entrenamiento utilizada. Por este motivo es difícil establecer una arquitectura o un método de entrenamiento que sea superior en todas las aplicaciones, sino que debe evaluarse bajo las condiciones concretas de cada problema [51].

Los resultados obtenidos muestran, de manera general, que la arquitectura, la tarea analizada y el usuario tienen una mayor influencia sobre el rendimiento que el método de entrenamiento. Aunque se observan diferencias entre las estrategias de entrenamiento, estas no se mantienen de forma consistente entre todas las arquitecturas, tareas y participantes.

### 5.1. Influencia de la arquitectura

La arquitectura *ShallowConvNet* presenta el mejor rendimiento global, con una precisión media aproximada del 76.6 %. *EEGNet* obtiene un resultado intermedio, cercano al 71.4 %, mientras que *DeepConvNet* alcanza la precisión media más baja, con aproximadamente un 69.4 %.

La superioridad de *ShallowConvNet* se mantiene entre las dos tareas y los diferentes métodos de entrenamiento. Además, esta arquitectura muestra un comportamiento relativamente estable frente a los cambios de configuración, aunque presenta una variabilidad considerable entre usuarios.

Por su parte, *EEGNet* obtiene resultados intermedios en la mayoría de las configuraciones. El estar orientado a diferentes paradigmas BCI puede favorecer su capacidad de generalización, aunque la escasez del conjunto de datos utilizado en este estudio puede hacer que esta arquitectura no alcance el rendimiento esperado.

*DeepConvNet* presenta generalmente los peores resultados y una mayor reducción de precisión en la tarea de movimiento. Una posible explicación es que su mayor número de parámetros requiera una mayor cantidad de datos para aprender y generalizar a nuevas observaciones. Es posible que esta arquitectura no haya podido obtener el rendimiento esperado debido al número limitado de participantes y sesiones disponibles,

Sin embargo, también hay que tener en cuenta que se ha utilizado una configuración de hiperparámetros común en todos los experimentos. Aunque esta decisión permite reducir la complejidad del estudio, puede favorecer a determinados modelos, ya que cada arquitectura podría requerir una configuración específica para obtener su rendimiento óptimo.

A falta de una búsqueda de parámetros exhaustiva, los resultados indican que la elección de la arquitectura tiene una influencia más evidente que la elección del método de entrenamiento.

### 5.2. Influencia de la tarea

La tarea estática alcanza una precisión media global aproximada del 76.0 %, mientras que la tarea de movimiento obtiene un valor cercano al 69.1 %. Esto supone una diferencia media de aproximadamente siete puntos porcentuales a favor de la tarea estática.

La mayoría de configuraciones de arquitecturas, métodos de entrenamiento y usuarios analizados presentan una mayor precisión en la tarea estática. Por tanto, los resultados indican que la tarea de movimiento presenta una mayor dificultad de generalización.

Una posible explicación es que los patrones EEG asociados a la tarea de movimiento sean más variables o menos diferenciables que los relacionados con la tarea estática. Los posibles factores que pueden contribuir a esta diferencia son: el cambio en la concentración de los participantes, la presencia de artefactos o las características concretas del protocolo de adquisición.

Sin embargo, los resultados obtenidos no permiten determinar cuál de estos factores es el responsable de la diferencia observada. Para ello sería necesario realizar un análisis más específico de las señales y de las condiciones experimentales de cada tarea.

### 5.3. Variabilidad entre usuarios

El propio usuario se presenta como uno de los factores con mayor influencia sobre los resultados. La desviación estándar de las precisiones medias de los participantes alcanza los 9.15 puntos porcentuales, mientras que el rango de medias globales va desde aproximadamente un 61.6 % para el **usuario 2** hasta un 81.3 % para el **usuario 4**, lo que supone una diferencia cercana a 20 puntos porcentuales entre ambos. Esta dispersión es superior a la observada entre los métodos de entrenamiento y sugiere una importante dependencia del rendimiento respecto al participante.

Además, el comportamiento de los usuarios no presenta una tendencia general. Por ejemplo, el **usuario 1** obtiene los mejores resultados en la tarea estática, mientras que el **usuario 4** presenta el mejor rendimiento en la tarea de movimiento. Aunque, en general, la precisión de la tarea estática supera a la de movimiento en aproximadamente 7 puntos, esta diferencia no se observa de la misma forma en todos los participantes. El **usuario 2**, por ejemplo, mejora 3 puntos porcentuales en la tarea de movimiento, mientras que el **usuario 3** experimenta una reducción cercana a 20 puntos.

Estas diferencias pueden estar relacionadas con la variabilidad fisiológica de las señales EEG, la calidad del registro, la colocación de los electrodos, la capacidad de concentración o la forma en que cada participante ejecuta las tareas. El hecho

de que el orden relativo de los usuarios cambie entre tareas indica que no todos los participantes se ven afectados de la misma manera por la complejidad de la tarea.

En conjunto, los resultados muestran una elevada variabilidad inter-sujeto.

### 5.4. Influencia de los métodos de entrenamiento

El método *Inter-Sesión* obtiene la mayor precisión media global, con aproximadamente un 74.8 %. Después va el entrenamiento *Inter-Usuarios*, con un 72.9 %, seguido del entrenamiento *Continuo*, con un 71.4 %, y del entrenamiento *Intra-Sesión*, con un 70.4 %.

El menor rendimiento del método *Intra-Sesión* era de esperar, ya que al utilizar datos de un único día y usuario, el modelo dispone de menos ejemplos para aprender y generalizar. Además puede presentar una mayor dependencia de las características particulares de esa sesión.

El método *Inter-Sesión* puede aprovechar información de sesiones anteriores del mismo usuario, lo que permite aumentar la cantidad de datos de entrenamiento. Además, al centrarse en un único participante, el modelo no se enfrenta a la variabilidad asociada a otros usuarios. Esto podría explicar su mayor precisión media y su menor desviación.

Los entrenamientos *Inter-Usuarios* y *Continuo* no presentan una mejora claramente diferenciada respecto al entrenamiento *Intra-Sesión*. Esto quiere decir que el uso de un modelo inicial entrenado con otros usuarios no garantiza un aumento de la precisión tras su adaptación al usuario objetivo.

Una posible explicación es que, las características compartidas entre participantes sean limitadas y que el conocimiento adquirido con otros usuarios no pueda transferirse directamente. También es posible que, durante la adaptación individual, el modelo modifique parte de los pesos generales aprendidos sin disponer todavía de suficientes datos del usuario objetivo, perdiendo así información relevante. Queda por determinar si una modificación en el modelo general o en el proceso de *fine-tuning*

puede mejorar el rendimiento del modelo específico.

En cualquier caso, los resultados no permiten establecer un método de entrenamiento claramente superior. Aunque *Inter-Sesión* presenta el mejor rendimiento medio, no presenta la misma mejora en todas las arquitecturas, tareas y usuarios. Esto no implica que los métodos sean equivalentes. Los resultados pueden indicar que sus diferencias son menores que las producidas por los otros factores.

## 5.5. Conclusión general

En general, se puede concluir que *ShallowConvNet* es la arquitectura más adecuada para este conjunto de datos, al presentar la mayor precisión media y un comportamiento comparativamente estable entre tareas y métodos de entrenamiento.

También se observa una diferencia clara entre las dos tareas estudiadas. La tarea estática obtiene mejores resultados que la tarea de movimiento de manera consistente, aunque no se puede obtener una razón concreta de esta diferencia a partir de los resultados obtenidos.

En cuanto a las estrategias de entrenamiento, *Inter-Sesión* presenta los mejores resultados medios, pero la mejora respecto a los demás métodos no es consistente. Los métodos que incorporan información de otros usuarios no consiguen una ventaja suficientemente grande como para justificar la mayor complejidad en su proceso de entrenamiento.

En las condiciones evaluadas, aumentar la complejidad de la estrategia de entrenamiento equivale necesariamente a una mejora proporcional de la precisión. La arquitectura seleccionada y las características particulares del usuario y de la tarea parecen ser factores más determinantes.

Finalmente, la elevada variabilidad entre usuarios sugiere que la capacidad de generalización de los modelos está extremadamente condicionada por las características individuales de las señales EEG. Esto puede dificultar la elección de una configuración óptima de arquitectura y método de entrenamiento que sea válida para todos



los participantes.

### 5.6. Trabajos futuros

#### 5.6.1. Máquina de estados finitos

Una posible línea de trabajo futuro consiste en integrar dos modelos distintos dentro de un mismo sistema de control, con el objetivo de aprovechar las dos tareas de imaginación motora para construir una BCI de bucle cerrado basada en una máquina de estados finitos. Este enfoque permitiría controlar de forma independiente el comportamiento del exoesqueleto y proporcionar al usuario un mayor grado de autonomía durante la interacción.

En este planteamiento, el sistema comenzaría en un estado de **parada**, en el que el exoesqueleto permanecería inmóvil mientras un modelo entrenado sobre la tarea estática (*Modelo de Inicio de la Marcha*) monitoriza continuamente la actividad cerebral del usuario. Cuando dicho modelo detectase una intención de inicio de la marcha, el sistema realizaría una transición al estado de **movimiento**.

Durante este segundo estado, el exoesqueleto ejecutaría la marcha asistida mientras un modelo entrenado sobre la tarea en movimiento (*Modelo de Detención*) analiza la señal EEG en busca de una intención de parada. Una vez detectada esta intención, el sistema regresaría al estado de **parada**, completando así el ciclo de control.

Con el objetivo de reducir la probabilidad de falsos positivos, las predicciones de ambos modelos podrían combinarse mediante una estrategia basada en una media ponderada. De esta forma, aunque cada estado disponga de un clasificador principal, la decisión final tendría en cuenta la información proporcionada por ambos modelos, aumentando potencialmente la robustez del sistema frente a errores de clasificación.

La Figura 5.1 muestra una representación conceptual de esta máquina de estados finitos y de las transiciones asociadas a cada uno de los modelos de clasificación.

Este procedimiento es consistente con diversas propuestas presentes en la literatura,

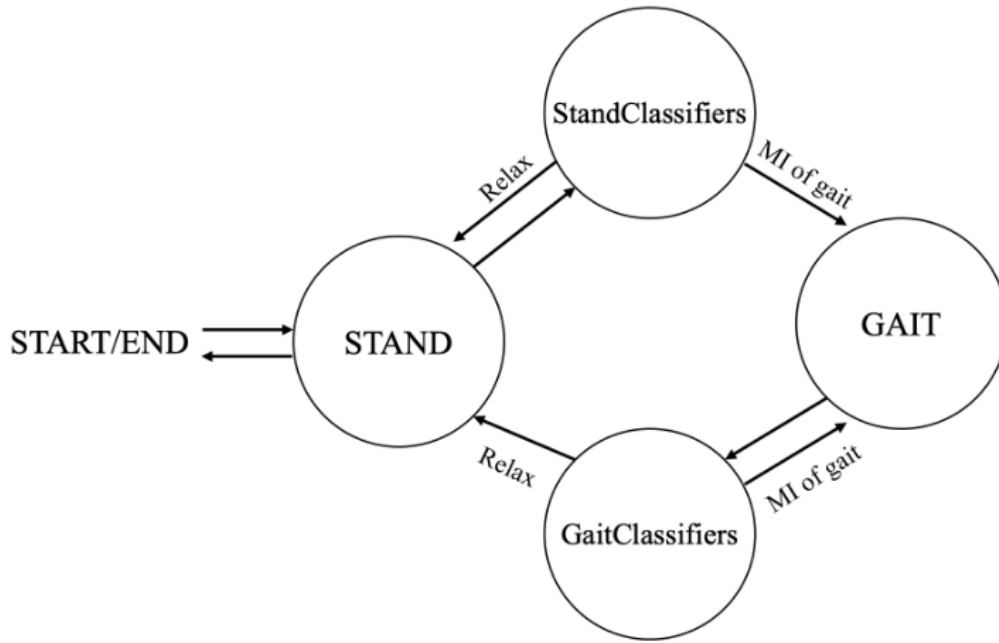


Figura 5.1: Ejemplo de arquitectura de control basada en una máquina de estados finitos para sistemas BCI. Tomado de [64]

donde el control de sistemas BCI se aborda mediante máquinas de estados finitos donde varios clasificadores de EEG son utilizados para detectar las distintas intenciones del usuario, tales como iniciar, mantener, o finalizar la marcha. En esta línea, Ferrero et al. [64] emplean una arquitectura de control basada en estados para gobernar la marcha de un exoesqueleto, mientras que los mismos autores [66] aplican un enfoque similar al control de una cinta de marcha.

Asimismo, Choi et al. [72] demostraron la viabilidad de este paradigma en sistemas de control asíncronos para exoesqueletos de miembro inferior, utilizando tres estados funcionales (parada, marcha y sentado) y transiciones activadas mediante imaginación motora.

### 5.6.2. Interpretabilidad de los modelos utilizados

Otra posible línea de investigación consiste en estudiar la interpretabilidad de los modelos entrenados. Tanto Lawhern et al. [52] como Schirrmeister et al. [20] aplicaron estas técnicas para analizar las características aprendidas por las arquitecturas propuestas en sus respectivos trabajos.

Un análisis interpretable permitiría determinar qué componentes de la señal tienen una mayor influencia en la clasificación de cada una de las tareas propuestas. En particular, podría estudiarse la relevancia de los distintos electrodos y regiones espaciales o las de distintas bandas de frecuencia e intervalos temporales.

Este análisis permitiría comprobar si los modelos basan sus decisiones en patrones neurofisiológicos comunes o si, por el contrario, dependen en exceso de artefactos o características particulares del conjunto de datos. Además, la comparación de los patrones aprendidos por *EEGNet*, *DeepConvNet* y *ShallowConvNet* ayudaría a explicar las diferencias de rendimiento observadas entre las arquitecturas.

### 5.6.3. Evaluación de la cantidad de datos necesaria

En los experimentos realizados se utilizaron para el entrenamiento los ocho primeros ensayos de cada día, de un total de once. Esto representa aproximadamente un 73 % de los ensayos disponibles, mientras que los tres restantes se reservaron para la evaluación.

Como trabajo futuro, sería interesante estudiar cómo varía el rendimiento de las arquitecturas al modificar la cantidad de información utilizada durante el entrenamiento. Para ello, podrían entrenarse los modelos utilizando una cantidad diferente de ensayos, pudiendo mantener un conjunto de evaluación común para garantizar que los resultados obtenidos sean comparables.

Este análisis permitiría observar la relación entre la cantidad de datos disponibles y la precisión alcanzada, así como identificar el punto a partir del cual la incorporación de nuevos ensayos no produce grandes mejoras.

También permitiría estimar la cantidad mínima de información necesaria para adaptar un modelo a un nuevo usuario, reduciendo el tiempo de calibración. Esto resulta especialmente relevante para el desarrollo de sistemas BCI, ya que una estrategia capaz de alcanzar un rendimiento adecuado con pocos ensayos facilitaría su adopción por parte de nuevos usuarios y reduciría el tiempo de aprendizaje.



## BIBLIOGRAFÍA

- [1] World Health Organization (WHO). «Lesión de la médula espinal,» Organización Mundial de la Salud. (16 de abr. de 2024), dirección: <https://www.who.int/es/news-room/fact-sheets/detail/spinal-cord-injury> (visitado 29-04-2026).
- [2] S. A. Bravo, J. A. Porras y M. J. Á. Urricelqu, «La exclusión social, una problemática estructural entre las personas con discapacidad,» *Áreas. Revista Internacional de Ciencias Sociales*, n.º 36, págs. 167-181, 1 de nov. de 2017, ISSN: 1989-6190. dirección: <https://revistas.um.es/areas/article/view/308211> (visitado 29-04-2026).
- [3] G. Morone y R. S. Calabrò, «Neurorehabilitation Insights in 2024: Where Neuroscience Meets Next-Gen Tech,» *Brain Sciences*, vol. 15, n.º 10, pág. 1043, 25 de sep. de 2025, ISSN: 2076-3425. DOI: 10.3390/brainsci15101043. PMID: 41154139. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12562376/> (visitado 04-05-2026).
- [4] M. T. Sadiq, M. Z. Aziz, A. Almogren, A. Yousaf, S. Siuly y A. U. Rehman, «Exploiting Pretrained CNN Models for the Development of an EEG-based Robust BCI Framework,» *Computers in Biology and Medicine*, vol. 143, pág. 105242, 1 de abr. de 2022, ISSN: 0010-4825. DOI: 10.1016/j.combiomed.2022.105242. dirección: <https://www.sciencedirect.com/science/article/pii/S0010482522000348> (visitado 27-04-2026).
- [5] S. Yun, «Advances, Challenges, and Prospects of Electroencephalography-Based Biomarkers for Psychiatric Disorders: A Narrative Review,» *Journal of Yeungnam Medical Science*, vol. 41, n.º 4, págs. 261-268, 9 de sep. de 2024, ISSN: 2799-8010. DOI: 10.12701/jyms.2024.00668. PMID: 39246060. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11534409/> (visitado 06-05-2026).

- [6] M. C. Emos y S. Agarwal, «Neuroanatomy, Upper Motor Neuron Lesion,» en *StatPearls*, Treasure Island (FL): StatPearls Publishing, 2026. PMID: 30725990. dirección: <http://www.ncbi.nlm.nih.gov/books/NBK537305/> (visitado 12-05-2026).
  
- [7] K. Margetis, J. M. Das y P. D. Emmady, «Spinal Cord Injuries,» en *StatPearls*, Treasure Island (FL): StatPearls Publishing, 2026. PMID: 32809556. dirección: <http://www.ncbi.nlm.nih.gov/books/NBK560721/> (visitado 12-05-2026).
  
- [8] National Institute of Neurological Disorders and Stroke. «Spinal Cord Injury — National Institute of Neurological Disorders and Stroke.» (), dirección: <https://www.ninds.nih.gov/health-information/disorders/spinal-cord-injury> (visitado 12-05-2026).
  
- [9] I. Corral López, M. Morales Castillo y M. O. González Oria. «Traumatismo medular,» Manuales Clínicos. (2 de nov. de 2023), dirección: <https://manualclinico.hospitaluvrocio.es/urgencias-de-traumatologia/patologia-de-cabeza-y-tronco/traumatismo-medular/> (visitado 12-05-2026).
  
- [10] S. J. Murphy y D. J. Werring, «Stroke: Causes and Clinical Features,» *Medicine (Abingdon, England : UK Ed.)*, vol. 48, n.º 9, págs. 561-566, sep. de 2020, ISSN: 1357-3039. DOI: 10.1016/j.mpmed.2020.06.002. PMID: 32837228. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7409792/> (visitado 12-05-2026).
  
- [11] Z. Gao, Z. Pang, Y. Chen, G. Lei, S. Zhu, G. Li, Y. Shen y W. Xu, «Restoring After Central Nervous System Injuries: Neural Mechanisms and Translational Applications of Motor Recovery,» *Neuroscience Bulletin*, vol. 38, n.º 12, págs. 1569-1587, 1 de dic. de 2022, ISSN: 1995-8218. DOI: 10.1007/s12264-

- 022-00959-x. dirección: <https://doi.org/10.1007/s12264-022-00959-x> (visitado 12-05-2026).
- [12] M. Puderbaugh y P. D. Emmady, «Neuroplasticity,» en *StatPearls*, Treasure Island (FL): StatPearls Publishing, 2026. PMID: 32491743. dirección: <http://www.ncbi.nlm.nih.gov/books/NBK557811/> (visitado 13-05-2026).
- [13] R. Di Palma, L. Falco, A. Coccia, F. Amitrano, G. Pagano y G. D'Addio, «Frontiers in neuro-rehabilitation: Neural basis of cognitive contributions to movement across neurological populations,» *Frontiers in Neurology*, vol. 16, 4 de jul. de 2025, ISSN: 1664-2295. DOI: 10.3389/fneur.2025.1638254. dirección: <https://www.frontiersin.org/journals/neurology/articles/10.3389/fneur.2025.1638254/full> (visitado 13-05-2026).
- [14] S. J. Luck, *An Introduction to the Event-Related Potential Technique*, 2.<sup>a</sup> ed. Cambridge, MA, USA: MIT Press, 30 de mayo de 2014, 416 págs., ISBN: 978-0-262-52585-5.
- [15] D. L. Sherman y N. V. Thakor, «EEG Signal Processing: Theory and Applications,» en *Neural Engineering*, B. He, ed., Cham: Springer International Publishing, 2020, págs. 97-129, ISBN: 978-3-030-43395-6. DOI: 10.1007/978-3-030-43395-6\_3. dirección: [https://doi.org/10.1007/978-3-030-43395-6\\_3](https://doi.org/10.1007/978-3-030-43395-6_3) (visitado 06-05-2026).
- [16] B. Jiang, X. Wang, D. He, S. Cheng, M. Liao, D. Miao, Q. Xia, Y. Zhao y G. Li, «A Comprehensive Review of Deep Learning for Motor Imagery EEG: From Healthy Subjects to Patients,» *Biomedical Signal Processing and Control*, vol. 117, pág. 109680, 15 de mayo de 2026, ISSN: 1746-8094. DOI: 10.1016/j.bspc.2026.109680. dirección: <https://www.sciencedirect.com/science/article/pii/S174680942600234X> (visitado 05-05-2026).

- [17] M. Roohi-Azizi, L. Azimi, S. Heysieattalab y M. Aamidfar, «Changes of the Brain's Bioelectrical Activity in Cognition, Consciousness, and Some Mental Disorders,» *Medical Journal of the Islamic Republic of Iran*, vol. 31, pág. 53, 3 de sep. de 2017, ISSN: 1016-1430. DOI: 10.14196/mjiri.31.53. PMID: 29445682. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC5804435/> (visitado 06-05-2026).
  
- [18] A. Craik, Y. He y J. L. Contreras-Vidal, «Deep learning for electroencephalogram (EEG) classification tasks: A review,» *Journal of Neural Engineering*, vol. 16, n.º 3, pág. 031 001, abr. de 2019, ISSN: 1741-2552. DOI: 10.1088/1741-2552/ab0ab5. dirección: <https://doi.org/10.1088/1741-2552/ab0ab5> (visitado 27-04-2026).
  
- [19] A. Telpaz, R. Webb y D. Levy, «Using EEG to Predict Consumers' Future Choices,» *Journal of Marketing Research*, 1 de ago. de 2015. DOI: 10.1509/jmr.13.0564.
  
- [20] R. T. Schirrmeister, J. T. Springenberg, L. D. J. Fiederer, M. Glasstetter, K. Eggensperger, M. Tangermann, F. Hutter, W. Burgard y T. Ball, «Deep learning with convolutional neural networks for EEG decoding and visualization,» *Human Brain Mapping*, vol. 38, n.º 11, págs. 5391-5420, 2017, ISSN: 1097-0193. DOI: 10.1002/hbm.23730. dirección: <https://onlinelibrary.wiley.com/doi/abs/10.1002/hbm.23730> (visitado 09-03-2026).
  
- [21] S. Saha y M. Baumert, «Intra- and Inter-subject Variability in EEG-Based Sensorimotor Brain Computer Interface: A Review,» *Frontiers in Computational Neuroscience*, vol. 13, pág. 87, 21 de ene. de 2020, ISSN: 1662-5188. DOI: 10.3389/fncom.2019.00087. PMID: 32038208. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6985367/> (visitado 06-05-2026).
  
- [22] W. Klonowski, «Everything You Wanted to Ask about EEG but Were Afraid to Get the Right Answer,» *Nonlinear Biomedical Physics*, vol. 3, pág. 2, 26 de ma-



- yo de 2009, ISSN: 1753-4631. DOI: 10.1186/1753-4631-3-2. PMID: 19470156. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC2698918/> (visitado 02-07-2026).
- [23] C. Tan, F. Sun y W. Zhang. «Deep Transfer Learning for EEG-based Brain Computer Interface.» arXiv: 1808.01752 [cs.CV]. (6 de ago. de 2018), dirección: <http://arxiv.org/abs/1808.01752> (visitado 06-06-2026), prepublicado.
- [24] R. J. Sugden, V.-L. L. Pham-Kim-Nghiem-Phu, I. Campbell, A. Leon y P. Diamandis, «Remote Collection of Electrophysiological Data with Brain Wearables: Opportunities and Challenges,» *Bioelectronic Medicine*, vol. 9, pág. 12, 21 de jun. de 2023, ISSN: 2332-8886. DOI: 10.1186/s42234-023-00114-5. PMID: 37340487. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10283168/> (visitado 06-06-2026).
- [25] C. Rommel, J. Paillard, T. Moreau y A. Gramfort, «Data augmentation for learning predictive models on EEG: A systematic comparison,» *Journal of Neural Engineering*, vol. 19, n.º 6, pág. 066 020, nov. de 2022, ISSN: 1741-2552. DOI: 10.1088/1741-2552/aca220. dirección: <https://doi.org/10.1088/1741-2552/aca220> (visitado 06-06-2026).
- [26] M. J. Young, D. J. Lin y L. R. Hochberg, «Brain-Computer Interfaces in Neurorecovery and Neurorehabilitation,» *Seminars in neurology*, vol. 41, n.º 2, págs. 206-216, abr. de 2021, ISSN: 0271-8235. DOI: 10.1055/s-0041-1725137. PMID: 33742433. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8768507/> (visitado 10-05-2026).
- [27] B. He, H. Yuan, J. Meng y S. Gao, «Brain-Computer Interfaces,» en *Neural Engineering*, B. He, ed., Cham: Springer International Publishing, 2020, págs. 131-183, ISBN: 978-3-030-43395-6. DOI: 10.1007/978-3-030-43395-

6\_4. dirección: [https://doi.org/10.1007/978-3-030-43395-6\\_4](https://doi.org/10.1007/978-3-030-43395-6_4) (visitado 06-05-2026).

- [28] S. Waldert, «Invasive vs. Non-Invasive Neuronal Signals for Brain-Machine Interfaces: Will One Prevail?» *Frontiers in Neuroscience*, vol. 10, 27 de jun. de 2016, ISSN: 1662-453X. DOI: 10.3389/fnins.2016.00295. dirección: <https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2016.00295/full> (visitado 01-06-2026).
- [29] J. R. Wolpaw y E. W. Wolpaw, *Brain-Computer Interfaces: Principles and Practice*. New York: Oxford University Press, 2012, ISBN: 978-0-19-993268-9.
- [30] M. F. Mridha, S. C. Das, M. M. Kabir, A. A. Lima, M. R. Islam e Y. Watanobe, «Brain-Computer Interface: Advancement and Challenges,» *Sensors (Basel, Switzerland)*, vol. 21, n.º 17, pág. 5746, 26 de ago. de 2021, ISSN: 1424-8220. DOI: 10.3390/s21175746. PMID: 34502636. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8433803/> (visitado 29-04-2026).
- [31] D. J. McFarland, J. Daly, C. Boulay y M. Parvaz, «Therapeutic Applications of BCI Technologies,» *Brain computer interfaces (Abingdon, England)*, vol. 47, n.º 1–2, págs. 37-52, 2017, ISSN: 2326-263X. DOI: 10.1080/2326263X.2017.1307625. PMID: 29527538. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC5843196/> (visitado 02-06-2026).
- [32] Q. Deng, Z. Fu, N. Ma y B. Wang, «Application and Future Directions of Brain-Computer Interfaces in Neurological Disorders: Technological Advances, Clinical Practices, and Challenges,» *Brain Hemorrhages*, vol. 6, n.º 6, págs. 306-314, 1 de dic. de 2025, ISSN: 2589-238X. DOI: 10.1016/j.hest.2025.09.002. dirección: <https://www.sciencedirect.com/science/article/pii/S2589238X2500083X> (visitado 02-06-2026).

- [33] B. J. Edelman, S. Zhang, G. Schalk, P. Brunner, G. Müller-Putz, C. Guan y B. He, «Non-Invasive Brain-Computer Interfaces: State of the Art and Trends,» *IEEE Reviews in Biomedical Engineering*, vol. 18, págs. 26-49, 2025, ISSN: 1941-1189. DOI: 10.1109/RBME.2024.3449790. dirección: <https://ieeexplore.ieee.org/abstract/document/10646518/references> (visitado 25-05-2026).
- [34] S. Krishnan e Y. Athavale, «Trends in Biomedical Signal Feature Extraction,» *Biomedical Signal Processing and Control*, vol. 43, págs. 41-63, 1 de mayo de 2018, ISSN: 1746-8094. DOI: 10.1016/j.bspc.2018.02.008. dirección: <https://www.sciencedirect.com/science/article/pii/S1746809418300399> (visitado 02-07-2026).
- [35] R. Yuvaraj, P. Thagavel, J. Thomas, J. Fogarty y F. Ali, «Comprehensive Analysis of Feature Extraction Methods for Emotion Recognition from Multichannel EEG Recordings,» *Sensors*, vol. 23, n.º 2, pág. 915, ene. de 2023, ISSN: 1424-8220. DOI: 10.3390/s23020915. dirección: <https://www.mdpi.com/1424-8220/23/2/915> (visitado 02-07-2026).
- [36] A. Subasi, «Chapter 4 - Feature Extraction and Dimension Reduction,» en *Practical Guide for Biomedical Signals Analysis Using Machine Learning Techniques*, Academic Press, 1 de ene. de 2019, págs. 193-275, ISBN: 978-0-12-817444-9. DOI: 10.1016/B978-0-12-817444-9.00004-0. dirección: <https://www.sciencedirect.com/science/article/pii/B9780128174449000040> (visitado 02-07-2026).
- [37] I. Stancin, M. Cifrek y A. Jovic, «A Review of EEG Signal Features and Their Application in Driver Drowsiness Detection Systems,» *Sensors (Basel, Switzerland)*, vol. 21, n.º 11, pág. 3786, 30 de mayo de 2021, ISSN: 1424-8220. DOI: 10.3390/s21113786. PMID: 34070732. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8198610/> (visitado 02-07-2026).

- [38] S. Siuly, Y. Li e Y. Zhang, *EEG Signal Analysis and Classification* (Health Information Science). Cham: Springer International Publishing, 2016, ISBN: 978-3-319-47652-0 978-3-319-47653-7. DOI: 10.1007/978-3-319-47653-7. dirección: <http://link.springer.com/10.1007/978-3-319-47653-7> (visitado 29-09-2025).
  
- [39] M. Mirnaziri, M. Rahimi, S. Alavikakhaki y R. Ebrahimpour, «Using Combination of  $\mu$ ,  $\beta$  and  $\gamma$  Bands in Classification of EEG Signals,» *Basic and Clinical Neuroscience*, vol. 4, n.º 1, págs. 76-87, 2013, ISSN: 2008-126X. PMID: 25337331. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4202559/> (visitado 21-05-2026).
  
- [40] H. Ramoser, J. Müller-Gerking y G. Pfurtscheller, «Optimal spatial filtering of single trial EEG during imagined hand movement,» *IEEE transactions on rehabilitation engineering: a publication of the IEEE Engineering in Medicine and Biology Society*, vol. 8, n.º 4, págs. 441-446, dic. de 2000, ISSN: 1063-6528. DOI: 10.1109/86.895946. PMID: 11204034.
  
- [41] B. Blankertz, R. Tomioka, S. Lemm, M. Kawanabe y K.-r. Muller, «Optimizing Spatial Filters for Robust EEG Single-Trial Analysis,» *IEEE Signal Processing Magazine*, vol. 25, n.º 1, págs. 41-56, 2008, ISSN: 1558-0792. DOI: 10.1109/MSP.2008.4408441. dirección: <https://ieeexplore.ieee.org/document/4408441> (visitado 02-07-2026).
  
- [42] K. K. Ang, Z. Y. Chin, H. Zhang y C. Guan, «Filter Bank Common Spatial Pattern (FBCSP) in Brain-Computer Interface,» en *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, jun. de 2008, págs. 2390-2397. DOI: 10.1109/IJCNN.2008.4634130. dirección: <https://ieeexplore.ieee.org/document/4634130> (visitado 02-07-2026).

- [43] K. K. Ang, Z. Y. Chin, C. Wang, C. Guan y H. Zhang, «Filter Bank Common Spatial Pattern Algorithm on BCI Competition IV Datasets 2a and 2b,» *Frontiers in Neuroscience*, vol. 6, 29 de mar. de 2012, ISSN: 1662-453X. DOI: 10.3389/fnins.2012.00039. dirección: <https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2012.00039/full> (visitado 02-07-2026).
- [44] S. Albawi, T. Abed Mohammed y S. ALZAWI, «Understanding of a Convolutional Neural Network,» 21 de ago. de 2017. DOI: 10.1109/ICEngTechnol.2017.8308186.
- [45] Y. LeCun, Y. Bengio y G. Hinton, «Deep learning,» *Nature*, vol. 521, n.º 7553, págs. 436-444, mayo de 2015, ISSN: 1476-4687. DOI: 10.1038/nature14539. dirección: <https://www.nature.com/articles/nature14539> (visitado 02-06-2026).
- [46] J. Schmidhuber, «Deep Learning in Neural Networks: An Overview,» *Neural Networks*, vol. 61, págs. 85-117, 1 de ene. de 2015, ISSN: 0893-6080. DOI: 10.1016/j.neunet.2014.09.003. dirección: <https://www.sciencedirect.com/science/article/pii/S0893608014002135> (visitado 02-06-2026).
- [47] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. MIT Press, 2016.
- [48] I. Rakhmatulin, M.-S. Dao, A. Nassibi y D. Mandic, «Exploring Convolutional Neural Network Architectures for EEG Feature Extraction,» *Sensors (Basel, Switzerland)*, vol. 24, n.º 3, pág. 877, 29 de ene. de 2024, ISSN: 1424-8220. DOI: 10.3390/s24030877. PMID: 38339594. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10856895/> (visitado 02-06-2026).
- [49] D. Sambyal y A. Sarwar, «Recent Developments in Cervical Cancer Diagnosis Using Deep Learning on Whole Slide Images: An Overview of Models, Techniques, Challenges and Future Directions,» *Micron*, vol. 173, pág. 103 520,

1 de oct. de 2023, ISSN: 0968-4328. DOI: 10.1016/j.micron.2023.103520. dirección: <https://www.sciencedirect.com/science/article/pii/S096843282300118X> (visitado 03-07-2026).

- [50] S. J. Pan y Q. Yang, «A Survey on Transfer Learning,» *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, n.º 10, págs. 1345-1359, oct. de 2010, ISSN: 1558-2191. DOI: 10.1109/TKDE.2009.191. dirección: <https://ieeexplore.ieee.org/document/5288526> (visitado 03-07-2026).
- [51] C. M. Köllöd, A. Adolf, G. Márton e I. Ulbert, «Deep Comparisons of Neural Networks from the EEGNet Family,» *Electronics*, vol. 12, n.º 12, pág. 2743, 20 de jun. de 2023, ISSN: 2079-9292. DOI: 10.3390/electronics12122743. arXiv: 2302.08797 [eess.SP]. dirección: <http://arxiv.org/abs/2302.08797> (visitado 19-05-2026).
- [52] V. J. Lawhern, A. J. Solon, N. R. Waytowich, S. M. Gordon, C. P. Hung y B. J. Lance, «EEGNet: A Compact Convolutional Network for EEG-based Brain-Computer Interfaces,» *Journal of Neural Engineering*, vol. 15, n.º 5, pág. 056013, 1 de oct. de 2018, ISSN: 1741-2560, 1741-2552. DOI: 10.1088/1741-2552/aace8c. arXiv: 1611.08024 [cs]. dirección: <http://arxiv.org/abs/1611.08024> (visitado 25-04-2025).
- [53] Y. Roy, H. Banville, I. M. Carneiro de Albuquerque, A. Gramfort, T. Falk y J. Faubert, «Deep Learning-Based Electroencephalography Analysis: A Systematic Review,» *Journal of Neural Engineering*, vol. 16, 14 de ago. de 2019. DOI: 10.1088/1741-2552/ab260c.
- [54] Z. Wan, R. Yang, M. Huang, N. Zeng y X. Liu, «A Review on Transfer Learning in EEG Signal Analysis,» *Neurocomputing*, vol. 421, págs. 1-14, 15 de ene. de 2021. DOI: 10.1016/j.neucom.2020.09.017.

- [55] J. Ma, B. Yang, W. Qiu, Y. Li, S. Gao y X. Xia, «A large EEG dataset for studying cross-session variability in motor imagery brain-computer interface,» *Scientific Data*, vol. 9, n.º 1, pág. 531, 1 de sep. de 2022, ISSN: 2052-4463. DOI: 10.1038/s41597-022-01647-1. dirección: <https://www.nature.com/articles/s41597-022-01647-1> (visitado 06-06-2026).
- [56] D.-J. Sung, K.-T. Kim, J.-H. Jeong, L. Kim, S. J. Lee, H. Kim y S.-J. Kim, «Improving Inter-Session Performance via Relevant Session-Transfer for Multi-Session Motor Imagery Classification,» *Heliyon*, vol. 10, n.º 17, e37343, 15 de sep. de 2024, ISSN: 2405-8440. DOI: 10.1016/j.heliyon.2024.e37343. dirección: <https://www.sciencedirect.com/science/article/pii/S2405844024133749> (visitado 03-06-2026).
- [57] D. Wu, Y. Xu y B.-L. Lu. «Transfer Learning for EEG-Based Brain-Computer Interfaces: A Review of Progress Made Since 2016,» arXiv.org. (13 de abr. de 2020), dirección: <https://arxiv.org/abs/2004.06286v4> (visitado 03-06-2026).
- [58] Z. Otarbay y A. Kyzirkanov, «Transfer learning for subject-independent motor imagery EEG classification using convolutional relational networks,» *Frontiers in Neuroscience*, vol. 19, 2 de ene. de 2026, ISSN: 1662-453X. DOI: 10.3389/fnins.2025.1691929. dirección: <https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2025.1691929/full> (visitado 03-06-2026).
- [59] S. An, M. Kang, S. Kim, P. Chikontwe, L. Shen y S. H. Park, «Subject-Adaptive Transfer Learning Using Resting State EEG Signals for Cross-Subject EEG Motor Imagery Classification,» en *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024*, M. G. Linguraru, Q. Dou, A. Feragen, S. Giannarou, B. Glocker, K. Lekadir y J. A. Schnabel, eds., vol. 15011, Cham: Springer Nature Switzerland, 2024, págs. 678-688, ISBN: 978-3-031-72119-9 978-3-031-72120-5. DOI: 10.1007/978-3-031-72120-5\_63.

dirección: [https://link.springer.com/10.1007/978-3-031-72120-5\\_63](https://link.springer.com/10.1007/978-3-031-72120-5_63) (visitado 03-06-2026).

- [60] W. Lu, H. Liu, H. Ma, T.-P. Tan y L. Xia, «Hybrid Transfer Learning Strategy for Cross-Subject EEG Emotion Recognition,» *Frontiers in Human Neuroscience*, vol. 17, pág. 1280241, 16 de nov. de 2023, ISSN: 1662-5161. DOI: 10.3389/fnhum.2023.1280241. PMID: 38034069. dirección: <https://pubmed.ncbi.nlm.nih.gov/articles/PMC10687359/> (visitado 03-06-2026).
- [61] M. Wimpff, B. Aristimunha, S. Chevallier y B. Yang. «Fine-Tuning Strategies for Continual Online EEG Motor Imagery Decoding: Insights from a Large-Scale Longitudinal Study.» ver. 1. (2025), dirección: <https://arxiv.org/abs/2502.06828> (visitado 03-06-2026), prepublicado.
- [62] C.-J. Huang, C.-F. Cao, K. K. Shyu, T.-M. Lee y P.-L. Lee, «Continual-Learning-Enhanced CNN-Transformer Framework for Real-Time Motor-Imagery BCI in Virtual Environments,» *Bioengineering*, vol. 13, n.º 5, pág. 536, mayo de 2026, ISSN: 2306-5354. DOI: 10.3390/bioengineering13050536. dirección: <https://www.mdpi.com/2306-5354/13/5/536> (visitado 03-06-2026).
- [63] A. Abu-Rmileh, E. Zakkay, L. Shmuelof y O. Shriki, «Co-adaptive Training Improves Efficacy of a Multi-Day EEG-Based Motor Imagery BCI Training,» *Frontiers in Human Neuroscience*, vol. 13, 14 de oct. de 2019, ISSN: 1662-5161. DOI: 10.3389/fnhum.2019.00362. dirección: <https://www.frontiersin.org/journals/human-neuroscience/articles/10.3389/fnhum.2019.00362/full> (visitado 03-06-2026).
- [64] L. Ferrero, V. Quiles, M. Ortiz, E. Iáñez y J. M. Azorín, «A BMI Based on Motor Imagery and Attention for Commanding a Lower-Limb Robotic Exoskeleton: A Case Study,» *Applied Sciences*, vol. 11, n.º 9, pág. 4106, ene. de 2021, ISSN: 2076-3417. DOI: 10.3390/app11094106. dirección: <https://www.mdpi.com/2076-3417/11/9/4106> (visitado 29-08-2025).



- [65] L. Ferrero, P. Soriano-Segura, J. Navarro, O. Jones, M. Ortiz, E. Iáñez, J. M. Azorín y J. L. Contreras-Vidal, «Brain-machine interface based on deep learning to control asynchronously a lower-limb robotic exoskeleton: A case-of-study,» *Journal of NeuroEngineering and Rehabilitation*, vol. 21, n.º 1, pág. 48, 5 de abr. de 2024, ISSN: 1743-0003. DOI: 10.1186/s12984-024-01342-9. dirección: <https://doi.org/10.1186/s12984-024-01342-9> (visitado 20-07-2026).
- [66] L. Ferrero, V. Quiles, M. Ortiz, E. Ianez y J. M. Azorin, «BCI Based on Lower-Limb Motor Imagery and a State Machine for Walking on a Treadmill,» en *2021 10th International IEEE/EMBS Conference on Neural Engineering (NER)*, Italy: IEEE, 4 de mayo de 2021, págs. 1-4, ISBN: 978-1-7281-4337-8. DOI: 10.1109/NER49283.2021.9441216. dirección: <https://ieeexplore.ieee.org/document/9441216/> (visitado 29-08-2025).
- [67] A. Kilicarslan, R. G. Grossman y J. L. Contreras-Vidal, «A robust adaptive denoising framework for real-time artifact removal in scalp EEG measurements,» *Journal of Neural Engineering*, vol. 13, n.º 2, pág. 026013, abr. de 2016, ISSN: 1741-2552. DOI: 10.1088/1741-2560/13/2/026013. PMID: 26863159.
- [68] D. Rosique Egea. «Código Fuente del TFM: Redes neuronales convolucionales para el análisis de señales EEG durante la marcha con un exoesqueleto,» GitHub. (), dirección: [https://github.com/DanielRosiqueEgea/eeg/tree/plotting\\_results](https://github.com/DanielRosiqueEgea/eeg/tree/plotting_results) (visitado 29-06-2026).
- [69] «Introduction to Altair in Python,» GeeksforGeeks. (19 de jul. de 2025), dirección: <https://www.geeksforgeeks.org/python/introduction-to-altair-in-python/> (visitado 07-04-2026).
- [70] «Why DuckDB,» DuckDB. (2026), dirección: [https://duckdb.org/why\\_duckdb.html](https://duckdb.org/why_duckdb.html) (visitado 07-04-2026).

- [71] P. Venkataanusha, C. Anuradha, D. Murty y S. Chebrolu, «Detecting Outliers in High Dimensional Data Sets Using Z-Score Methodology,» *International Journal of Innovative Technology and Exploring Engineering*, vol. 9, págs. 48-53, 30 de nov. de 2019. DOI: 10.35940/ijitee.A3910.119119.
  
- [72] J. Choi, K. Kim, J. Jihyeok, L. Kim, S. Lee y H. Kim, «Developing a Motor Imagery-Based Real-Time Asynchronous Hybrid BCI Controller for a Lower-Limb Exoskeleton,» *Sensors*, vol. 20, pág. 7309, 19 de dic. de 2020. DOI: 10.3390/s20247309.

## ANEXOS

### A. Esquema de la base de datos

La tabla A.1 muestra el esquema de la base de datos relacional utilizada para almacenar los resultados de los experimentos realizados. Esta base de datos ha permitido organizar y estructurar la información obtenida durante el proceso de entrenamiento y evaluación de los modelos, facilitando su posterior análisis y comparación.

La organización de esta base de datos corresponde a tres tablas principales: **Runs\_t**, **Trials\_t** y **Classes\_t**. La tabla **Runs\_t** contiene información general sobre cada ejecución del experimento. La tabla **Trials\_t** almacena los resultados obtenidos durante la etapa de test de cada ejecución, separados por usuario, día y ensayo.

Finalmente, la tabla **Classes\_t** contiene los resultados de cada clase individual dentro de cada ensayo, esta tabla nos permite analizar el rendimiento del modelo en cada clase específica y evaluar su capacidad para diferenciar entre las distintas categorías presentes en el conjunto de datos. Se puede obtener así métricas adicionales como la tasa de falsos positivos y falsos negativos, que son fundamentales para comprender el comportamiento del modelo en situaciones de clasificación desequilibrada o cuando ciertas clases son más críticas que otras.

|                          |                        |                        |                   |
|--------------------------|------------------------|------------------------|-------------------|
| Nombre_Tabla:            | <b>Runs_t</b>          | <b>Trials_t</b>        | <b>Classes_t:</b> |
| N_Entradas:              | 1917                   | 5751                   | 15336             |
| <b>Run metadata</b>      |                        |                        |                   |
| 1                        | run_id                 | run_id                 | run_id            |
| 2                        | run_timestamp          | run_timestamp          | run_timestamp     |
| 3                        | sample_id              | sample_id              | sample_id         |
| 4                        | title                  |                        |                   |
| 5                        | file_path              |                        |                   |
| 6                        | comment                |                        |                   |
| <b>Dimensions</b>        |                        |                        |                   |
| 7                        | training_method        | training_method        | training_method   |
| 8                        | model_name             | model_name             | model_name        |
| 9                        | task_mode              | task_mode              | task_mode         |
| 10                       | user                   | user                   | user              |
| 11                       | day                    | day                    | day               |
| 12                       |                        | trial_id               | trial_id          |
| 13                       |                        |                        | class             |
| <b>Metrics</b>           |                        |                        |                   |
| 14                       | acc1                   | acc1                   |                   |
| 15                       | val_accuracy           | val_accuracy           |                   |
| 16                       | val_precision_macro    | val_precision_macro    | precision         |
| 17                       | val_recall_macro       | val_recall_macro       | recall            |
| 18                       | val_f1_macro           | val_f1_macro           | f1                |
| 19                       | val_precision_weighted | val_precision_weighted |                   |
| 20                       | val_recall_weighted    | val_recall_weighted    |                   |
| 21                       | val_f1_weighted        | val_f1_weighted        | support           |
| 22                       | confusion_matrix_json  | confusion_matrix_json  |                   |
| <b>Training metrics:</b> |                        |                        |                   |
| 27                       | final_train_loss       |                        |                   |
| 28                       | final_train_acc        |                        |                   |
| 29                       | final_training_rate    |                        |                   |
| 30                       | final_val_loss         |                        |                   |
| 31                       | final_val_acc          |                        |                   |
| 32                       | best_epoch             |                        |                   |
| 33                       | best_val_loss          |                        |                   |
| 34                       | best_val_acc           |                        |                   |
| <b>Extras</b>            |                        |                        |                   |
| 23                       | training_samples       |                        |                   |
| 24                       | validation_samples     |                        |                   |
| 25                       | epochs_planned         |                        |                   |
| 26                       | epochs_ran             |                        |                   |

Tabla A.1: Esquema de la base de datos relacional utilizada para almacenar los resultados de los experimentos realizados.

## B. Tablas complementarias de resultados

### B.1. Tasa de falsos positivos y falsos negativos

La tabla B.2 muestra un ejemplo de las tasas de falsos positivos y falsos negativos para cada combinación de modelo y método de entrenamiento.

| Task   | Model          | Training Method | False Positive Rate | False Negative Rate |
|--------|----------------|-----------------|---------------------|---------------------|
| Motion | DeepConvNet    | Intra-Session   | 0.425               | 0.298               |
|        |                | Inter-Session   | 0.304               | 0.335               |
|        |                | Inter-Users     | 0.258               | 0.412               |
|        |                | Continuous      | 0.262               | 0.442               |
|        | EEGNet         | Intra-Session   | 0.241               | 0.297               |
|        |                | Inter-Session   | 0.266               | 0.264               |
|        |                | Inter-Users     | 0.280               | 0.332               |
|        |                | Continuous      | 0.302               | 0.337               |
|        | ShallowConvNet | Intra-Session   | 0.261               | 0.268               |
|        |                | Inter-Session   | 0.245               | 0.277               |
|        |                | Inter-Users     | 0.261               | 0.292               |
|        |                | Continuous      | 0.273               | 0.317               |
| Static | DeepConvNet    | Intra-Session   | 0.296               | 0.249               |
|        |                | Inter-Session   | 0.232               | 0.260               |
|        |                | Inter-Users     | 0.220               | 0.269               |
|        |                | Continuous      | 0.205               | 0.310               |
|        | EEGNet         | Intra-Session   | 0.242               | 0.263               |
|        |                | Inter-Session   | 0.189               | 0.231               |
|        |                | Inter-Users     | 0.184               | 0.299               |
|        |                | Continuous      | 0.194               | 0.282               |
|        | ShallowConvNet | Intra-Session   | 0.203               | 0.209               |
|        |                | Inter-Session   | 0.191               | 0.206               |
|        |                | Inter-Users     | 0.180               | 0.218               |
|        |                | Continuous      | 0.180               | 0.221               |

Tabla B.2: Tasas de falsos positivos y falsos negativos para cada combinación de modelo, método de entrenamiento y tarea objetivo. Los valores más altos se muestran en tonos rojizos, mientras que los valores más bajos se muestran en tonos verdosos.

### B.2. Precisión media y desviación estandar de cada usuario

En la Tabla B.3 se muestran las precisiones medias y desviaciones estándar de cada usuario separadas por tarea objetivo. La ultima fila agrupa las precisiones medias y desviaciones estándar de todos los usuarios.

| user       | task_mode | mu                 | sd                  |
|------------|-----------|--------------------|---------------------|
| 0          | motion    | 0.6460258233       | 0.08461602092       |
|            | static    | 0.7074175824       | 0.07006995298       |
| 1          | motion    | 0.7646520147       | 0.142811407         |
|            | static    | 0.8853010116       | 0.07966058612       |
| 2          | motion    | 0.6313841599       | 0.11436005          |
|            | static    | 0.600093985        | 0.1028584688        |
| 3          | motion    | 0.6465894466       | 0.1249193032        |
|            | static    | 0.8550125313       | 0.07263570034       |
| 4          | motion    | 0.8097168597       | 0.1225040144        |
|            | static    | 0.8335526316       | 0.09043700305       |
| <b>All</b> | motion    | 0.6988864889021312 | 0.13932102491049542 |
|            | static    | 0.7762575802763694 | 0.13573189082767018 |

Tabla B.3: Precisión media y desviación estándar por usuario y tarea.

### B.3. Precisión media separada por tarea, método y modelo

| Método Entrenamiento | Modelo         | Estática | Movimiento | Diferencia |
|----------------------|----------------|----------|------------|------------|
| Intra-Sesión         | DeepConvNet    | 0.70     | 0.63       | 0.07       |
|                      | EEGNet         | 0.70     | 0.68       | 0.02       |
|                      | ShallowConvNet | 0.79     | 0.73       | 0.06       |
| Inter-Sesión         | DeepConvNet    | 0.75     | 0.67       | 0.08       |
|                      | EEGNet         | 0.79     | 0.72       | 0.07       |
|                      | ShallowConvNet | 0.80     | 0.74       | 0.06       |
| Inter-Usuarios       | DeepConvNet    | 0.76     | 0.66       | 0.10       |
|                      | EEGNet         | 0.76     | 0.68       | 0.09       |
|                      | ShallowConvNet | 0.81     | 0.73       | 0.07       |
| Continuo             | DeepConvNet    | 0.74     | 0.64       | 0.10       |
|                      | EEGNet         | 0.77     | 0.68       | 0.10       |
|                      | ShallowConvNet | 0.79     | 0.70       | 0.09       |
| Diferencia Media     |                |          |            | 0.07       |

### B.4. Evolución de la precisión por días separada por modelos

Las figuras B.2, B.3 y B.4 muestran la evolución de la precisión por usuario y día para cada arquitectura.

Se pueden observar diferencias significativas en la evolución de la precisión entre los distintos usuarios y días, lo que indica que el rendimiento del modelo puede variar considerablemente dependiendo de los datos de entrenamiento y las condiciones específicas de cada usuario. Estas variaciones pueden deberse a factores individuales,

como la calidad de la señal EEG, la fatiga del usuario o la familiaridad con la tarea, así como a factores externos, como el entorno de grabación o el equipo utilizado.

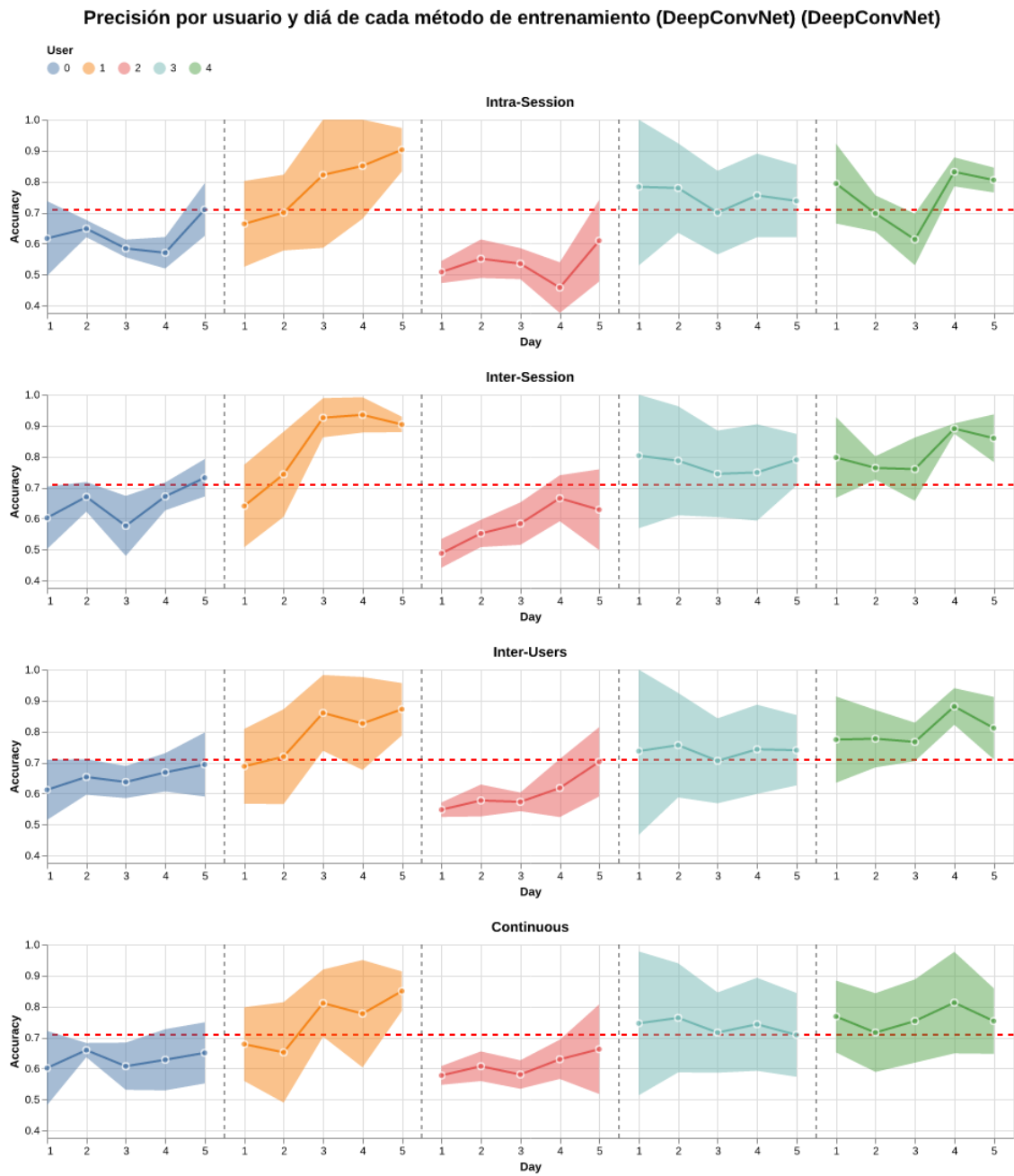


Figura B.2: Variabilidad entre usuarios y día para cada método de entrenamiento para DeepConvNet.



Figura B.3: Variabilidad entre usuarios y día para cada método de entrenamiento para ShallowConvNet.



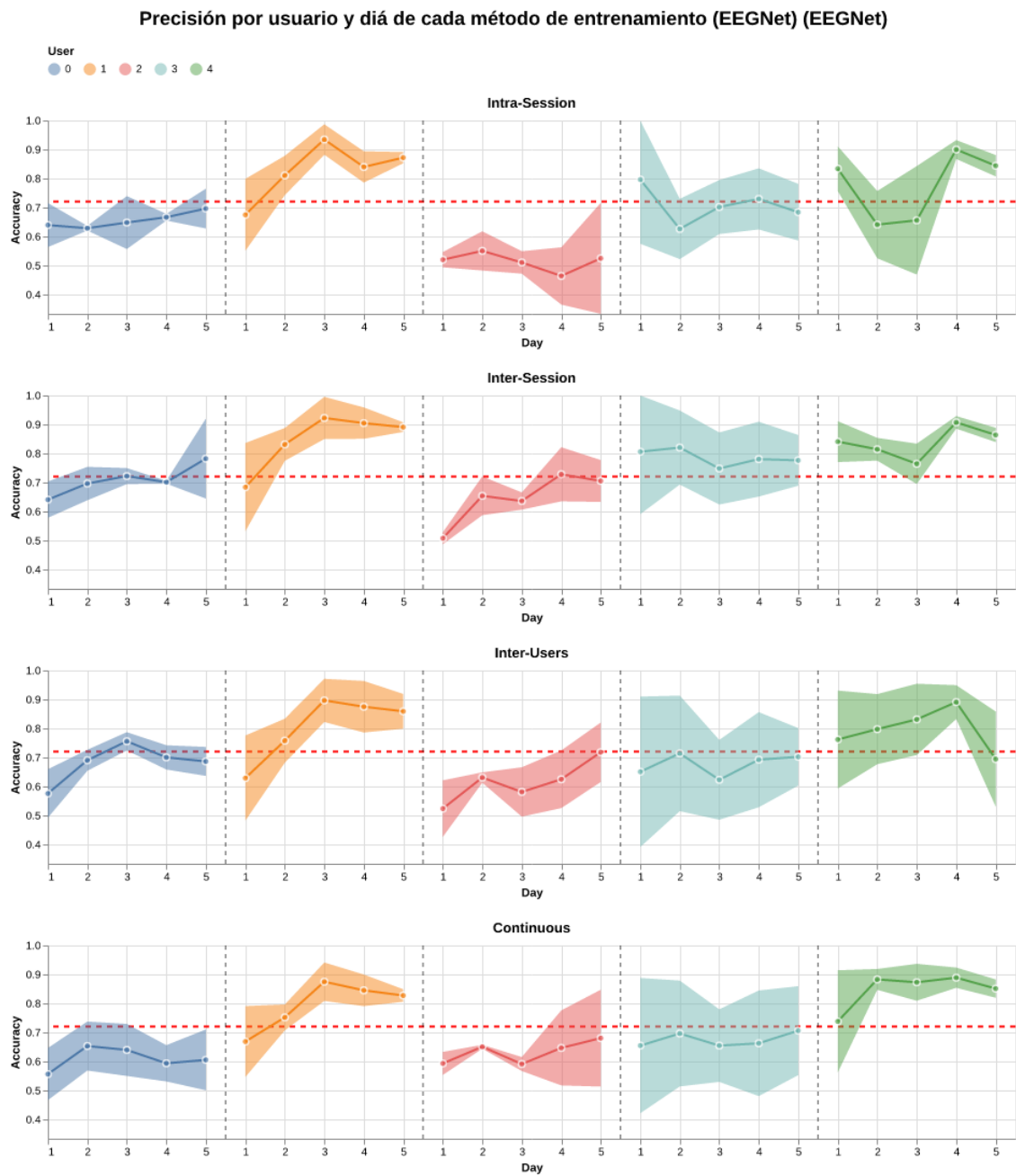


Figura B.4: Variabilidad entre usuarios y día para cada método de entrenamiento para EEGNet.

## C. Queries complementarias

Con el objetivo de facilitar la reproducibilidad del análisis, el repositorio asociado al trabajo contiene las consultas SQL empleadas para obtener las tablas agregadas y generar las figuras presentadas en la memoria. En este anexo se incluyen las consultas principales, mientras que el conjunto completo puede consultarse en el directorio correspondiente del repositorio.

### C.1. Filtrado de Outliers

Para realizar el filtrado de outliers utilizando el método Z-Score, se han definido nuevas vistas sobre cada una de las tablas **Runs\_t**, **Trials\_t** y **Classes\_t**. Manteniendo la estructura, pero eliminando valores atípicos. Además, se añaden los campos referentes a la configuración utilizada para el entrenamiento, de manera que se mantiene todo en la misma tabla.

Las queries utilizadas para la creación de las nuevas tablas “limpias” son las siguientes:

```
1 CREATE
2 OR REPLACE VIEW runs_clean_z AS
3 WITH
4     task_statistics AS (
5         SELECT
6             task_mode ,
7             AVG(acc1) AS mean_acc1 ,
8             STDDEV_SAMP(acc1) AS std_acc1
9         FROM
10             runs_t
11         WHERE
12             acc1 IS NOT NULL
13         GROUP BY
14             task_mode
15     )
16 SELECT
17     t.*,
```

```
18     p.*,
19     (t.acc1 - s.mean_acc1) / NULLIF(s.std_acc1, 0) AS z_score
20 FROM
21     runs_t AS t
22     JOIN params_t AS p ON t.run_id = p.run_id
23     JOIN task_statistics AS s ON t.task_mode = s.task_mode
24 WHERE
25     t.acc1 IS NOT NULL
26     AND t.acc1 <= 1
27     AND ABS((t.acc1 - s.mean_acc1) / NULLIF(s.std_acc1, 0)) <= 3;
28
29 CREATE
30 OR REPLACE VIEW trials_clean_z AS
31 WITH
32     task_statistics AS (
33         SELECT
34             task_mode,
35             AVG(acc1) AS mean_acc1,
36             STDDEV_SAMP(acc1) AS std_acc1
37         FROM
38             trials_t
39         WHERE
40             acc1 IS NOT NULL
41         GROUP BY
42             task_mode
43     )
44 SELECT
45     t.*,
46     p.*,
47     (t.acc1 - s.mean_acc1) / NULLIF(s.std_acc1, 0) AS z_score
48 FROM
49     trials_t AS t
50     JOIN params_t AS p ON t.run_id = p.run_id
51     JOIN task_statistics AS s ON t.task_mode = s.task_mode
52 WHERE
53     t.acc1 IS NOT NULL
54     AND t.acc1 <= 1
55     AND ABS((t.acc1 - s.mean_acc1) / NULLIF(s.std_acc1, 0)) <= 3;
56
```

```
57 CREATE
58 OR REPLACE VIEW classes_clean_z AS
59 WITH
60     task_statistics AS (
61         SELECT
62             task_mode,
63             AVG(precision) AS mean_acc1,
64             STDDEV_SAMP(precision) AS std_acc1
65         FROM
66             classes_t
67         WHERE
68             precision IS NOT NULL
69         GROUP BY
70             task_mode
71     )
72 SELECT
73     t.*,
74     p.*,
75     (t.precision - s.mean_acc1) / NULLIF(s.std_acc1, 0) AS z_score
76 FROM
77     classes_t AS t
78     JOIN params_t AS p ON t.run_id = p.run_id
79     JOIN task_statistics AS s ON t.task_mode = s.task_mode
80 WHERE
81     t.precision IS NOT NULL
82     AND t.precision <= 1
83     AND ABS(
84         (t.precision - s.mean_acc1) / NULLIF(s.std_acc1, 0)
85     ) <= 3;
```

Listing 1: Queries para la creación de las tablas “limpias”.